

The Complete JSCC Companion

All Seven Guides, One Volume

September 2026

How to use this volume

This volume collects, unchanged, the seven study guides written for a low-delay, robust, channel-optimized image-coding (JSCC) project, and binds them into a single narrative. The guides themselves are frozen: each keeps its original pagination, its original section numbers, and its original cross-references, exactly as released. Everything *around* them is new: this front matter, a corrections section you should read before the guides themselves, one-page divider sheets that bridge each part to the next, four new supplement chapters filling genuine gaps the audit found, and a topic index at the back.

The parts are ordered the way the material is most naturally learned. **Part I** (background handbook) refreshes the prerequisites and maps the whole project, so skim it first and return to it whenever a chapter assumes something you have forgotten. **Part II** is the core: joint source-channel coding theory, from quantization through channels with memory to channel-optimized vector quantization and a complete image-coder design. **Part III** re-derives every formula Part II leans on, so read the two interleaved, one section at a time. **Part IV** is the other half of the channel story: error-correcting codes, from finite fields to Reed-Solomon decoding and modern iterative codes. **Part V** turns theory into real codecs (JPEG, PNG, and friends), **Part VI** walks one small image through the entire pipeline with every number shown, and **Part VII** covers the machine-learning extension: deep learning for classifying the decoded images. **Part VIII** is new material written for this volume: modulation and soft decisions, fitting burst models to data, statistics for experiments, and deployment tradeoffs.

You do not need to read cover to cover. The intended loop is: attempt the project task, hit a wall, find the wall in the topic index, read the indicated section, and return to work. The divider page before each part tells you what that part covers, how it connects to what came before, and what to watch for while reading.

Assumed background map

This volume assumes two things and refreshes everything else. Assumed: single- and multivariable calculus (differentiation, the chain rule, basic integration), and working fluency in Python with NumPy (vectors, broadcasting, random seeds). Pointers to standard external references for these: any mainstream calculus text, and the NumPy user guide.

Everything else the project needs is refreshed *inside* the volume, in these places:

- **Probability** (random variables, Bayes, MMSE estimation): Part I, §2; used constantly in Parts II–IV.
- **Linear algebra** (eigen-decomposition, orthogonal transforms): Part I, §11; the GF(2) null-space picture additionally in Part IV, §4.

- **Optimization** (Lagrange multipliers, alternating minimization, convexity): Part I, §10; the engine behind Lloyd, LBG, COVQ, and EM.
- **Modular arithmetic and finite fields**: Part IV, §§2–3, built from scratch with a worked GF(16).
- **Information theory**: Part I, §3 for the essentials; full channel-model theory in Part II.
- **Machine learning**: Part I, §13 for the classification setup; the complete treatment is Part VII.
- **Statistics for experiments** (confidence intervals, comparing systems): new in Part VIII, Chapter S3.

Corrections and clarifications

An independent audit of the seven frozen guides found the following errors. The guides themselves are unchanged; read each item here with the corrected version before the corresponding section.

1. **Plotkin bound, asymptotic form (Part IV, asymptotic bounds).** The guide states the threshold as $\delta > 1/4$; the correct binary statement is: if the relative distance satisfies $\delta > 1/2$, then the asymptotic rate is forced to $R = 0$. The finite-length form in the guide ($d > n/2$ forces a bounded code size) is correct. Intuition: two binary codewords at relative distance δ agree on a $1 - \delta$ fraction of positions; once $\delta > 1/2$ the pigeonhole argument behind Plotkin’s proof bites.
2. **Capacity formula $C = 1 - \bar{H}(Z)$ (Part II).** For binary additive noise this formula requires the noise process to be stationary *and* ergodic (information-stable). For stationary but *nonergodic* noise it is only an *upper* bound; achievability needs information-spectrum / worst-component reasoning. This is exactly why the Pólya channel — stationary, nonergodic — has capacity 0 even though the formula would suggest otherwise. Read “ $C = 1 - \bar{H}(Z)$ ” everywhere as carrying the ergodicity hypothesis.
3. **JPEG quality-10 DC levels (Part V).** With DC quantization step 80 over the orthonormal 8×8 DCT DC range of roughly -1024 to 1016 , there are about $(1016 - (-1024))/80 \approx 26$ distinct DC levels — roughly 10 units of block-mean intensity each — not “ ~ 3 ” as stated.
4. **Hölder equality step (Part III, $f^{1/3}$ derivation).** From $u^3 = f\lambda^{-2}$ and $v^{3/2} = \lambda$, the Hölder equality condition requires $f\lambda^{-2} \propto \lambda$ (not $\propto \lambda^2$). The final result $\lambda \propto f^{1/3}$ is unchanged.
5. **GF(16) product display (Part IV, §3.1).** The product $0xb \times 0x6 = 0xf$ is correct, but the displayed intermediate expansion is wrong. The correct chain, with primitive polynomial $x^4 + x + 1$ (so $x^4 = x + 1$, $x^5 = x^2 + x$):
$$(x^3 + x + 1)(x^2 + x) = x^5 + x^4 + x^3 + x = (x^2 + x) + (x + 1) + x^3 + x = x^3 + x^2 + x + 1 = 1111_2 = 0xf.$$
6. **Missing blackboard-bold Z (Part IV).** The macro `\Z` was never defined, so $\mathbb{Z}$ renders as blank: read “form the ring $[blank]_n$ ” as $\mathbb{Z}_n$ and “6 has zero divisors” as referring to $\mathbb{Z}_6$.
7. **Pillow optimize default (Part V).** `optimize` defaults to `False`; pass `optimize=True` to obtain optimized Huffman tables. The guide’s claim about the default is wrong.
8. **JPEG Annex K tables (Part V).** The Annex K Huffman/quantization tables are *informative examples*, not decoder defaults; a real interchange stream must define its tables via DHT/DQT segments.
9. **PNG filter selection (Part V).** “PNG tries all five filters per row” describes one common encoder heuristic, not a format requirement; the format only requires the chosen filter byte per scanline.
10. **Stale cross-references (Part I).** The pointer “G5 §17” should read “the ECC guide’s unnumbered formula appendix (after §16)”; references to “five guides” predate the final two guides — there are seven.
11. **SmallCNN input size (Part VII, Ch. 8).** `Linear(32*8*8, 128)` assumes a 32×32 input; do not combine it literally with the `RandomResizedCrop(224)` example without resizing the Linear layer. The “ ~ 70 – 75% with the small CNN” remark refers to training the Ch. 8 SmallCNN; Appendix B contains the separate ResNet-18 transfer-learning script.

Contents

How to use this volume	1
Assumed background map	1
Corrections and clarifications	3
 Part I — Background Theory Handbook	 5
 Part II — Joint Source–Channel Coding: The Core Theory	 21
 Part III — Formula Companion: Every Derivation Earned	 48
 Part IV — Error-Correcting Codes: The Mathematics of Protection	 65
 Part V — Real Codecs: How Images Are Actually Compressed	 83
 Part VI — One Image, Every Number: A Complete Numerical Walk-through	 97
 Part VII — Deep Learning for Image Classification	 108
 Part VIII — Supplement: Four Gap-Fill Chapters	 127
1 Supplement S1: From bits to waveforms and back	128
2 Supplement S2: Fitting burst models to measured data	129
3 Supplement S3: Statistics for experiments	130
4 Supplement S4: Deployment tradeoffs	130
Where to find what: topic index	133

Part I — Background Theory Handbook

`project-background.pdf` (15 pages)

Start here. This part is the reference shelf for the entire volume: fifteen tight chapters covering probability, information theory, rate–distortion, source coding, channel coding, channels with memory, JSCC, deep-learning JSCC, optimization, linear algebra, programming practice, ML classification, wireless basics, and evaluation methodology — each ending with the project relevance, the common pitfalls, and pointers into the deeper guides. Chapter 16 maps the project description’s phases onto a study plan, so if you read nothing else first, read that. Afterwards, use this part the way a textbook index is used: when a later part assumes something you have forgotten, come back and look it up. Watch for the “why this project needs it” paragraphs — they are the fastest way to connect abstract theory to the system you are building.

Background Theory for a JSCC Image-Coding Project

A Thorough Reference Companion

Companion to the five deep-dive guides on joint source–channel coding

September 2026

Abstract

This handbook collects, in one place, the background theory behind a project on low-delay, robust, channel-optimized image coding (joint source–channel coding, JSCC) over discrete channels with memory, burst noise, and erasures — including the optional deep-learning JSCC extension and the final image-classification component. It is a *reference*, not a textbook: each chapter states what each area is, lists its key formulas and results cleanly, explains why the project needs it, and flags the mistakes students most often make. Full derivations live in the five companion guides; every chapter ends with pointers to the exact sections that contain them. When you forget something mid-project, start here.

Contents

1	How to use this handbook	3
2	Probability refresher	3
3	Information theory essentials	4
4	Rate–distortion theory	5
5	Source coding	6
6	Channel coding essentials	7
7	Channel models with memory	7
8	Joint source–channel coding	8
9	Deep-learning-based JSCC (optional extension)	9
10	Optimization techniques	10
11	Linear algebra and calculus refresher	10
12	Programming for this project	11
13	Machine learning for image classification	11
14	Wireless communications basics	12
15	Evaluation methodology	13
16	Project roadmap and study plan	13

A	Notation and symbol glossary	14
B	Formulas at a glance	14

1 How to use this handbook

The five deep guides are: **G1** *Joint Source–Channel Coding for Image Transmission over Noisy Channels with Memory* (the theory core); **G2** *The Formulas Behind JSCC* (derivations with visualizations); **G3** *From Pixels to Bits to Pixels* (practical codecs); **G4** *One Small Image, End to End* (the numerical walkthrough + code); **G5** *Error-Correcting Codes* (ECC mathematics). This handbook (**G6**) sits above them: broad coverage, shallow depth, complete indexing.

Each chapter follows the same template: **what it is** (2–5 tight paragraphs), **key results** (formulas stated, not proved), **why this project needs it** (tied to a concrete project task), **pitfalls** (what students mix up). Chapter 15 maps the whole project onto this handbook and the five guides as a study plan.

2 Probability refresher

What it is. A **random variable** X is a number produced by a random experiment; its **distribution** is described by a density $p_X(x)$ (continuous) or mass function (discrete). The **expectation** $\mathbb{E}[g(X)] = \int g(x)p_X(x) dx$ is the probability-weighted average — every distortion, rate, and capacity in this project is an expectation. **Conditional probability** $\mathbb{P}(A | B) = \mathbb{P}(A, B)/\mathbb{P}(B)$ and **Bayes’ rule** $p_{X|Y}(x | y) = p_{Y|X}(y | x)p_X(x)/p_Y(y)$ let you invert observations into beliefs about causes; the COVQ decoder is literally Bayes’ rule in action. Two variables are **independent** if $p_{X,Y} = p_X p_Y$; otherwise their **covariance** $\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$ and **correlation** $\rho = \text{Cov}/(\sigma_X \sigma_Y)$ measure linear dependence. The **law of large numbers** says sample averages converge to expectations (why Monte Carlo simulation works); the **central limit theorem** says sums of many small independent effects look Gaussian (why Gaussian models are everywhere).

Key results. **MMSE estimation:** for squared error, the best estimate of X given observation Y is the **conditional mean** $\hat{X} = \mathbb{E}[X | Y]$, achieving $\text{MMSE} = \mathbb{E}[\text{Var}(X | Y)]$. Every optimal decoder in these guides is a conditional mean. **Common distributions** (all shown in Figure 1):

Distribution	Density	Where it appears
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$	DCT DC coefficients, thermal noise, CLT limit
Laplacian	$\frac{\lambda}{2} e^{-\lambda x-\mu }$	DCT AC coefficients (heavy tails near zero)
Uniform $[a, b]$	$1/(b-a)$	Quantization error model, dithering
Bernoulli(p) Binomial	$p^k(1-p)^{n-k}$	Bit errors, error counts in a block

Why this project needs it. You will fit Gaussian/Laplacian models to DCT coefficients, compute posterior means in the COVQ decoder, simulate Bernoulli bit errors, and average everything by Monte Carlo. The conditional mean is the single most-used formula in the project.

Mini-example (Bayes). A bit is sent over a BSC with crossover 0.1; you receive 1. A priori the bit was equally likely 0 or 1. Then $\mathbb{P}(X = 0 | Y = 1) = \frac{0.1 \cdot 0.5}{0.5} = 0.1$ — the posterior just restates the channel. If instead the prior were $\mathbb{P}(X = 0) = 0.9$ (zeros dominate), the posterior becomes $\frac{0.1 \cdot 0.9}{0.1 \cdot 0.9 + 0.9 \cdot 0.1} = 0.5$: prior and likelihood fight to a draw. COVQ decoders do exactly this computation, with codevectors instead of bits.

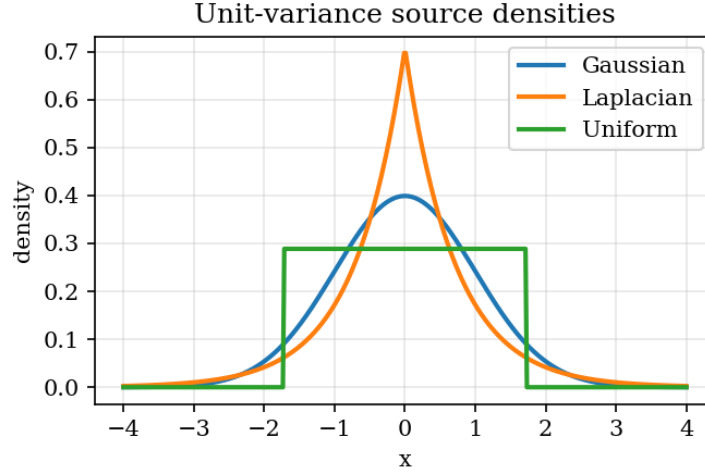


Figure 1: Unit-variance source densities: Gaussian (DCT DC coefficients), Laplacian (DCT AC coefficients), uniform (quantization-noise model).

Pitfalls. $\mathbb{E}[X | Y]$ is a *random variable* (a function of Y), not a number — students treat it as a constant. Uncorrelated ($\rho = 0$) does **not** imply independent (only for jointly Gaussian). The Laplacian is *not* “almost Gaussian”: its sharp peak at zero is exactly why AC coefficients quantize the way they do.

Deeper: G1 §2 (Mathematical preliminaries) for the full toolkit; G2 §8 for the conditional mean worked out in the 1-bit COVQ example.

3 Information theory essentials

What it is. **Entropy** $H(X) = -\sum_x p(x) \log_2 p(x)$ is the average surprise of X in bits — the fundamental unit of information. **Joint entropy** $H(X, Y)$ and **conditional entropy** $H(X | Y) = H(X, Y) - H(Y)$ measure remaining uncertainty after observing Y . **Mutual information** $I(X; Y) = H(X) - H(X | Y)$ is how much Y tells you about X . **KL divergence** $D(p||q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} \geq 0$ measures how far distribution q is from the truth p — it is the price of a wrong model, and appears when fitting channel models. For processes with memory, the **entropy rate** $\bar{H} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n)$ is the per-symbol uncertainty; memory *reduces* it, which is why burst models matter.

Key results. **Source coding theorem:** n symbols from a source with entropy rate $\bar{H}$ can be compressed to $n\bar{H}$ bits, and no further — entropy is the lossless limit. **Channel coding theorem:** reliable communication is possible iff the rate $R < C$, where **capacity** $C = \max_{p_X} I(X; Y)$. Closed forms you will use:

$$\text{BSC: } C = 1 - h_b(p), \quad \text{BEC: } C = 1 - \varepsilon, \quad h_b(p) = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad (1)$$

and for stationary binary additive noise, $C = 1 - \bar{H}(Z)$ (Figure 2). **Separation theorem:** for a memoryless channel, separate source and channel coding is optimal *as block length* $\rightarrow \infty$. It breaks the moment delay, complexity, or block length is finite — or the channel has memory that an interleaver cannot cheaply erase. That failure is the entire reason this project exists.

Why this project needs it. Capacity tells you what rate is even possible on your channel; entropy tells you what rate your source demands; separation’s failure at finite delay is the

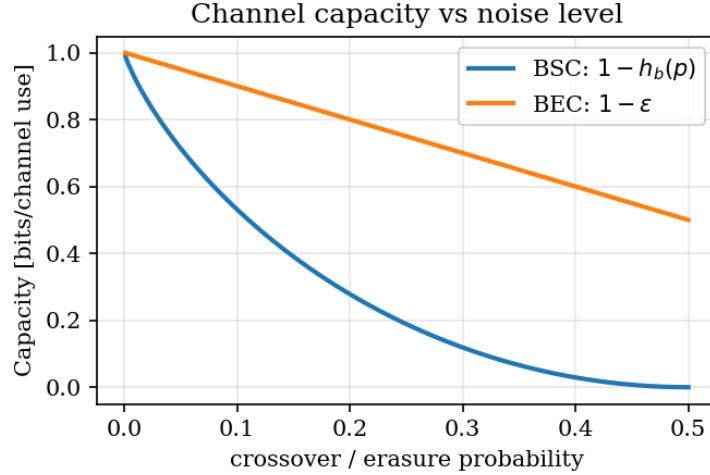


Figure 2: BSC and BEC capacity vs. noise level: every tenth of crossover probability costs real, computable rate.

project’s founding argument; KL divergence is how you will fit burst models to measured error traces.

Pitfalls. Capacity is a *rate* (bits per channel use), not a distortion — it says nothing about image quality directly. $I(X;Y)$ is symmetric but $D(p||q)$ is not. Students often quote “ $C = 1 - h_b(p)$ ” for channels with memory, where it is false — memory needs the entropy-rate form. *Deeper:* G1 §2 for entropy/rate definitions; G2 §10 for the $C = 1 - \bar{H}(Z)$ and NEC derivations; G1 §6 for channels with memory.

4 Rate–distortion theory

What it is. Lossy compression trades **rate** R (bits per sample) against **distortion** D (e.g. MSE). The **rate–distortion function** $R(D) = \min_{p_{\hat{X}|X}: \mathbb{E}[d(X,\hat{X})] \leq D} I(X;\hat{X})$ is the smallest rate any lossy code can achieve at distortion D — the lossy analogue of entropy. It is convex, decreasing, and $R(0) = H(X)$ for discrete sources. Its operational meaning: no code beats it, and (asymptotically) codes exist that attain it. Every quantizer you design lives somewhere above this curve; the gap measures your inefficiency.

Key results. For a Gaussian source with variance σ^2 under squared error,

$$R(D) = \begin{cases} \frac{1}{2} \log_2 \frac{\sigma^2}{D}, & 0 < D \leq \sigma^2, \\ 0, & D \geq \sigma^2, \end{cases} \quad (2)$$

(Figure 3): each halving of distortion costs exactly $\frac{1}{2}$ bit — the rate-domain twin of the 6 dB/bit rule. Above σ^2 , zero rate suffices (just output the mean). The **water-filling** form generalizes this to parallel Gaussian sources: give bits only to components whose variance exceeds the “water level” — the continuous version of bit allocation.

Why this project needs it. $R(D)$ is the yardstick: your COVQ system’s PSNR-vs-rate curve should be compared against it, and the gap to $R(D)$ tells you whether to improve the quantizer or accept the channel’s verdict.

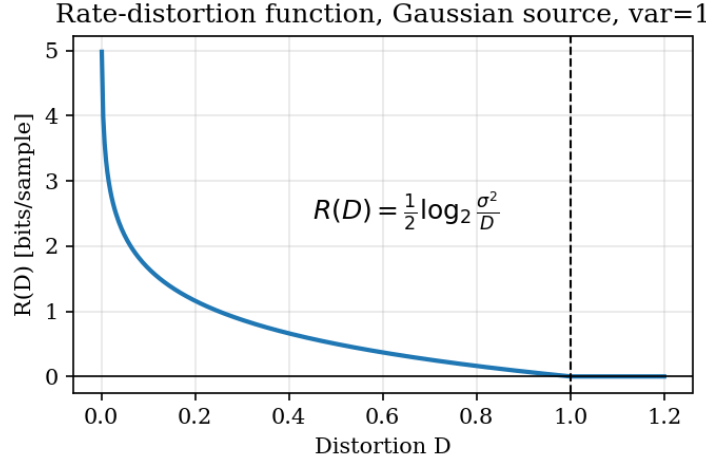


Figure 3: Gaussian rate-distortion function ($\sigma^2 = 1$): halving D always costs $\frac{1}{2}$ bit. Real quantizers sit above this curve.

Pitfalls. $R(D)$ is asymptotic (infinite block length) — a scalar quantizer at low rates sits well above it; that gap is normal, not a bug. Do not confuse $R(D)$ (lossy, distortion-constrained) with entropy (lossless).

Deeper: G1 §§3–5 for the quantizers whose gap to $R(D)$ you will measure; G2 §4 for the 6 dB/bit connection.

5 Source coding

What it is. Source coding removes redundancy from the signal. **Quantization** maps continuous amplitudes to a finite set: a **scalar quantizer** partitions $\mathbb{R}$ into cells with a reconstruction level each; a **vector quantizer** does the same in $\mathbb{R}^k$, exploiting correlation and geometry for extra gain. **Transform coding** first rotates the signal (DCT/KLT) to concentrate energy into few coefficients, then quantizes those. **Entropy coding** (Huffman, arithmetic) losslessly compresses the quantizer's output symbols down toward their entropy. **Predictive coding** sends only the unpredictable residual (JPEG's DC differences, PNG's filters).

Key results. **Lloyd–Max conditions** (scalar, MSE): (i) each cell is the nearest-neighbor region of its level; (ii) each level is the centroid $\mathbb{E}[X \mid X \in \text{cell}]$. Alternating the two (Lloyd's algorithm) converges to a local optimum. High-resolution formulas: uniform quantizer $D \approx \Delta^2/12$; optimal point density $\propto f(x)^{1/3}$; distortion $D^*(N) \approx \frac{1}{12N^2} (\int f^{1/3})^3$; **6 dB per bit**. **Bit allocation** across transform coefficients: $b_i = \bar{b} + \frac{1}{2} \log_2(\sigma_i^2/\rho^2) + \frac{1}{2} \log_2(h_i/H)$ (Lagrangian; round to integers greedily). **KLT** decorrelates and is high-resolution optimal; the **DCT** is its cheap fixed substitute. **Huffman coding** assigns shorter codewords to likelier symbols and is optimal among prefix codes (within 1 bit of entropy). **Arithmetic coding** reaches fractional bits per symbol by encoding the whole message as one interval — better for skewed distributions, at more complexity.

Why this project needs it. The project's source coder is a bank of channel-optimized quantizers on DCT coefficients with fixed bit allocation — every one of these results is a component you will implement or a baseline you will beat.

Pitfalls. Lloyd’s algorithm finds *local* optima — always try several initializations. Huffman is optimal per-symbol, not per-message (arithmetic wins on skewed binary alphabets). The DCT is not the KLT; it is an approximation that happens to be excellent for natural images.

Mini-example (Huffman). Symbols $\{a, b, c, d\}$ with probabilities $(0.5, 0.25, 0.125, 0.125)$: Huffman gives $a \rightarrow 0, b \rightarrow 10, c \rightarrow 110, d \rightarrow 111$, average length 1.75 bits vs. entropy 1.75 bits — optimal here. Note the code is a prefix code: no codeword is a prefix of another, so decoding needs no separators.

Deeper: G1 §§3–5 for the full theory; G2 §§1–6 for every derivation; G3 §3 for JPEG’s real pipeline; G4 for the numbers.

6 Channel coding essentials

What it is. Channel coding adds structured redundancy so the decoder can detect and correct channel damage. It **costs** rate (a rate- k/n code sends $n - k$ parity symbols per k data symbols) and delay (block codes wait for the whole block; interleavers wait longer), and **buys** reliability — the classic cliff: below the code’s threshold, near-perfect; above it, collapse. **Hamming codes** correct one error per block via syndrome lookup. **Reed–Solomon** codes work on symbol alphabets (e.g. bytes), meet the Singleton bound ($d_{\min} = n - k + 1$, MDS), and are the canonical *burst-error* and *erasure* codes — CDs, QR codes, deep-space links. **Convolutional codes** add memory to the encoder and decode with the **Viterbi algorithm** (shortest path through the trellis). **LDPC** and **turbo** codes iterate simple local decoding steps and approach capacity; **polar** codes provably achieve it. **Fountain codes** (LT, Raptor) are rateless: they fix *erasures* (vanished packets), not errors.

Key results. A code with minimum distance $d_{\min}$ corrects any pattern of t errors and e erasures with

$$2t + e + 1 \leq d_{\min} \quad (3)$$

— an erasure costs *half* an error, because its location is free information. **Interleaving** spreads burst errors across many codewords so a random-error code can handle them, at the price of delay and memory. Fundamental bounds (Hamming, Singleton, Plotkin, Gilbert–Varshamov) bracket what any code can do; Shannon capacity says what exists asymptotically.

Why this project needs it. Channel coding is the **baseline you compare against**: the project asks you to beat (or match, at lower complexity/delay) tandem schemes like “VQ + Reed–Solomon + interleaver.” You must understand what you are replacing to argue the replacement is worthwhile.

Pitfalls. Coding gain is measured at *matched total rate* — comparing a rate-1/2 coded system against an uncoded one at the same source rate is cheating. Interleaving delay is often the real reason a tandem scheme is impractical, not its complexity. Erasures are *not* “half as bad” in every sense — the $2t + e + 1$ rule is about guaranteed correction, not typical performance.

Deeper: G5 for the complete ECC treatment: §6 Hamming, §12 Reed–Solomon, §13 Viterbi, §14 LDPC/turbo/polar, §15 fountain codes.

7 Channel models with memory

What it is. Real wireless channels have **memory**: errors arrive in bursts during fades. The **BSC** (each bit flipped w.p. p) and **BEC** (each bit erased w.p. ε) are the memoryless baselines. The **Gilbert–Elliott** model is a two-state Markov chain (good/bad states, different error rates) — the simplest burst model. The **binary Markov** noise model generalizes it and is what

the project’s DCT coder is trained on. The **Pólya-contagion channel** models errors as an urn process where each error breeds more errors: stationary, exchangeable, but **nonergodic** — each transmission lot has its own beta-drawn crossover, so the ordinary capacity is *zero*. The **finite-memory** version is ergodic with closed-form capacity. **Queue-based channels** give transparent block probabilities and computable capacity $C = 1 - \bar{H}(Z)$, and can be fitted to measured burst traces. The **noise-erasure channel** adds erasures to additive noise; feedback does not raise its capacity (but helps under cost constraints).

Key results.

Model	Memory mechanism	Signature result
BSC / BEC	none	$C = 1 - h_b(p) / 1 - \varepsilon$
Gilbert–Elliott	2-state Markov	burst-error baseline
Binary Markov	2-state noise chain	project’s training channel
Pólya	urn contagion	nonergodic: $C = 0$; ML is nearest- <i>or</i> -farthest
Finite-memory Pólya	M -state urn	ergodic; $C_M \uparrow$ in M
Queue-based (QBC)	finite queue + urn	$C = 1 - \bar{H}(Z)$; fittable
Noise-erasure (NEC)	noise + erasures	$C = (1 - \varepsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})]$; $C_{\text{FB}} = C$

Why this project needs it. The project brief *specifies* the channel: discrete, with memory, burst noise, and erasures. You will simulate it, train your quantizers on its transition matrix $P(j | i)$, and test mismatch when the real channel drifts from the model.

Pitfalls. “Memory” and “correlation” are not interchangeable: a Markov model specifies *conditional* dynamics, correlation only a number. Nonergodicity (Pólya) is the subtle one — time averages do not converge to ensemble averages, so a single long simulation can mislead. Always simulate channels from the *stationary* distribution unless you want transients.

Deeper: G1 §6 for the full model zoo; G2 §§9–10 for the Pólya and capacity derivations.

8 Joint source–channel coding

What it is. **Tandem** (separate) coding chains a source coder, a channel coder, and usually an interleaver. **Joint source–channel coding** designs one system for the end-to-end objective directly. The flagship here is **channel-optimized quantization (COVQ)**: choose the encoder partition and decoder codebook to minimize

$$D = \sum_i \int_{\mathcal{S}_i} p(\mathbf{x}) \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 d\mathbf{x}, \quad (4)$$

averaging over *both* source and channel. The optimal encoder picks the index minimizing channel-averaged distance; the optimal decoder outputs the posterior mean $\mathbb{E}[\mathbf{X} | J = j]$ — hedging across likely corruptions. The encoder reduces to a linear discriminant via **effective codevectors** $\tilde{\mathbf{y}}_i = \sum_j P(j | i) \mathbf{y}_j$, so runtime cost matches classical VQ. **Index assignment** (which bit-label goes on which codevector) matters: neighbors in Hamming space should be neighbors in signal space. With erasures, the decoder learns a codevector per erasure pattern — erasure flags become free soft information.

Key results. **Graceful degradation vs. cliff effect:** JSCC quality falls smoothly as the channel worsens; tandem schemes collapse at the channel code’s threshold. **Empty regions:** noisy channels can leave most encoder cells unused (61/256 in one published experiment) — the optimizer’s way of saying those codevectors aren’t worth their confusion risk. **The low-delay argument:** no interleaver, no long block code, no iterative decoder — encoding is table lookups and dot products, which is why JSCC fits sensors and real-time links. Mismatch (trained channel $\neq$ actual channel) degrades smoothly, not catastrophically.

Why this project needs it. This *is* the project: design, implement, and evaluate a channel-optimized image coder, and compare it against tandem alternatives at matched rate, delay, and complexity.

Pitfalls. COVQ codebooks are **channel-specific** — retrain (or verify robustness) when the channel changes. Do not “fix” empty encoder cells; they are the answer, not a bug. Fair comparison means matched *total* rate including parity and matched delay — the two places tandem comparisons usually cheat.

Deeper: G1 §§7–9 for the theory, geometry, and the DCT image coder; G2 §§7–8 for the derivations and worked example.

9 Deep-learning-based JSCC (optional extension)

What it is. Instead of deriving the encoder/decoder from a channel model, learn them: an **encoder neural network** maps the image to channel symbols, a (differentiable) **channel layer** injects noise, and a **decoder network** reconstructs — trained end-to-end to minimize MSE (or a perceptual loss) by gradient descent. This is an autoencoder whose bottleneck *is* the noisy channel. Variants add **learned entropy models** (a neural net predicting the probability of each latent, enabling arithmetic coding of the latents) and attention/transformer backbones. The classical COVQ lesson reappears: the learned decoder approximates a posterior mean, and the encoder learns channel-aware representations.

Key results. Recipe: encoder f_θ , channel $p(y \mid x)$ (reparameterized so gradients flow, e.g. $y = x + \sigma z$), decoder g_ϕ ; loss $\mathbb{E}\|x - g_\phi(\text{channel}(f_\theta(x)))\|^2$; train on a dataset, test on held-out images. **Finite-input/finite-output adaptation:** real channels take discrete symbols, but neural nets like continuous values — handle this with quantization-aware training (straight-through estimators), learned constellations, or a soft-to-hard annealing schedule. Fair comparison with COVQ means: *same* channel model, *same* rate (channel uses per pixel), *same* test set, *same* metric (PSNR at least, SSIM ideally).

Why this project needs it. It is the project’s optional extension: if classical COVQ works, ask whether a learned system beats it and at what complexity cost — and whether the learned encoder rediscovers the theory (it usually does).

Pitfalls. A neural JSCC trained at one SNR and tested at another can cliff worse than COVQ — test the mismatch curve, not a point. Bigger networks win by memorizing the test distribution; keep the test set strictly held out. “End-to-end learned” does not mean assumption-free: the channel layer is still a model.

Deeper: G1 §12 (Extensions) for research directions; G1 §10 for the evaluation methodology to copy.

10 Optimization techniques

What it is. Almost every algorithm in this project is **alternating minimization**: fix the decoder, optimize the encoder; fix the encoder, optimize the decoder; repeat. Lloyd’s algorithm, LBG, COVQ training, and the EM algorithm are all this pattern. The workhorse for constrained problems is the **Lagrange multiplier**: to minimize $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, solve $\nabla f = \lambda \nabla g$ — this single recipe yields the bit allocation formula and the water-filling solution. **Convexity** decides whether a stationary point is global: convex problems have one optimum; quantization problems are *not* convex, so expect local optima and restart. **Gradient descent** follows $-\nabla f$ downhill; stochastic/mini-batch versions power neural training. The **EM algorithm** alternates an expectation step (posterior over hidden variables) with a maximization step — structurally identical to the COVQ encoder/decoder alternation, with “hidden variable” = transmitted index.

Key results. Lagrange recipe: form $L = f + \lambda g$, set $\nabla L = 0$, use the constraint to fix λ . Alternating minimization decreases the objective monotonically but converges to a *local* optimum in non-convex problems — multiple initializations are mandatory, not optional. For squared-error problems the “maximization” step is usually a conditional mean in closed form.

Mini-example (bit allocation). Two coefficients, $\sigma_1^2 = 16$, $\sigma_2^2 = 1$, $B = 4$ bits, same shape factor. Geometric mean $\rho^2 = \sqrt{16 \cdot 1} = 4$, $\bar{b} = 2$: $b_1 = 2 + \frac{1}{2} \log_2(16/4) = 3$, $b_2 = 2 + \frac{1}{2} \log_2(1/4) = 1$. The high-variance coefficient steals a bit from the low-variance one; distortion $D = 2 \cdot H \cdot 4 \cdot 2^{-4}$ instead of the $16 \cdot 2^{-4} + 1 \cdot 2^{-4}$ an equal split would give.

Why this project needs it. You will derive the bit allocation, implement Lloyd/LBG/COVQ training loops, set stopping criteria, and debug non-convergence — all of which is applied optimization, not magic.

Pitfalls. “The loss went down” does not mean “it converged” — check relative improvement *and* a minimum iteration count. The most common bug in alternating schemes is a stale quantity (e.g. recomputing the partition but not the effective codevectors). Differentiating through an arg min (as in learned quantization) is not valid without a straight-through-style approximation.

Deeper: G2 §5 for the Lagrangian done fully; G1 §10 for training loop practice; G5 §13 for Viterbi as dynamic programming.

11 Linear algebra and calculus refresher

What it is. Vectors in $\mathbb{R}^k$, matrices as linear maps, inner products $\mathbf{x}^t \mathbf{y}$ and norms $\|\mathbf{x}\|^2$. An **orthogonal** matrix A ($A^t A = I$) rotates without stretching: $\|A\mathbf{x}\| = \|\mathbf{x}\|$ — which is why transform coding preserves MSE. For a covariance matrix K (symmetric positive semidefinite), the **eigen-decomposition** $K = U\Lambda U^t$ gives uncorrelated coordinates: eigenvalues λ_i are the variances along eigenvector directions $\mathbf{u}_i$, and $\text{trace } K = \sum \lambda_i$ is total energy while $\det K = \prod \lambda_i$ measures “volume.” The KLT is exactly this decomposition; its coding gain is the arithmetic/geometric mean ratio of the eigenvalues. **Matrix calculus** essentials: $\nabla_{\mathbf{x}} \|\mathbf{x}\|^2 = 2\mathbf{x}$, $\nabla_{\mathbf{x}} (\mathbf{a}^t \mathbf{x}) = \mathbf{a}$, $\nabla_X \text{trace}(X^t A X) = (A + A^t)X$ — differentiating quadratics is how every centroid/posterior-mean condition is derived.

Key results. Orthogonal transforms preserve Euclidean distance and MSE. Eigenvalues of the covariance = variances of decorrelated components; a flat spectrum means the transform buys nothing. Hadamard’s inequality ($\prod d_i \geq \det K$ for diagonal d_i) is the engine behind KLT optimality. The 2-D DCT is separable: $Y = A X A^t$.

Why this project needs it. You will read covariance eigen-spectra to judge transforms, verify DCT orthogonality numerically, and differentiate distortion objectives to check optimality conditions.

Pitfalls. Eigenvectors are only defined up to sign — sign flips between implementations are not bugs. “Orthogonal” ($A^t A = I$) vs. “orthonormal set” vs. “orthogonal vectors” get conflated; transform coding needs the matrix property. Forgetting that $\|A\mathbf{x}\| = \|\mathbf{x}\|$ is exactly what makes MSE transform-invariant.

Deeper: G1 §5 for the KLT proof; G2 §6 for Hadamard worked out.

12 Programming for this project

What it is. The working stack is **Python + NumPy/SciPy/Matplotlib**: NumPy for vectorized array math (DCTs, partitions, Monte Carlo), SciPy for optimization and special functions, Matplotlib for every plot you will put in the report. The three rules of fast NumPy: vectorize inner loops, precompute what does not change per iteration (effective codevectors!), and never grow arrays in a loop. **RNG discipline:** seed every experiment (`np.random.default_seed`), log the seed with the result, rerun critical numbers with ≥ 3 seeds and report mean $\pm$ std. **Float precision:** use float64 for training accumulations; probabilities that underflow (block likelihoods) belong in log-domain.

Key results (methodology, not theorems). **Simulation protocol:** keep training, validation, and test data separate (training images $\neq$ test images); use one shared channel code module between training and evaluation; baselines at matched rate (a source-only VQ, a tandem VQ with BCH/RS and an interleaver, and the clean-channel bound); sweep over bit-error rate and correlation, not single points; ablations per component; mismatch grids. **When to leave Python:** the encode/decode hot path in a real product wants a compiled language with SIMD and no garbage collector — prototype in Python, port the inner loop if speed matters; keep the Python as the reference implementation. **Hygiene:** version control everything, log hyperparameters with results, never tune on the test set.

Why this project needs it. The project is graded on implementation and honest evaluation as much as theory. Most failed replications fail on methodology (one seed, tuned test set, mismatched rates), not mathematics.

Pitfalls. `np.random.seed` global state vs. `default_rng` streams — pick the latter and pass generators explicitly. Integer division and dtype promotion silently corrupt fixed-point-style code. A “mismatch experiment” where train and test use different channel *code paths* measures a bug, not robustness.

Deeper: G1 §10 (Implementation blueprint) is the full checklist; G4’s `walkthrough.py` is a model reference implementation.

13 Machine learning for image classification

What it is. **Supervised learning:** given labeled examples (image, class), learn a map minimizing prediction error on *unseen* data. **CNNs stack:** *convolutional* layers (learned local filters, translation-equivariant), *pooling* (downsampling for invariance and efficiency), *fully-connected* layers, and a **softmax** output with **cross-entropy loss** $-\sum_c y_c \log \hat{y}_c$ — the workhorse classifier. Split data into **train** / **validation** / **test**; tune on validation, report once on test. **Metrics:** accuracy, **confusion matrix** (which classes confuse), **precision/recall** and **F1** (critical when

classes are imbalanced — e.g. rare forest fires). **Transfer learning:** start from a network pretrained on ImageNet and fine-tune — with small datasets this beats training from scratch. **Data augmentation** (crops, flips, noise) enlarges the effective dataset.

Key results. Softmax + cross-entropy is maximum-likelihood fitting of class probabilities; its gradient is simply $(\hat{y} - y)$. $F1 = 2PR/(P + R)$ balances precision and recall. The project’s specific question has a known-good answer pattern: **train the classifier on degraded images** (compressed + channel noise), not just clean ones — matched train/test conditions recover most of the lost accuracy, because the network learns to ignore quantization/blocking artifacts.

Why this project needs it. The final component: classify decoded images (e.g. forest-fire detection from sensor/satellite imagery) and measure how compression + channel noise move the metrics. The interesting result is not “accuracy drops” but the *curve* of accuracy vs. channel quality, and whether retraining on degraded data flattens it.

Pitfalls. Accuracy lies on imbalanced data (99% “no fire” accuracy with zero fires detected) — lead with precision/recall. Test images must be unseen *and* from the same degradation distribution you claim. Do not let the classifier see the clean originals at test time if the deployed system will not have them.

Mini-example (confusion matrix). 1000 test images, 50 real fires; the classifier flags 40 images, 30 correctly:

	Predicted fire	Predicted no fire
Actual fire	30 (TP)	20 (FN)
Actual no fire	10 (FP)	940 (TN)

Accuracy = 97%, but precision = $30/40 = 75\%$ and recall = $30/50 = 60\%$ — the numbers that matter when a missed fire costs more than a false alarm. $F1 = 2 \cdot 0.75 \cdot 0.60 / 1.35 \approx 0.67$.

Deeper: No deep guide covers CNNs (out of scope for the five) — this chapter plus a standard reference (e.g. Goodfellow–Bengio–Courville, *Deep Learning*, Ch. 9) is the intended background.

14 Wireless communications basics

What it is. A **wireless channel** turns bits into waveforms (**modulation:** e.g. QPSK maps bit pairs to four phases) and back, through a medium that fades, interferes, and adds noise. **SNR** (signal-to-noise ratio) sets the raw bit-error rate; **fading** — multipath interference varying with motion — is *why* errors come in bursts: a deep fade corrupts a run of symbols, then the channel recovers. Bits travel in **packets/frames** with headers (addresses, sequence numbers, checksums); **latency** is delay per packet, **throughput** is bits per second, and they trade against each other (long blocks raise throughput, latency with it). **Power constraints** limit transmit energy — the reason low-complexity, low-delay schemes win in sensors and phones.

Key results. Fading $\rightarrow$ bursty errors $\rightarrow$ the Markov/Pólya models of Chapter 6: the channel model is an *abstraction* of this physics. Modulation + hard decisions produce the BSC-like bit channel; soft demodulator outputs (LLRs) carry reliability information that erasure and soft decoders exploit. Shannon’s separation assumed infinite delay; wireless control loops and voice/video cannot wait.

Why this project needs it. Just enough to understand what the discrete channel model abstracts away — and to argue, in the writeup, why burst noise plus erasures is the honest model for fading packet networks.

Pitfalls. SNR in dB is logarithmic: 3 dB doubles power, 10 dB multiplies by ten. “The channel” in papers usually means the *discrete* bit-level channel after modulation/demodulation, not the waveform channel. Throughput $\neq$ goodput (headers and retransmissions eat the difference).
Deeper: G1 §6 for the discrete models; G3 §6 for packets and why one bad bit hurts.

15 Evaluation methodology

What it is. How you prove the system works. **PSNR** $= 10 \log_{10}(255^2/\text{MSE})$ is the standard fidelity metric; **SSIM** adds perceptual structure (luminance, contrast, correlation) and tracks human judgment better. The **rate–distortion curve** (PSNR vs. bits/pixel) is the real comparison object — single (rate, PSNR) points prove nothing. **Baselines, at matched rate:** source-only coder, tandem coder + channel code + interleaver, clean-channel bound. **Mismatch testing:** train at one channel, test on a grid around it, report the surface. **Statistics:** ≥ 3 seeds, mean $\pm$ std, error bars on curves.

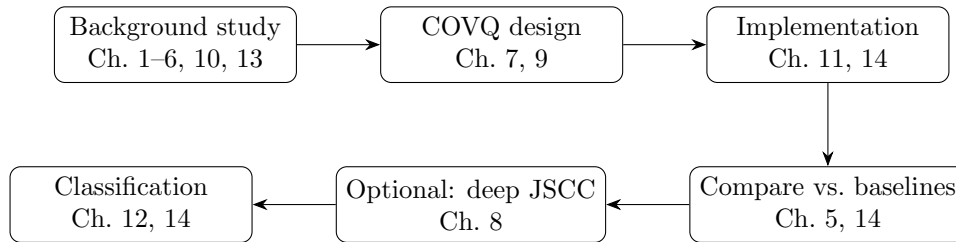
Key results. No theorem here — the result is discipline: matched total rate *including* parity bits and matched delay, held-out test data, ablations attributing each dB to a component, and mismatch surfaces instead of cherry-picked points.

Why this project needs it. “Performance and complexity will be analyzed and compared to competitive alternatives” is a direct quote from the brief. This chapter is the rubric.

Pitfalls. Comparing at matched *source* rate while the baseline spends half its bits on parity. Tuning on the test set (report the test number once). Single-seed “wins” that vanish on reseed. PSNR without SSIM on structured artifacts (blocking).

Deeper: G1 §10 for the full protocol; G4 for a worked evaluation.

16 Project roadmap and study plan



Project phase	Handbook chapters	Deep guides (in order)
Background study	1–6, 10, 13	G1 §§2–6, G2 §§1–6, G5 §§1–9
COVQ design	7, 9	G1 §§7–8, G2 §§7–8
Implementation	11, 14	G1 §§9–10, G4 (full), G3 §§2–3
Comparison vs. alternatives	5, 6, 14	G5 (ECC baselines), G1 §11
Optional deep JSCC	8, 9, 11	G1 §12; ML background Ch. 12
Classification component	12, 14	project datasets; transfer learning
Writeup	all	cheat sheets: G1 App. A, G5 §17

Minimum-viable checklists. *Before COVQ design:* state the Lloyd–Max conditions; write the end-to-end distortion (7.1); explain why the decoder is a conditional mean. *Before implementation:* run G4’s script; simulate your channel and plot a bursty trace; build $P(j | i)$ and check rows sum to 1. *Before comparison:* list baselines with matched rates; fix the metric set (PSNR + SSIM); freeze the test set. *Before classification:* baseline CNN accuracy on clean images; decide precision/recall vs. accuracy for your classes. *Before writeup:* every claim has a numbered figure or table; every number has a seed.

A Notation and symbol glossary

Symbol	Meaning
$X, \mathbf{X}$	source sample / vector; I transmitted index, J received index
$\mathcal{S}_i, \mathbf{y}_j$	encoder cell i ; decoder codevector for received j
$P(j i)$	channel transition: received j given sent i
$D, R, R(D)$	distortion; rate; rate–distortion function
$H, I(X; Y), D(p q)$	entropy; mutual information; KL divergence
$\bar{H}$	entropy rate (processes with memory)
C	channel capacity; h_b binary entropy function
$d_{\min}, t, e$	code minimum distance; correctable errors; correctable erasures
$\mathbb{E}[X Y]$	conditional mean = MMSE estimator
dB, bpp	decibels $10 \log_{10}(\cdot)$; bits per pixel

B Formulas at a glance

The twenty formulas you will reach for most often, in one place.

$$\begin{aligned}
&\text{Bayes: } p(x | y) = p(y | x)p(x)/p(y) \\
&\text{MMSE estimator: } \hat{X} = \mathbb{E}[X | Y] \\
&\text{Entropy: } H(X) = - \sum_x p(x) \log_2 p(x) \\
&\text{Mutual information: } I(X; Y) = H(X) - H(X | Y) \\
&\text{KL divergence: } D(p||q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} \\
&\text{BSC / BEC capacity: } C_{\text{BSC}} = 1 - h_b(p), \quad C_{\text{BEC}} = 1 - \varepsilon \\
&\text{Gaussian } R(D): R(D) = \frac{1}{2} \log_2(\sigma^2/D), \quad D \leq \sigma^2 \\
&\text{Lloyd–Max: cells = nearest-neighbor regions, levels} = \mathbb{E}[X | X \in \text{cell}] \\
&\text{Uniform quantizer: } D \approx \Delta^2/12, \quad 6 \text{ dB per bit} \\
&\text{Bit allocation: } b_i = \bar{b} + \frac{1}{2} \log_2(\sigma_i^2/\rho^2) + \frac{1}{2} \log_2(h_i/H) \\
&\text{COVQ encoder: } \mathcal{S}_i = \{\mathbf{x} : \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 \text{ minimal}\} \\
&\text{COVQ decoder: } \mathbf{y}_j = \mathbb{E}[\mathbf{X} | J = j] \\
&\text{ECC bound: } 2t + e + 1 \leq d_{\min} \\
&\text{PSNR: } 10 \log_{10}(255^2/\text{MSE}) \\
&\text{Cross-entropy: } L = - \sum_c y_c \log \hat{y}_c \\
&\text{Precision/Recall/F1: } P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}
\end{aligned}$$

When in doubt, open the deep guide. When really in doubt, run the script.

Part II — Joint Source–Channel Coding: The Core Theory

`jscg-guide.pdf` (26 pages)

This is the heart of the volume and the direct theoretical basis of the project. It builds the full JSCC story: scalar and vector quantization, transform coding and bit allocation, then the channel side — discrete channels with memory (BSC, Markov, Pólya, queue-based, noise-erasure), their capacities and decoding rules — then the joint design itself: channel-optimized vector quantization, error-and-erasure COVQ, index assignment, and a complete DCT image coder with an implementation blueprint. It leans on Part I’s information theory (§3) and probability (§2) throughout; when a derivation is compressed here, the full version lives in Part III. Read it for the *why*: why separation fails under delay and complexity constraints, why the decoder is a posterior mean, and why bursts change everything.

Joint Source–Channel Coding for Image Transmission over Noisy Channels with Memory

A Complete Guide: Mathematics, Intuition, Research, and Implementation

Compiled from course notes and research papers
on quantization, transform coding, and channels with memory

September 2026

Abstract

This guide streamlines nine source documents—lecture notes on scalar quantization and transform coding, a master’s thesis, and six research papers—into one coherent treatment of a single engineering problem: *how to send images over noisy wireless channels with memory, well*. It develops the mathematics from the ground up (quantization theory, transform coding, bit allocation, channel models, capacity), explains the key research results and the intuition behind them, and ends with a concrete implementation blueprint: algorithms in pseudocode, numerical safeguards, simulation methodology, and evaluation checklists. The central thread is *channel-optimized quantization*: instead of designing the source coder and the channel coder separately, the quantizer itself is designed to minimize end-to-end distortion over the noisy channel, exploiting—rather than interleaving away—the channel’s memory.

Contents

1	Introduction	4
1.1	The problem	4
1.2	How to use this guide	4
1.3	Source map	4
1.4	Notation	5
2	Mathematical preliminaries	5
2.1	Random variables and expectation	5
2.2	Entropy, mutual information, entropy rate	5
2.3	Distortion, SNR, PSNR	6
2.4	The end-to-end viewpoint	6
3	Scalar quantization	6
3.1	Definitions	6
3.2	The two necessary conditions for optimality	7
3.3	The Lloyd–Max algorithm	7
3.4	The uniform quantizer and the $\Delta^2/12$ law	8
3.5	High-resolution theory: companding and optimal point density	8
3.6	The 6 dB per bit rule	8

4	Vector quantization	9
4.1	Definitions	9
4.2	Why vectors beat scalars	9
4.3	Initialization: splitting	9
4.4	A preview of what noise changes	9
5	Transform coding	9
5.1	Setup and the key invariance	9
5.2	The Karhunen–Loève transform	10
5.3	The discrete cosine transform	10
5.4	Two-dimensional transforms and basis images	11
5.5	Optimal bit allocation	11
5.6	JPEG and subband/wavelet coding in brief	11
6	Channel models with memory	12
6.1	The binary symmetric channel and errors-and-erasures	12
6.2	Bursty noise: Gilbert–Elliott and binary Markov models	12
6.3	The Pólya-contagion channel	13
6.4	The finite-memory contagion channel	15
6.5	Queue-based channels (QBCs)	15
6.6	Noise-erasure channels and feedback	15
6.7	Which model when	16
7	Channel-optimized vector quantization	16
7.1	The end-to-end distortion objective	16
7.2	Necessary conditions for optimality	17
7.3	Fast encoder implementation	17
7.4	The COVQ training algorithm	17
7.5	Geometry: what noise does to the cells	18
7.6	Complexity	18
7.7	Worked example: 1-bit COVQ over a BSC	19
7.8	Tandem vs. joint: why separation loses here	19
8	Errors, erasures, and free soft information	19
8.1	The channel model	19
8.2	Why erasures are free soft information	19
8.3	The subband COVQ image coder: architecture and findings	19
9	A complete DCT image coder for bursty channels	20
9.1	System pipeline	20
9.2	Coefficient statistics	21
9.3	Channel-optimized scalar quantizers per coefficient	21
9.4	Fixed vs. optimal bit allocation	21
9.5	What the experiments showed	21
10	Implementation blueprint	21
10.1	Software architecture	22
10.2	Training the codebooks	22
10.3	Simulating the channel	22
10.4	Encoder/decoder runtime	23
10.5	Numerical safeguards checklist	23
10.6	Evaluation methodology	23

10.7 Complexity budgeting	24
11 What the research showed	24
12 Extensions and open directions	25
A Key formulas cheat sheet	26
Annotated bibliography	26

1 Introduction

1.1 The problem

A wireless link is not a clean pipe. Bits get flipped, sometimes erased, and—crucially—errors arrive in *bursts*: if the channel just corrupted a bit, the next bit is more likely to be corrupted too. The classical engineering response is *separation*: compress the image with a source coder (e.g. JPEG), protect the bits with a channel code, and interleave to break up the bursts so the channel *looks* memoryless to the decoder. Shannon’s separation theorem says this is optimal—in the limit of infinite delay and complexity.

Real systems do not live in that limit. This guide is about the alternative: *joint source–channel coding* (JSCC), and in particular *channel-optimized quantization* (COVQ). The idea is disarmingly simple: design the quantizer so that it directly minimizes the *end-to-end* distortion—source quantization error *plus* channel-induced error—for the actual noisy channel, including its memory. No interleaver, no separate channel code, no pretence that the channel is memoryless. The mathematics that makes this work touches quantization theory, transform coding, Markov channel models, and information theory; the implementation touches numerical optimization, Monte Carlo simulation, and careful experiment design. This guide covers all of it.

1.2 How to use this guide

- **Chapters 2–5** build the foundations: probability and information-theoretic prerequisites, scalar quantization (Lloyd–Max, high-resolution theory), vector quantization, and transform coding with optimal bit allocation. If you only want the noisy-channel material, you can skim these, but the COVQ chapters assume them.
- **Chapter 6** is a self-contained tour of channel models with memory: the BSC, erasure channels, Gilbert–Elliott, the Pólya-contagion channel and its finite-memory version, queue-based channels, and noise-erasure channels with feedback.
- **Chapters 7–9** develop channel-optimized quantization: the theory, the errors-and-erasures extension, and a complete DCT-based image coder for bursty channels.
- **Chapter 10** is the implementation blueprint: software architecture, pseudocode, numerical safeguards, simulation methodology, and evaluation checklists. Read this before writing code.
- **Chapters 11–12** summarize what the research actually showed and where to go next.

1.3 Source map

The nine source documents and where their content lives in this guide:

Source	Main chapters
Scalar quantization lecture notes (Linder, Queen’s)	3
Transform coding lecture notes (Linder, Queen’s)	5
MSc thesis: Cheng, <i>Channel Optimized Quantization of Images over Binary Channels with Memory</i> (Queen’s, 1997)	7, 9, 10
Farvardin & Vaishampayan, COVQ performance & complexity	7
Andersson & Skoglund, COVQ image coder: bit errors & erasures	8
Alajaji & Fuja, Pólya-contagion channel (1994)	6
Zhong, Alajaji & Takahara, queue-based channels (2007)	6
Song, Alajaji & Linder, noise-erasure channels & feedback	6
Fady’s report: DCT image coder over bursty channels	9

1.4 Notation

Scalars are lowercase (x), vectors bold lowercase ($\mathbf{x}$), matrices uppercase (A). Random variables are uppercase (X), realizations lowercase (x). $\mathbb{E}[\cdot]$ is expectation, $\mathbb{P}(\cdot)$ probability. $\|\mathbf{x}\|^2 = \sum_i x_i^2$. $\log$ is base 2 when it measures bits (entropy, mutual information) and natural log where calculus demands it; the base is stated when it matters. $h_b(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is the binary entropy function.

2 Mathematical preliminaries

2.1 Random variables and expectation

A (real-valued) random variable X is described by its distribution; for a continuous X with density f_X , $\mathbb{P}(X \in A) = \int_A f_X(x) dx$. Expectation is $\mathbb{E}[g(X)] = \int g(x) f_X(x) dx$ (sums for discrete X). Variance: $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$. Two workhorse densities in this guide:

- **Gaussian** $\mathcal{N}(\mu, \sigma^2)$: $f(x) = (2\pi\sigma^2)^{-1/2} \exp(-(x - \mu)^2/2\sigma^2)$.
- **Laplacian** with parameter λ : $f(x) = (\lambda/2) \exp(-\lambda|x - \mu|)$; heavier tails than the Gaussian, the standard model for AC transform coefficients of images.

2.2 Entropy, mutual information, entropy rate

For discrete X with mass function p :

$$H(X) = - \sum_x p(x) \log p(x) \quad (\text{bits}), \quad (1)$$

the fundamental limit of lossless compression. Mutual information $I(X; Y) = H(X) - H(X | Y)$ measures dependence. For a stationary process $\{Z_i\}$, the **entropy rate**

$$\bar{H}(Z) = \lim_{n \rightarrow \infty} \frac{1}{n} H(Z_1, \dots, Z_n) = \lim_{n \rightarrow \infty} H(Z_n | Z_{n-1}, \dots, Z_1) \quad (2)$$

is the per-symbol uncertainty once the past is known. For an additive-noise channel $Y_i = X_i \oplus Z_i$ with Z independent of X and stationary, the capacity is the beautifully simple

$$C = 1 - \bar{H}(Z) \quad \text{bits per use (binary case)}, \quad (3)$$

because $I(X^n; Y^n) = H(Y^n) - H(Z^n) \leq n - H(Z^n)$. Almost every capacity result in Chapter 6 is a variation on this theme: *find the entropy rate of the noise*.

2.3 Distortion, SNR, PSNR

A **distortion measure** $d(x, \hat{x}) \geq 0$ scores reconstruction quality. This guide uses squared error $d(x, \hat{x}) = (x - \hat{x})^2$ (and $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ for vectors) throughout, because it makes the mathematics tractable: expectations split, centroids are means, orthogonal transforms preserve it. For a coder producing reconstruction $\hat{X}$ of source X ,

$$D = \mathbb{E}[d(X, \hat{X})], \quad \text{SNR} = 10 \log_{10} \frac{\mathbb{E}[X^2]}{D} \text{ dB}. \quad (4)$$

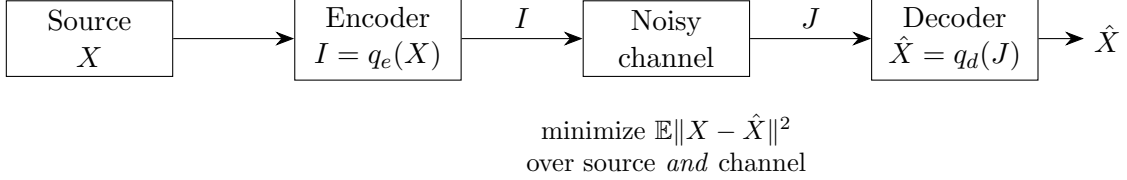
For images with peak pixel value MAX (255 for 8-bit),

$$\text{PSNR} = 10 \log_{10} \frac{\text{MAX}^2}{D} \text{ dB}, \quad (5)$$

the standard reported metric in Chapters 8–9. A 6 dB gain halves the root MSE; differences below ~ 0.5 dB are rarely visible or reproducible, a fact worth remembering when reading performance plots.

2.4 The end-to-end viewpoint

Classical source coding minimizes D assuming a *noiseless* channel between quantizer index and reconstruction. The moment the channel can corrupt indices, the right objective is the **end-to-end distortion**: average over *both* the source randomness and the channel randomness. Every design equation in Chapter 7 comes from writing down that single expectation and optimizing it. Keep this picture in mind:



3 Scalar quantization

Scalar quantization is the atomic operation of lossy compression: map each real number to one of finitely many representatives. Everything in vector quantization and COVQ is a generalization of this chapter, so it pays to understand it cold.

3.1 Definitions

Definition 3.1 (Scalar quantizer). A scalar quantizer is a map $Q : \mathbb{R} \rightarrow \mathcal{C}$ where the **codebook** $\mathcal{C} = \{y_1, \dots, y_N\}$ is a finite set of **reproduction levels** (codewords). The **partition cells** $\mathcal{R}_i = \{x : Q(x) = y_i\}$ tile the real line. Equivalently, Q is an **encoder** $q_e : \mathbb{R} \rightarrow \{1, \dots, N\}$ followed by a **decoder** $q_d : \{1, \dots, N\} \rightarrow \mathcal{C}$ with $Q(x) = q_d(q_e(x))$.

With N levels the **rate** is fixed at $R = \log_2 N$ bits per sample. The quality measure is the expected distortion

$$D(Q) = \mathbb{E}[d(X, Q(X))], \quad (6)$$

and for squared error

$$D(Q) = \mathbb{E}[(X - Q(X))^2] = \sum_{i=1}^N \int_{\mathcal{R}_i} (x - y_i)^2 f_X(x) dx. \quad (7)$$

The design problem: choose $\{\mathcal{R}_i\}$ and $\{y_i\}$ to minimize $D(Q)$.

3.2 The two necessary conditions for optimality

Fix the codebook and optimize the partition, then fix the partition and optimize the codebook. Both steps have closed forms.

Proposition 3.2 (Nearest-neighbor condition). *For fixed $\{y_i\}$, the partition minimizing $D(Q)$ assigns each x to the closest codeword:*

$$q_e(x) = \arg \min_i d(x, y_i), \quad (8)$$

i.e. $\mathcal{R}_i = \{x : d(x, y_i) \leq d(x, y_j) \ \forall j\}$ (ties broken arbitrarily but consistently).

Intuition. For each x independently, pick the y_i with smallest distortion—no coupling across x values, so the pointwise optimum is globally optimal.

Proposition 3.3 (Centroid condition). *For fixed $\{\mathcal{R}_i\}$, the optimal codewords are the **generalized centroids**:*

$$y_i = \arg \min_y \mathbb{E}[d(X, y) \mid X \in \mathcal{R}_i]. \quad (9)$$

For squared error this is the ordinary conditional mean:

$$y_i = \mathbb{E}[X \mid X \in \mathcal{R}_i] = \frac{\int_{\mathcal{R}_i} x f_X(x) dx}{\int_{\mathcal{R}_i} f_X(x) dx}. \quad (10)$$

Intuition. Once you know which cell x fell into, the best single guess under squared error is the mean of that cell.

3.3 The Lloyd–Max algorithm

Alternating the two conditions gives the Lloyd–Max iteration:

```

1 function LloydMax(samples x[1..T], N, tol):
2   # initialize codebook, e.g. uniform over [min(x), max(x)]
3   y[1..N] = uniform_init(x, N)
4   D_prev = +inf
5   repeat:
6     # (a) nearest-neighbor partition
7     for t = 1..T: i[t] = argmin_i (x[t]-y[i])^2
8     # (b) centroid update
9     for i = 1..N:
10      cell = { x[t] : i[t]==i }
11      y[i] = mean(cell) if cell nonempty else reinit(i)
12    D = (1/T) * sum_t (x[t]-y[i[t]])^2
13    gain = (D_prev - D)/D_prev
14    D_prev = D
15  until gain < tol
16  return y[1..N], i[1..T]
```

Listing 1: Lloyd–Max for scalar quantization.

Key facts for implementers:

- Each step can only *decrease* (never increase) the training distortion, so the algorithm converges—but only to a **local** optimum (a fixed point of the two conditions). Run from several initializations and keep the best.
- **Empty cells** happen (a codeword attracts no samples). Never divide by zero: reinitialize the orphaned codeword (split the cell with the highest distortion, or place it at a random sample).
- On a finite training set, replace densities by empirical averages; the centroid is then just the sample mean of the cell.
- Cost per iteration: $O(TN)$ distance computations; trivially vectorizable.

3.4 The uniform quantizer and the $\Delta^2/12$ law

For a uniform quantizer with step size Δ (and N large enough that overload distortion is negligible), the quantization error is approximately uniform on $[-\Delta/2, \Delta/2]$, giving the famous

$$D \approx \frac{\Delta^2}{12}. \quad (11)$$

This is the baseline everything else is measured against, and the seed of all high-resolution analysis.

3.5 High-resolution theory: companding and optimal point density

At high rate, an optimal quantizer can be built by **companding**: apply a monotone compressor $G : \mathbb{R} \rightarrow (0, 1)$, quantize uniformly with N levels, then expand with G^{-1} . The distortion satisfies

$$D(Q_{G,N}) \approx \frac{1}{12N^2} \int_{\mathbb{R}} \frac{f_X(x)}{G'(x)^2} dx. \quad (12)$$

Minimizing over G (via Hölder's inequality) gives the optimal **point density**

$$G'(x) = \frac{f_X(x)^{1/3}}{\int_{\mathbb{R}} f_X(u)^{1/3} du}, \quad (13)$$

i.e. put more levels where $f_X^{1/3}$ is large—a compromise between f_X itself (which would waste levels on sharp peaks) and uniform spacing. The resulting minimum distortion is

$$D^*(N) \approx \frac{1}{12N^2} \left(\int_{\mathbb{R}} f_X(x)^{1/3} dx \right)^3. \quad (14)$$

Bucklew and Wise made this exact asymptotically: $\lim_{N \rightarrow \infty} N^2 D^*(N) = \frac{1}{12} \|f_X\|_{1/3}$ under mild moment conditions. The takeaway for implementation: the *shape* of the optimal quantizer is dictated by $f^{1/3}$, and Lloyd–Max on enough data discovers it automatically.

3.6 The 6 dB per bit rule

Write $R = \log_2 N$, so $N^{-2} = 2^{-2R}$ and $D^*(N) \approx 2^{-2R} C(f)$ with $C(f)$ independent of N . Then

$$\text{SNR} \approx 10 \log_{10} \frac{\mathbb{E}[X^2]}{2^{-2R} C(f)} = 20R \log_{10} 2 + 10 \log_{10} \frac{\mathbb{E}[X^2]}{C(f)} \approx 6.02 R + \text{const dB}. \quad (15)$$

Each extra bit buys about 6 dB of SNR. This is the quickest sanity check in quantization: if adding a bit to a well-designed quantizer buys far less than 6 dB, something is wrong (bad initialization, empty cells, or the high-resolution regime doesn't apply).

4 Vector quantization

4.1 Definitions

Definition 4.1 (Vector quantizer). A k -dimensional vector quantizer is $Q : \mathbb{R}^k \rightarrow \mathcal{C}$ with codebook $\mathcal{C} = \{\mathbf{y}_1, \dots, \mathbf{y}_M\}$ of M codevectors. The rate is $R = \frac{1}{k} \log_2 M$ bits per dimension. Distortion: $D(Q) = \mathbb{E}[\|\mathbf{X} - Q(\mathbf{X})\|^2]$.

The nearest-neighbor and centroid conditions carry over verbatim with $\|\cdot\|^2$ in place of $(\cdot)^2$; the centroid of cell $\mathcal{R}_i$ is $\mathbb{E}[\mathbf{X} \mid \mathbf{X} \in \mathcal{R}_i]$. Alternating them is the **LBG (Linde–Buzo–Gray) algorithm**, the vector version of Listing 1.

4.2 Why vectors beat scalars

Even at the same rate, VQ wins for three distinct reasons:

1. **Space-filling advantage:** spheres pack better than cubes; optimal vector cells are more spherical than rectangular scalar cells.
2. **Shape advantage:** VQ adapts to the *shape* of a non-uniform density; scalar quantization cannot.
3. **Memory advantage:** for sources with dependence across samples, VQ exploits it directly; scalar quantization needs a transform first (Chapter 5).

4.3 Initialization: splitting

LBG is local, so initialization matters. The standard **splitting** method: start with the single centroid $M = 1$; split each codevector $\mathbf{y}$ into $\mathbf{y} \pm \epsilon$; run LBG to convergence; repeat until M codevectors are reached. This grows a good codebook hierarchically and largely avoids poor local optima. Always keep the standard guards: empty-cell reinitialization, relative-distortion stopping criterion, multiple restarts for the final reported number.

4.4 A preview of what noise changes

Everything above assumes the decoder sees the encoder's index perfectly. Over a noisy channel, index i sent may arrive as index $j \neq i$ —and the damage depends on *which* wrong codevector $\mathbf{y}_j$ is substituted. The fix is not to patch VQ after the fact but to put the channel transition probabilities $P(j | i)$ *inside* the distortion objective. That is Chapter 7.

5 Transform coding

Scalar quantization treats each sample independently. Real signals (images, audio) are *correlated*: knowing one pixel tells you about its neighbors. Transform coding removes that correlation first, then quantizes the decorrelated coefficients independently—getting most of VQ's memory advantage at scalar complexity.

5.1 Setup and the key invariance

Let $\mathbf{X} = (X_1, \dots, X_k)^t$ and let A be a $k \times k$ **orthogonal** matrix ($A^t = A^{-1}$). Define transform coefficients $\mathbf{Y} = A\mathbf{X}$, quantize each Y_i with a scalar quantizer Q_i to get $\hat{\mathbf{Y}}$, and invert: $\hat{\mathbf{X}} = A^{-1}\hat{\mathbf{Y}} = A^t\hat{\mathbf{Y}}$.

Proposition 5.1 (Distortion invariance). *For orthogonal A , $\|\mathbf{X} - \hat{\mathbf{X}}\| = \|\mathbf{Y} - \hat{\mathbf{Y}}\|$, hence*

$$D_{\text{tc}} = \mathbb{E}\|\mathbf{X} - \hat{\mathbf{X}}\|^2 = \sum_{i=1}^k \mathbb{E}[(Y_i - Q_i(Y_i))^2]. \quad (16)$$

Proof. $\|\mathbf{Y} - \hat{\mathbf{Y}}\| = \|A\mathbf{X} - AA^{-1}\hat{\mathbf{Y}}\| = \|A(\mathbf{X} - \hat{\mathbf{X}})\| = \|\mathbf{X} - \hat{\mathbf{X}}\|$ since orthogonal matrices preserve norms. $\square$

The end-to-end MSE is exactly the sum of the coefficient quantization errors. The design problem splits cleanly: **(a)** choose A to make the Y_i easy to quantize (unequal variances, independent-ish), **(b)** allocate bits among the Q_i .

5.2 The Karhunen–Loève transform

Let $R_X = \mathbb{E}[\mathbf{X}\mathbf{X}^t]$ be the autocorrelation matrix (symmetric, positive semidefinite). Diagonalize it: $R_X = U\Lambda U^t$ with U orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_k)$, $\lambda_1 \geq \dots \geq \lambda_k \geq 0$. The **KLT matrix** is $T = U^t$.

Proposition 5.2 (KLT decorrelates). *For $\mathbf{Y} = T\mathbf{X}$, $\mathbb{E}[Y_i Y_j] = 0$ for $i \neq j$ and $\mathbb{E}[Y_i^2] = \lambda_i$.*

Proof. $R_Y = \mathbb{E}[\mathbf{Y}\mathbf{Y}^t] = U^t R_X U = U^t (U\Lambda U^t) U = \Lambda$, diagonal. $\square$

Why decorrelation helps: independent-ish coefficients with very unequal variances λ_i let bit allocation (Section 5.5) spend bits where they matter and starve the near-zero-variance directions. The KLT is optimal in a strong sense:

Theorem 5.3 (High-resolution optimality of the KLT). *For Gaussian $\mathbf{X}$ with nonsingular R_X , under the high-resolution distortion formula and optimal bit allocation, the KLT minimizes D_{tc} over all orthogonal A .*

Proof sketch. With $W_i(b) = h_g \sigma_i^2 2^{-2b_i}$ (Gaussian shape factor h_g common to all coefficients) and optimal allocation, the distortion for transform A is $D_{\text{tc},A} = k h_g 2^{-2\bar{b}} (\prod_i \sigma_i^2)^{1/k}$ with $\sigma_i^2 = \mathbb{E}[Y_i^2]$, the diagonal of R_Y . By Hadamard's inequality, $\det(R_Y) \leq \prod_i \sigma_i^2$ with equality iff R_Y is diagonal; and $\det(R_Y) = \det(R_X)$ for orthogonal A . Hence $D_{\text{tc},A} \geq k h_g 2^{-2\bar{b}} \det(R_X)^{1/k}$, with equality for the KLT, which diagonalizes R_Y . $\square$

The **transform coding gain** over plain scalar quantization (for equal-variance inputs, $\mathbb{E}[X_i^2] = \sigma_X^2$) is

$$\frac{D_{\text{PCM}}}{D_{\text{tc}}} = \frac{\frac{1}{k} \sum_i \lambda_i}{(\prod_i \lambda_i)^{1/k}} \geq 1, \quad (17)$$

the ratio of arithmetic to geometric mean of the eigenvalues—strictly greater than 1 unless the inputs are already uncorrelated. *The more uneven the eigenvalue spread, the more transform coding wins.*

5.3 The discrete cosine transform

The KLT is signal-dependent: it must be recomputed whenever R_X changes. The workhorse replacement is the fixed, signal-independent **DCT**:

$$t_{mn} = \begin{cases} \sqrt{\frac{2}{k}} \cos\left(\frac{\pi}{k} m(n + \frac{1}{2})\right), & m = 1, \dots, k-1, \\ \sqrt{\frac{1}{k}}, & m = 0, \end{cases} \quad n = 0, \dots, k-1. \quad (18)$$

It is orthogonal, computable in $O(k \log k)$ via the FFT (mirror the sequence, take the FFT, scale the first k outputs), compacts energy superbly for highly correlated data, and closely approximates the KLT for Markov-type sources. This is why JPEG uses the 8×8 DCT.

5.4 Two-dimensional transforms and basis images

For a $k \times k$ pixel block X , the separable 2-D transform is $Y = AXA^t$ (transform rows, then columns), inverted by $X = A^t Y A$. Writing $A^t = [\mathbf{a}_0 \cdots \mathbf{a}_{k-1}]$,

$$X = \sum_{i,j=0}^{k-1} Y_{ij} B_{ij}, \quad B_{ij} = \mathbf{a}_i \mathbf{a}_j^t, \quad (19)$$

so each coefficient Y_{ij} weights a **basis image** B_{ij} (the familiar 8×8 DCT checkerboards, from flat gray up to rapid oscillation). Quantizing Y coarsely discards high-frequency basis images first—exactly what the eye tolerates.

5.5 Optimal bit allocation

Problem. Coefficients $X_1, \dots, X_k$ with densities f_i ; quantize X_i with b_i bits ($\sum_i b_i \leq B$) to minimize total MSE $D(\mathbf{b}) = \sum_i W_i(b_i)$, where $W_i(b)$ is the optimal b -bit distortion for X_i .

Under the high-resolution formula $W_i(b) = \frac{1}{12} \|f_i\|_{1/3} 2^{-2b}$, write $\sigma_i^2 = \text{Var}(X_i)$ and factor out scale: $W_i(b) = h_i \sigma_i^2 2^{-2b}$ where the **shape factor** h_i depends only on the density's shape (same for all Gaussians).

Theorem 5.4 (Optimal bit allocation). *With b_i real-valued, $D(\mathbf{b}) = \sum_i h_i \sigma_i^2 2^{-2b_i}$ under $\sum_i b_i = B$ is minimized iff*

$$b_i = \bar{b} + \frac{1}{2} \log_2 \frac{\sigma_i^2}{\rho^2} + \frac{1}{2} \log_2 \frac{h_i}{H}, \quad \bar{b} = \frac{B}{k}, \quad \rho^2 = \left(\prod_i \sigma_i^2 \right)^{1/k}, \quad H = \left(\prod_i h_i \right)^{1/k}. \quad (20)$$

The minimum distortion is $D_{\text{opt}} = kH\rho^2 2^{-2\bar{b}}$: as if all k variables had variance ρ^2 , shape H , and $\bar{b}$ bits each.

Proof sketch. Lagrange multiplier on

$$J(\mathbf{b}, \lambda) = \sum_i h_i \sigma_i^2 2^{-2b_i} + \lambda \left(\sum_i b_i - B \right) :$$

$\partial J / \partial b_j = -2 \ln 2 h_j \sigma_j^2 2^{-2b_j} + \lambda = 0$ gives $b_j = \frac{1}{2} \log_2 h_j + \frac{1}{2} \log_2 \sigma_j^2 + \hat{\lambda}$; the constraint $\sum_j b_j = B$ fixes $\hat{\lambda}$. Strict convexity of D makes this sufficient. $\square$

Reading the formula: give more bits to high-variance coefficients (σ_i^2 term) and to “harder” densities (h_i term); the geometric means ρ^2, H are the reference levels. Figure 1 shows a typical allocation.

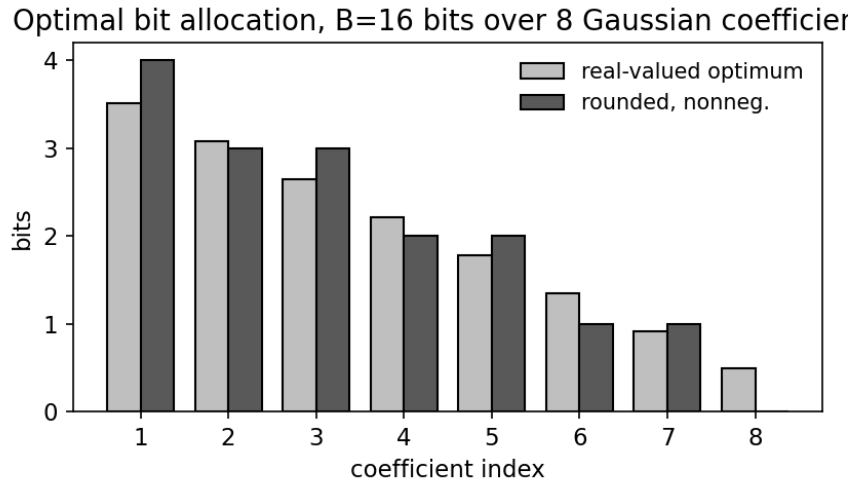


Figure 1: Optimal bit allocation across 8 Gaussian coefficients with geometrically decaying variances, $B = 16$ bits total.

In practice the b_i must be nonnegative integers. The practical recipe: use the analytic solution, clip negatives to zero, round, and fix the budget heuristically (a greedy bit-by-bit allocation is exactly optimal under the high-resolution model). For the image coder of Chapter 9, an even simpler fixed (zonal) allocation loses only ~ 1 dB—a recurring lesson: *exact optimality of bit allocation matters less than getting the big decisions right.*

5.6 JPEG and subband/wavelet coding in brief

JPEG = 8×8 DCT + uniform quantization with per-coefficient step sizes (implicit bit allocation) + runlength/Huffman lossless coding of the quantized indices, after RGB $\rightarrow$ YCbCr conversion with chroma subsampling. Rule-of-thumb quality for color photos: 0.25 bpp fair–good, 0.5 bpp good–very good, 1.5 bpp excellent, >2.5 bpp visually lossless (vs. 24 bpp raw). At very low rates the 8×8 block grid becomes visible (blocking artifacts).

Subband/wavelet coding replaces the block transform by a filter bank (typically dyadic: split lowpass/highpass, recurse on the lowpass), quantizing each subband with its own bit allocation. Wavelets (JPEG 2000) eliminate blocking and win clearly at low rates, at the cost of more complex machinery. The COVQ image coder in Chapter 8 uses a 4-level, 13-subband decomposition with vectors formed *within* each subband.

6 Channel models with memory

A channel model is a contract: it tells the code designer what the noise *can* do, in exchange for mathematical tractability. This chapter tours the models used across the source papers, from the memoryless BSC up to the Pólya-contagion and queue-based channels whose memory is the whole point. The unifying question is always: *what is the entropy rate of the noise?* For binary additive noise $Y_i = X_i \oplus Z_i$ with Z independent of X and stationary, capacity is $C = 1 - \bar{H}(Z)$.

6.1 The binary symmetric channel and errors-and-erasures

The **BSC** with crossover ε : each bit flips independently with probability ε . Capacity $1 - h_b(\varepsilon)$. It is the default “no-memory” baseline.

The **binary symmetric erasure channel (BSEC)** adds a third output: each bit is correct with probability $(1 - \alpha)(1 - \beta)$, flipped with probability $\alpha(1 - \beta)$, and **erased** (replaced by a flag e) with probability β . Erasures are gold: the decoder *knows* which positions are unreliable. Chapter 8 shows how a channel-optimized quantizer turns erasure flags into soft information for free.

6.2 Bursty noise: Gilbert–Elliott and binary Markov models

Wireless errors come in bursts. The classical **Gilbert–Elliott** model is a two-state hidden Markov chain (“good” state with small crossover, “bad” state with large crossover). For quantizer design it is convenient to model the *noise process* $\{Z_i\}$ directly as a **first-order homogeneous Markov chain**: with states 0, 1,

$$\Pi = \begin{pmatrix} 1 - p_{01} & p_{01} \\ p_{10} & 1 - p_{10} \end{pmatrix}, \quad (21)$$

stationary bit-error rate $\text{BER} = \pi_1$ and correlation controlled by p_{01}, p_{10} . This is the channel used for the DCT image coder in Chapter 9, and the $M = 1$ case of the finite-memory Pólya channel below can match *any* irreducible two-state chain.

Implementation note: simulating this channel is one line per bit (draw the next state from the current row of Π); estimating (BER, correlation) from a measured error trace is just counting transitions. Always simulate with the *stationary* initial distribution unless you explicitly want transient effects.

6.3 The Pólya-contagion channel

The most distinctive model in the set comes from Alajaji & Fuja (1994): errors that *spread like a contagious disease*. The motivation was empirical—defect clusters in semiconductor memories fit Pólya–Eggenberger statistics far better than Poisson—but the mathematics is a beautiful cautionary tale about memory and capacity.

The urn model. An urn holds T balls: R red, S black ($T = R + S$). Draw a ball; then return it *plus* Δ extra balls of the same color. Define $\rho = R/T$, $\sigma = S/T = 1 - \rho$, $\delta = \Delta/T$.

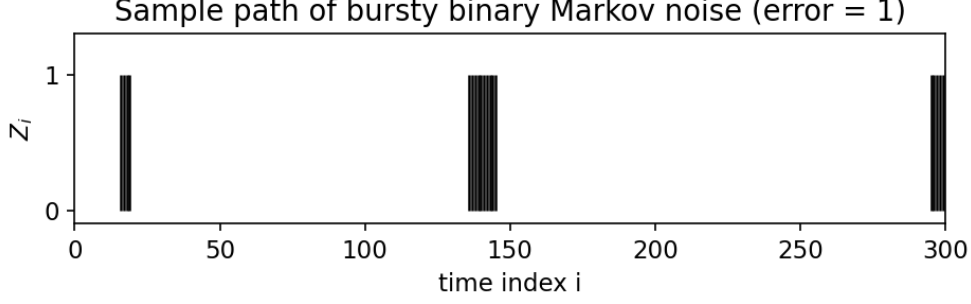


Figure 2: Bursty error trace from a two-state Markov noise model: long clean stretches punctuated by error bursts—the regime where exploiting memory beats interleaving it away.

The noise bit is

$$Z_i = \begin{cases} 1 & \text{ith ball drawn is red (an error),} \\ 0 & \text{ith ball drawn is black,} \end{cases} \quad (22)$$

and the channel is $Y_i = X_i \oplus Z_i$. We take $\Delta > 0$ (contagion: each error breeds more errors) and $\rho < 1/2$. For $\Delta = 0$ this collapses to a BSC with crossover ρ .

Let $S_n = Z_1 + \dots + Z_n$ be the channel state (total red drawn). Conditioning on the state gives the per-symbol law

$$\mathbb{P}(Y_i = y_i \mid X_i = x_i, S_{i-1} = s_{i-1}) = \left[\frac{\rho + s_{i-1}\delta}{1 + (i-1)\delta} \right]^{y_i \oplus x_i} \left[\frac{\sigma + (i-1-s_{i-1})\delta}{1 + (i-1)\delta} \right]^{1-(y_i \oplus x_i)}, \quad (23)$$

and multiplying out, the n -block transition probability depends only on the Hamming distance $d = d(\mathbf{x}, \mathbf{y})$:

$$\mathbb{P}(\mathbf{Y} = \mathbf{y} \mid \mathbf{X} = \mathbf{x}) = \frac{\rho(\rho + \delta) \cdots (\rho + (d-1)\delta) \sigma(\sigma + \delta) \cdots (\sigma + (n-d-1)\delta)}{(1 + \delta)(1 + 2\delta) \cdots (1 + (n-1)\delta)}. \quad (24)$$

Properties. The channel is *symmetric* (depends only on $\mathbf{x} \oplus \mathbf{y}$, so uniform inputs maximize mutual information), *stationary*, and the noise is *exchangeable*: $\mathbb{P}(Z_i = 1) = \rho$ for all i with positive correlation

$$\text{Cor}(Z_i, Z_j) = \frac{\delta}{1 + \delta} > 0, \quad i \neq j. \quad (25)$$

But it is **not ergodic**: the sample mean S_n/n converges a.s. not to a constant but to a $\text{Beta}(\rho/\delta, \sigma/\delta)$ random variable. In words: each transmission *lot* has its own effective crossover probability, drawn once from a beta distribution—some lots are nearly clean, others are disasters, and no amount of averaging within a lot reveals which you got.

ML decoding: closest or farthest. Write the transition probability as $g(d)$ with $g(d) = A \cdot \Gamma(\rho/\delta + d) \Gamma(\sigma/\delta + n - d)$. Since Γ is strictly log-convex, g is strictly log-convex in d : a U-shaped curve with unique minimum at

$$d_0 = \frac{n}{2} + \frac{1 - 2\rho}{2\delta}, \quad (26)$$

symmetric about d_0 (Figure 3). Maximum-likelihood decoding therefore maps the received word to the codeword whose distance is **farthest from** d_0 : the *closest* codeword if all candidates lie below d_0 , the *farthest* if they lie above—an “apparent epidemic” (huge distance) is evidence the lot was bad and the transmitted word was the complement-like extreme. When $\delta \rightarrow 0$ ($d_0 > n - 0.5$), this reduces to ordinary minimum-distance decoding, as it must for the BSC.

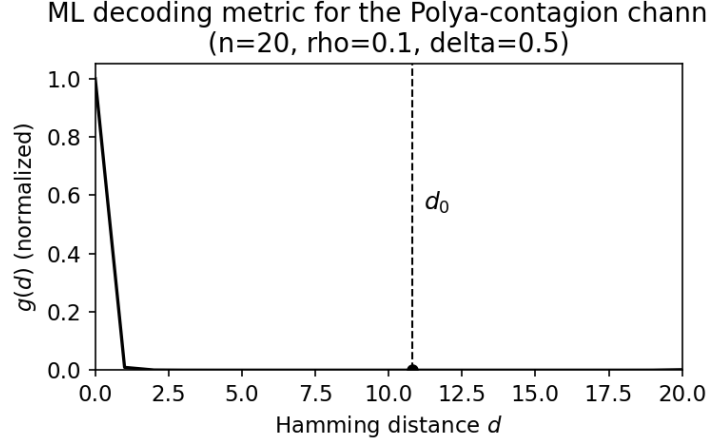


Figure 3: The Pólya channel’s ML metric $g(d)$ vs. Hamming distance d : strictly log-convex (U-shaped) with minimum at d_0 . ML decoding picks the codeword farthest from d_0 —nearest *or* farthest in Hamming distance.

Capacity is zero. By De Finetti’s theorem, exchangeable binary noise is a *mixture* of i.i.d. (Bernoulli) processes: the Pólya channel is an **averaged channel** whose components are BSCs with crossover $\theta \sim \text{Beta}(\rho/\delta, \sigma/\delta)$. Its capacity is $C = 1 - \overline{H}(Z) = 1 - \text{ess sup}_\theta h_b(\theta) = 0$, because the beta distribution puts positive mass near $\theta = 1/2$. This is a genuine counterexample to the folk adage “memory can only increase capacity”: here memory *destroyed* it. The ϵ -capacity is more forgiving: $C_\epsilon = 1 - F_V^{-1}(1 - \epsilon)$ with $V = h_b(U)$, $U \sim \text{Beta}(\rho/\delta, \sigma/\delta)$.

Lesson for the practitioner: nonergodicity is the enemy. If your channel has a “lot-to-lot” variability that never averages out (deep fades that persist for the whole codeword), no clever coding within a lot can beat it—you need diversity *across* lots (interleaving across fades) or outage-based design criteria.

6.4 The finite-memory contagion channel

The infinite memory of the Pólya urn is unrealistic (the millionth draw should not remember the first). The finite-memory fix: M draws after adding the Δ contagion balls, *remove* them again. The urn then always holds $T + M\Delta$ balls, and

$$\mathbb{P}(Z_n = 1 \mid Z_{n-1} = e_{n-1}, \dots) = \frac{\rho + (e_{n-1} + \dots + e_{n-M})\delta}{1 + M\delta}, \quad (27)$$

so $\{Z_n\}$ is a **stationary ergodic M th-order Markov** process. Blocking M consecutive bits into super-states gives a 2^M -state chain that is irreducible and aperiodic, hence strongly mixing and ergodic. For $M = 1$ the transition matrix

$$\Pi = \frac{1}{1 + \delta} \begin{pmatrix} \sigma + \delta & 1 - \sigma \\ \sigma & 1 - \sigma + \delta \end{pmatrix} \quad (28)$$

can match any irreducible two-state Markov chain by choice of (σ, δ) .

Capacity has a closed form. With $L_k = \frac{\prod_{j=0}^{k-1} (\rho + j\delta) \prod_{l=0}^{M-k-1} (\sigma + l\delta)}{\prod_{m=1}^{M-1} (1 + m\delta)}$,

$$C_M = 1 - \sum_{k=0}^M \binom{M}{k} L_k h_b\left(\frac{\rho + k\delta}{1 + M\delta}\right), \quad (29)$$

nondecreasing in M , converging as $M \rightarrow \infty$ to $1 - \int_0^1 h_b(z) f_Z(z) dz$ with f_Z the $\text{Beta}(\rho/\delta, \sigma/\delta)$ density. Note the subtlety: as M grows the n -fold transition probabilities converge to the Pólya channel's—but the capacities do *not* converge to 0. Capacity is discontinuous in this limit because ergodicity is lost.

6.5 Queue-based channels (QBCs)

Zhong, Alajaji & Takahara (2007) proposed a more implementation-friendly bursty-noise model. Picture a **queue of M bits** plus an urn: the noise bit Z_n is generated from the queue contents and an urn draw, then shifted into the queue. The result is again a stationary ergodic M th-order Markov noise process driving $Y_i = X_i \oplus Z_i$, but with two practical advantages:

- **Transparent block probabilities:** $\mathbb{P}(\mathbf{Z} = \mathbf{z})$ factors cleanly, so likelihood computations and simulations are straightforward.
- **Computable capacity:** $C = 1 - \bar{H}(Z)$, with the entropy rate evaluable from the Markov structure.

The QBC is a workhorse for *fitting*: given a target bursty channel (e.g. a Gilbert–Elliott model), match BER and correlation coefficient and minimize the KL-divergence rate to get a QBC surrogate with honest, computable probabilities. A useful equivalence: the **uniform QBC** (urn balanced) is statistically identical to the finite-memory Pólya channel with matched memory order, BER, and correlation—so results proved for one transfer to the other.

Design moral: when you need bursty noise in a simulator or a likelihood, prefer the model whose probabilities you can actually write down and compute. The QBC exists precisely because the Gilbert–Elliott model, while intuitive, makes block probabilities awkward.

6.6 Noise-erasure channels and feedback

Song, Alajaji & Linder extend the picture to channels with *erasures* and *feedback*. The **noise-erasure channel (NEC)**: a noise process $\{Z_i\}$ over an alphabet including a special erasure symbol e ; the output is

$$Y_i = \begin{cases} h(X_i, Z_i), & Z_i \neq e, \\ e, & Z_i = e, \end{cases} \quad (30)$$

for q -ary inputs. Under invertibility conditions on h , the channel is quasi-symmetric, so the **uniform input maximizes** block mutual information, and with erasure probability ε ,

$$C = (1 - \varepsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})], \quad (31)$$

where $\tilde{Z}_i = \mathbf{1}\{Z_i = e\}$ is the erasure-indicator process. Read it as: you get the erasure channel's $(1 - \varepsilon) \log q$, minus the residual noise uncertainty *beyond* what the erasure pattern already reveals.

Feedback does *not* increase the (unconstrained) capacity: $C_{\text{FB}} = C$. But—and this is the practically relevant part—feedback *can* increase the **capacity–cost function** when inputs have cost constraints and the noise-erasure process is Markov: knowing past outputs lets the encoder spend its cost budget where it helps most. For hidden-Markov erasure indicators the entropy rate $\bar{H}(\tilde{Z})$ can be squeezed by recursive bounds $H(\tilde{Z}_\ell \mid \tilde{Z}^{\ell-1}, Z_1) \leq \bar{H}(\tilde{Z}) \leq H(\tilde{Z}_\ell \mid \tilde{Z}^{\ell-1})$, tightening with ℓ .

6.7 Which model when

Situation	Use	Why
Quick baseline, no memory	BSC	One parameter; closed form
Known erasures (packet drops)	BSEC / NEC	Erasures are free info
Bursty bit errors, need simulation	Binary Markov / QBC	Easy to simulate; QBC has clean probs
Fitting a measured burst channel	QBC matched by BER+corr.	Minimizes KL rate; computable capacity
Deep, persistent fades (non-ergodic)	Pólya (averaged BSCs)	Captures lot-to-lot variability; $C = 0$ warning
Cost-constrained coding with feedback	NEC + feedback	Feedback helps the cost-capacity tradeoff

7 Channel-optimized vector quantization

We now reach the center of the guide. A classical vector quantizer is designed for a noiseless channel and then thrown onto a noisy one; a **channel-optimized vector quantizer (COVQ)** is designed *for* the noisy channel from the start. The development below follows Farvardin & Vaishampayan, with complexity and geometry results from the COVQ performance/complexity paper and the errors-and-erasures extension deferred to Chapter 8.

7.1 The end-to-end distortion objective

Encoder: $\mathbf{X} \mapsto I \in \{1, \dots, M\}$ via partition cells $\{\mathcal{S}_i\}$. Channel: $I \mapsto J$ with known transition probabilities $P(j | i)$. Decoder: $J \mapsto \mathbf{y}_J$. The **end-to-end MSE** averages over source *and* channel:

$$D = \sum_{i=1}^M \int_{\mathcal{S}_i} p_{\mathbf{X}}(\mathbf{x}) \sum_{j=1}^M P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 d\mathbf{x}. \quad (32)$$

Compare with clean-channel VQ: the inner sum over j weighted by $P(j | i)$ is new. Sending index i no longer guarantees reconstruction $\mathbf{y}_i$; it yields a *distribution* over reconstructions, and the encoder must account for all of them.

7.2 Necessary conditions for optimality

Optimal encoder (decoder fixed). For each $\mathbf{x}$, choose the index minimizing the *expected* distortion over the channel:

$$\mathcal{S}_i = \left\{ \mathbf{x} : \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 \leq \sum_j P(j | i') \|\mathbf{x} - \mathbf{y}_j\|^2 \quad \forall i' \right\}. \quad (33)$$

Intuition: the encoder is still nearest-neighbor—but nearest to a *channel-averaged* notion of distance. Index i is “close” to $\mathbf{x}$ if the reconstructions i is likely to produce are close to $\mathbf{x}$ on average.

Optimal decoder (encoder fixed). For each received j , the best reconstruction is the posterior mean:

$$\mathbf{y}_j = \mathbb{E}[\mathbf{X} | J = j] = \frac{\sum_i \mathbb{P}(I = i) P(j | i) \mathbf{c}_i}{\sum_i \mathbb{P}(I = i) P(j | i)}, \quad \mathbf{c}_i = \mathbb{E}[\mathbf{X} | I = i], \quad (34)$$

where $\mathbf{c}_i$ is the centroid of cell $\mathcal{S}_i$. *Intuition:* receiving j tells you I was probably some i with large $P(j | i)$; hedge across them with the posterior weights. On a noiseless channel ($P(j | i) = \delta_{ij}$) this collapses to $\mathbf{y}_i = \mathbf{c}_i$, the classical centroid condition.

7.3 Fast encoder implementation

Naïve evaluation of (33) costs $O(M^2)$ distance computations per vector. Expanding the square,

$$\sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 = \|\mathbf{x}\|^2 - 2\mathbf{x}^t \left(\sum_j P(j | i) \mathbf{y}_j \right) + \sum_j P(j | i) \|\mathbf{y}_j\|^2, \quad (35)$$

the $\|\mathbf{x}\|^2$ term is common to all i , so encoding reduces to a **linear** discriminant per index:

$$i^*(\mathbf{x}) = \arg \max_i \{ \mathbf{x}^t \tilde{\mathbf{y}}_i - \frac{1}{2} b_i \}, \quad \tilde{\mathbf{y}}_i = \sum_j P(j | i) \mathbf{y}_j, \quad b_i = \sum_j P(j | i) \|\mathbf{y}_j\|^2. \quad (36)$$

Precompute the **effective codevectors** $\tilde{\mathbf{y}}_i$ and offsets b_i once per training iteration; encoding is then M inner products—the same order as clean-channel VQ. This is the complexity result at the heart of the Farvardin–Vaishampayan performance/complexity analysis: *channel optimization need not cost more than classical VQ at encode time.*

7.4 The COVQ training algorithm

```

1 function TrainCOVQ(samples X[1..T] in R^k, M, P(j|i), tol):
2   # initialize: LBG codebook or splitting; y = codevectors
3   y[1..M] = init_codebook(X, M)
4   D_prev = +inf
5   repeat:
6     # (a) channel-aware partition [Eq. (7.2), via effective
7       codevectors]
8     tilde_y[i] = sum_j P(j|i)*y[j]; b[i] = sum_j P(j|i)*||y[j]||^2
9     for t = 1..T:
10      i[t] = argmax_i ( X[t].tilde_y[i] - 0.5*b[i] )
11    # (b) cell probabilities, centroids
12    for i = 1..M:
13      cell_i = { X[t] : i[t]==i }; pI[i] = |cell_i|/T
14      c[i] = mean(cell_i) if nonempty else reinit
15    # (c) posterior-mean decoder [Eq. (7.3)]
16    for j = 1..M:
17      num = sum_i pI[i]*P(j|i)*c[i]; den = sum_i pI[i]*P(j|i)
18      y[j] = num/den (if den>0 else c[j])
19    D = (1/T)*sum_t sum_j P(j|i[t])*||X[t]-y[j]||^2
20    gain = (D_prev-D)/D_prev; D_prev = D
21  until gain < tol
22  return y[1..M] # decoder codebook; encoder uses (tilde_y, b)

```

Listing 2: COVQ training (discrete-alphabet channel).

This is alternating minimization of (32): each of steps (a) and (c) is conditionally optimal, so D decreases monotonically and the algorithm converges to a **stationary point** (local optimum). Practical notes:

- **Initialize from a clean-channel VQ** (or splitting) and let the channel warp it; random initialization wastes the structure.

- The channel matrix $P(j | i)$ for an L -bit index over a BSC is $P(j | i) = \varepsilon^{d_H(i,j)}(1-\varepsilon)^{L-d_H(i,j)}$ with Hamming distance d_H —but the *binary labeling* of indices matters enormously (see index assignment below).
- **Index assignment** (which codevector gets which L -bit label) should place frequently-confused indices on similar codevectors. For the BSC, Gray-like labelings help; the full optimization is a quadratic assignment problem, so use heuristics (binary switching algorithm) or accept natural order and let training compensate.
- Step (c) needs $p_I[i]$: estimate from the training partition, or iterate with a source model.

7.5 Geometry: what noise does to the cells

Three qualitative changes versus clean-channel VQ:

1. **Warped cells.** Decision boundaries are still hyperplanes (the discriminant is linear in $\mathbf{x}$) but shifted and rotated relative to the clean-channel Voronoi diagram, because $\tilde{\mathbf{y}}_i$ blends each codevector with its likely corruptions.
2. **Empty encoding regions.** On noisy channels some indices are never the best choice for any $\mathbf{x}$ —their cells are empty. Farvardin & Vaishampayan report a striking case: for a correlated Gauss–Markov source with $k = 8$, $M = 256$, BSC crossover 0.1, only **61 of 256** encoding regions remained nonempty. The channel effectively *reduces* the codebook: don’t be surprised, and don’t “fix” it by forcing cells nonempty—the empty indices are the optimizer telling you those codevectors aren’t worth their confusion risk. (Do guard the division in step (c).)
3. **Decoder shrinkage.** Posterior means $\mathbb{E}[\mathbf{X} | J = j]$ shrink toward the prior mean relative to clean centroids—the decoder hedges because J is unreliable.

7.6 Complexity

Per training iteration: $O(TM)$ for encoding (inner products), $O(M^2)$ for the decoder update, $O(M^2k)$ to form effective codevectors. At runtime, encoding is M inner products in $\mathbb{R}^k$ —identical order to classical VQ. The channel model enters only through the $M \times M$ matrix $P(j | i)$, precomputed once.

7.7 Worked example: 1-bit COVQ over a BSC

Take scalar $X \sim \mathcal{N}(0, 1)$, $M = 2$ levels, BSC(ε) with natural labeling. By symmetry the encoder is $\mathcal{S}_1 = (-\infty, t]$, $\mathcal{S}_2 = (t, \infty)$ with $t = 0$ at optimality, and codevectors $\pm y$ with

$$y = \mathbb{E}[X | J = 1] = \frac{(1-\varepsilon)\mathbb{E}[X | X > 0] + \varepsilon\mathbb{E}[X | X < 0]}{(1-\varepsilon)\frac{1}{2} + \varepsilon\frac{1}{2}} = (1-2\varepsilon)\sqrt{\frac{2}{\pi}}. \quad (37)$$

The clean-channel answer would be $y = \sqrt{2/\pi} \approx 0.798$; the noisy channel **shrinks** it by $(1-2\varepsilon)$ —at $\varepsilon = 0.1$, $y \approx 0.638$. The end-to-end distortion is $D = 1 - (1-2\varepsilon)^2 \cdot 2/\pi$. Two lessons in one line: *the decoder hedges toward zero, and the price of noise is quadratic in $(1-2\varepsilon)$* . Every COVQ you train is a high-dimensional version of this shrinkage.

7.8 Tandem vs. joint: why separation loses here

A tandem system (source-optimized VQ + channel code + interleaver) pays three taxes: redundancy bits for the channel code, delay/complexity for interleaving, and—most fundamentally—a **mismatch tax**: the source coder is designed for a noiseless channel it doesn’t have. COVQ

pays none of these; its only tax is that the codebook is channel-specific (retrain if the channel changes substantially—though Chapter 9 shows graceful degradation under mismatch). When delay and complexity budgets are tight—the wireless image scenario—joint design wins, often by many dB.

8 Errors, erasures, and free soft information

Some channels don’t just flip bits—they *erase* them, handing the decoder a flag that says “this bit is unreliable.” Andersson & Skoglund’s COVQ-based image coder shows how to exploit this inside the quantizer, with no extra redundancy.

8.1 The channel model

Each transmitted bit is independently: correct with probability $(1 - \alpha)(1 - \beta)$, flipped with probability $\alpha(1 - \beta)$, erased with probability β . For an L -bit index, the encoder alphabet has 2^L symbols but the decoder observes 3^L **patterns** (each bit $\in \{0, 1, e\}$). The COVQ decoder therefore has 3^L codevectors $\mathbf{y}_j$ —one per erasure pattern—while the encoder still chooses among 2^L indices. The training equations (32)–(34) apply unchanged, with j ranging over the enlarged output alphabet.

8.2 Why erasures are free soft information

A classical hard-decision system treats an erased bit as an error like any other. The COVQ decoder instead learns a *distinct* reconstruction for every erasure pattern: observing $\mathbf{j} = (0, e, 1, \dots)$ it outputs $\mathbb{E}[\mathbf{X} \mid \mathbf{J} = \mathbf{j}]$, which automatically weights the surviving bits more and hedges on the erased positions. The erasure flags act as **reliability side information**—soft decoding—without any change to the encoder or any transmitted redundancy. The authors note the connection to **multiple-description quantization**: each erasure pattern is like receiving a different subset of descriptions, and the decoder is optimized for every subset simultaneously.

8.3 The subband COVQ image coder: architecture and findings

The paper’s image system, in brief:

- 4-level, 13-subband wavelet-style decomposition of the image.
- Vectors formed *within* each subband (exploiting residual spatial correlation that scalar subband quantization leaves behind).
- **Equal-slope** bit allocation across subbands (marginal return per bit equalized—the operational version of Chapter 5’s Lagrangian).
- Fixed-rate indices over the error-and-erasure channel; decoder codebook sized 3^L per vector quantizer.
- Training data modeled as Gaussian mixtures fitted to real images.

Findings that matter for your implementations:

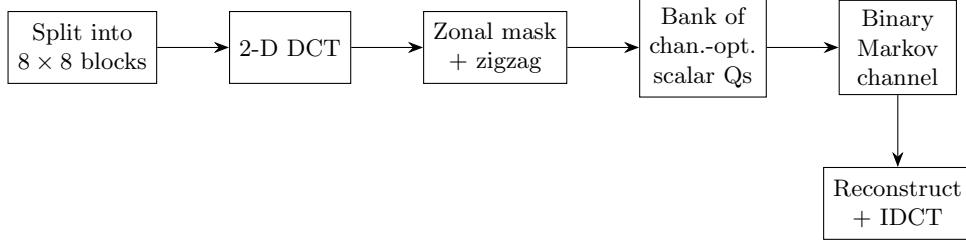
1. **Graceful degradation**: as channel quality worsens, the COVQ system’s PSNR degrades smoothly, while the tandem (BCH-protected) VQ systems exhibit threshold/cliff effects.
2. **Beating tandem**: over the simulated range of (α, β) , the COVQ system outperformed the tested BCH-protected VQ baselines—joint design’s robustness beats separate coding’s fragility when the channel model is imperfect.

3. **Mismatch tolerance:** training at one (α, β) and testing at another costs surprisingly little, because the posterior-mean decoder is inherently conservative.

9 A complete DCT image coder for bursty channels

This chapter distills Cheng’s MSc thesis and Fady’s report: a full, implementable image transmission system for binary channels *with memory*, built from the pieces of Chapters 3–7.

9.1 System pipeline



1. Split the image into 8×8 blocks; apply the 2-D DCT.
2. **Zonal masking:** keep a fixed zigzag-ordered subset of coefficients per block (the low-frequency zone); discard the rest. Fixed zones need no side information and cost only ~ 1 dB versus fully optimal allocation.
3. **Zigzag scan** orders coefficients by increasing spatial frequency, so the kept zone is a prefix of the scan.
4. Each kept coefficient position gets its own **channel-optimized scalar quantizer** (the scalar special case of Chapter 7: thresholds from (33), levels from (34)).
5. Transmit the fixed-length indices over a **binary Markov channel** (first-order, parameterized by BER and correlation).
6. Decoder: table lookup per coefficient, inverse zigzag, IDCT.

9.2 Coefficient statistics

The quantizer design needs a source model per coefficient position:

- **DC coefficient:** approximately **Gaussian** (block means vary smoothly).
- **AC coefficients:** approximately **Laplacian** (mostly near zero, heavy tails).

Fit (μ, σ) per position from training images (or use the high-resolution formulas with the Laplacian/Gaussian shape factors). The Laplacian model is why the optimal point density $f^{1/3}$ matters: it puts fine levels near zero where AC coefficients concentrate.

9.3 Channel-optimized scalar quantizers per coefficient

For coefficient position p with b_p bits ($N_p = 2^{b_p}$ levels) over the binary Markov channel: the transition matrix $P(j | i)$ for b_p -bit indices is built from the bit-level Markov model (marginalizing the chain over b_p consecutive bits—a small dynamic program, or Monte Carlo for large b_p). Then run Listing 2 in its scalar form ($k = 1$). Because the channel has memory, $P(j | i)$ is *not* a simple function of Hamming distance—keep the full matrix.

9.4 Fixed vs. optimal bit allocation

The thesis compares the fixed zonal table against fully optimal (image- and channel-adaptive) allocation: the loss is only about **1 dB** in the studied cases, while the fixed table eliminates all side information and adaptation logic. *Engineering judgment*: take the 1 dB hit and keep the system simple, unless you are operating at the absolute rate–distortion frontier.

9.5 What the experiments showed

Headline numbers (1.19 bpp, noisy *and* highly correlated binary Markov channel):

- vs. **source-optimized** scalar quantization (no channel awareness): gains up to **~5 dB** (Baboon) and **~12 dB** (Lena). The 12 dB figure is not a typo: under severe burst noise, a channel-blind quantizer’s indices are scrambled into garbage, while the channel-optimized one degrades gracefully. Gains are largest exactly where the channel is worst.
- Fixed allocation $\approx$ 1 dB below optimal allocation.
- **Mismatch** (design channel $\neq$ actual channel): degradation is smooth and modest—the system does not fall off a cliff when the BER/correlation estimates are off.

The thesis also reports COVQ (vector) variants of the same system with further gains, at higher training and encoding complexity—the standard rate–distortion–complexity tradeoff.

10 Implementation blueprint

This chapter turns the theory into a build plan. It is written as if you are about to implement the DCT image coder of Chapter 9 (with the COVQ core of Chapter 7) in Python/NumPy or MATLAB, and want to avoid the standard traps.

10.1 Software architecture

Separate four concerns into four modules:

Module	Responsibility
<code>source_model</code>	Load images; DCT blocks; fit per-coefficient (μ, σ) or histograms; produce training vectors
<code>channel</code>	Sample error traces; build $P(j i)$ matrices; exact block probabilities (QBC/FMCC) or Monte Carlo
<code>quantizer</code>	Lloyd–Max / LBG / COVQ training (Listings 1, 2); bit allocation
<code>evaluate</code>	End-to-end simulation; PSNR/SNR; sweeps over (BER, correlation); mismatch tests; plots

The golden rule: **the channel module is shared** between training (to build $P(j | i)$) and evaluation (to corrupt indices). If training and testing use different channel code paths, your “mismatch” experiments are measuring a bug, not robustness.

10.2 Training the codebooks

1. **Data.** Use a diverse training set (natural images at the target resolution). For DCT coefficients, a few hundred thousand 8×8 blocks is plenty for scalar quantizers; vector quantizers need more (rule of thumb: ≥ 50 –100 samples per codevector per dimension—below that, centroids are noise).

2. **Initialization.** Splitting (Section 4.3) from the global centroid; for COVQ, warm-start from the clean-channel codebook, then run channel-aware iterations. Never random-init the final reported run.
3. **Convergence.** Stop on relative distortion gain $< 10^{-4}$ – 10^{-5} *and* a minimum iteration count (e.g. 20); cap at a maximum (e.g. 500). Log the distortion trace—a healthy run drops fast then plateaus; oscillation means a bug (usually stale effective codevectors or a sign error in the discriminant).
4. **Channel matrix.** For b -bit indices over a binary Markov channel, compute $P(j | i)$ exactly by running the chain (2^b states $\times b$ steps is trivial for $b \leq 8$); for larger b use Monte Carlo with enough trials that each row sums to 1 within 10^{-6} . Normalize rows explicitly—a row summing to 0.999 will poison the decoder update.
5. **Restarts.** Train ≥ 3 times with perturbed initializations; report the best *training* distortion but evaluate all on held-out data. If restarts disagree wildly, your M is too large for your data.

10.3 Simulating the channel

```

1 function MarkovNoise(n, Pi, pi0):
2     # Pi: 2x2 transition matrix; pi0: stationary initial distribution
3     s = (rand() < pi0[1]) ? 1 : 0
4     z = zeros(n)
5     for t = 1..n:
6         z[t] = s
7         s = (rand() < Pi[s][1]) ? 1 : 0
8     return z           # XOR with transmitted bits

```

Listing 3: Binary Markov noise simulator.

For the QBC or finite-memory Pólya channel, implement the urn/queue update literally—it is a few lines and guarantees the probabilities match the theory. **Seed your RNG** and log the seed with every result; rerun critical numbers with 3+ seeds and report mean $\pm$ std. A single-seed PSNR comparison is not a result.

10.4 Encoder/decoder runtime

- **Encoder:** precompute $(\tilde{y}_i, b_i)$ after training; per input, M dot products + argmax. For scalar quantizers this collapses to threshold comparisons—precompute sorted thresholds and use binary search.
- **Decoder:** pure table lookup: received index/pattern $\rightarrow$ codevector. For the erasure channel, the table has 3^L entries per quantizer—fine for $L \leq 8$ (6561 entries); for larger L , compute $\mathbb{E}[\mathbf{X} | \mathbf{j}]$ on the fly from the surviving bits or prune rare patterns.
- **Fixed point:** if deploying to integer arithmetic, re-verify distortion after quantizing the codebook itself—a surprising number of “mystery losses” come from float $\rightarrow$ int conversion of thresholds.

10.5 Numerical safeguards checklist

- Work in **log-domain** for probabilities (block probabilities underflow fast: $0.9^{1000} \approx 10^{-46}$).
- Guard every division by a cell probability or $P(J = j)$: `den = max(den, tiny)` with a documented fallback.

- Empty cells: reinitialize, never silently keep stale codevectors.
- Verify $\sum_j P(j | i) = 1$ for all i after building the channel matrix.
- Sanity-check against closed forms: 1-bit Gaussian COVQ over BSC (Section 7.7); uniform-quantizer $\Delta^2/12$; 6 dB/bit on a clean channel; KLT gain formula on synthetic Gaussian data. If these don't match, the framework is broken—don't trust any image result until they do.
- Monitor effective codebook size (nonempty cells). A sudden collapse (e.g. $256 \rightarrow 12$) usually means the channel matrix is wrong (check row normalization and label ordering), not that the theory is broken.

10.6 Evaluation methodology

1. **Metrics:** PSNR on standard test images (Lena, Baboon, Peppers, plus images *not* in training); report rate in bpp. Also report SNR for synthetic sources where closed forms exist.
2. **Baselines (always include):** (a) source-optimized quantizer + no channel coding (shows the price of channel blindness); (b) tandem: source-optimized quantizer + BCH/Reed–Solomon + interleaver at matched total rate (the honest competitor); (c) clean-channel upper bound (no noise).
3. **Sweeps:** PSNR vs. BER at fixed correlation; PSNR vs. correlation at fixed BER; PSNR vs. rate. Plot curves, not single points.
4. **Ablations:** fixed vs. optimal bit allocation; natural vs. optimized index assignment; scalar vs. vector; with/without erasure exploitation. Each ablation answers “which piece earned the dBs?”
5. **Mismatch tests:** train at (BER_0, ρ_0) , test on a grid around it. Report the PSNR surface—flat is good.
6. **Statistics:** ≥ 3 RNG seeds; mean $\pm$ std; never tune on the test set.

10.7 Complexity budgeting

Stage	Scalar COVQ	Vector COVQ (k dims)	Notes
Training (one iter.)	$O(TN)$	$O(TMk + M^2k)$	$T = \#$ samples
Channel matrix build	$O(N^2b)$	$O(M^2Lb)$	once; $L =$ index bits
Encoding (per input)	$O(N)$ / bsearch	$O(Mk)$	inner products
Decoding (per index)	$O(1)$ lookup	$O(1)$ lookup	tables
Memory	$O(N)$	$O(Mk + 3^L)$	erasure tables

The practical message: training is the expensive part and happens offline; runtime is table lookups and dot products—this is why COVQ is deployable on modest hardware.

11 What the research showed

One-paragraph verdicts, paper by paper—what was proved or measured, and what it means for you.

Scalar quantization notes (Linder). The complete high-resolution theory: Lloyd–Max conditions, the $f^{1/3}$ optimal point density, and the 6 dB/bit rule. *Use it as:* the correctness baseline for every quantizer you implement, and the source of the bit-allocation formulas.

Transform coding notes (Linder). KLT definition and optimality proof, DCT properties and fast computation, the Lagrangian bit-allocation solution, JPEG and subband/wavelet overviews. *Use it as:* the reference for why your transform stage looks the way it does, and for the $b_i = \bar{b} + \frac{1}{2} \log_2(\sigma_i^2/\rho^2) + \dots$ formula.

Farvardin & Vaishampayan: COVQ performance & complexity. The end-to-end distortion objective, the channel-aware encoder and posterior-mean decoder conditions, and—crucially—the effective-codevector implementation making COVQ encoding as cheap as classical VQ. Plus the sobering geometry: noisy channels empty out most encoding regions (61/256 in their Gauss–Markov example). *Use it as:* the theoretical core of any COVQ implementation.

Andersson & Skoglund: COVQ image coder, bit errors & erasures. Erasure flags become free soft information via a 3^L -entry decoder; the subband COVQ image coder degrades gracefully and beats BCH-protected tandem VQ over the tested range, with good mismatch tolerance. *Use it as:* the template for handling erasures, and evidence that joint design beats tandem when the channel is nasty.

Cheng (MSc thesis, 1997) & Fady’s report: DCT coder over bursty channels. The full system: 8×8 DCT, zonal masking, zigzag, per-coefficient channel-optimized scalar quantizers over a binary Markov channel. Fixed allocation ≈ 1 dB from optimal; gains up to ~ 5 dB (Baboon) / ~ 12 dB (Lena) at 1.19 bpp under severe burst noise; smooth mismatch behavior. *Use it as:* the end-to-end recipe to replicate first.

Alajaji & Fuja: Pólya-contagion channel (1994). A stationary *nonergodic* bursty channel: ML decoding picks the codeword farthest from d_0 (nearest *or* farthest); capacity is exactly 0 (averaged BSCs with beta-distributed crossover); the finite-memory version is ergodic with closed-form capacity increasing in M . *Use it as:* the deep model of burst noise, and a warning that nonergodic fading kills capacity no matter the code.

Zhong, Alajaji & Takahara: queue-based channels (2007). Stationary ergodic M th-order Markov noise with transparent block probabilities and computable capacity $1 - \bar{H}(Z)$; fit to Gilbert–Elliott targets by BER/correlation matching; uniform QBC $\equiv$ finite-memory Pólya channel. *Use it as:* your default bursty-noise simulator and fitter.

Song, Alajaji & Linder: noise-erasure channels & feedback. Capacity $C = (1 - \varepsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})]$ with uniform inputs optimal; feedback doesn’t raise C but can raise the capacity–cost function under input costs. *Use it as:* the theory behind erasure channels and the reason to consider feedback when you have a cost budget, not when you want raw capacity.

12 Extensions and open directions

- **Feedback-driven COVQ.** The NEC result says feedback helps under cost constraints—design a COVQ whose encoder adapts its index assignment or bit allocation from ACK/NAK or channel-state feedback.

- **Soft-decision decoding.** Replace hard/erased bits with LLRs from the demodulator; the posterior-mean decoder (34) generalizes naturally to $\mathbf{y}(\mathbf{l}) = \mathbb{E}[\mathbf{X} \mid \mathbf{L} = \mathbf{l}]$.
- **Learned JSCC.** Modern deep JSCC (autoencoder-style) rediscovers COVQ’s lesson end-to-end; the classical theory here is the right baseline and the right source of inductive biases (posterior-mean decoders, channel-aware encoders).
- **Unequal error protection by design.** COVQ already protects important codevectors more (via index assignment and cell geometry); make it explicit with hierarchical indices.
- **Progressive/resilient streaming.** Order coefficients by the bit-allocation priority and marry the fixed zonal table to a packetization that degrades gracefully—the erasure-channel results suggest the decoder side is already halfway there.
- **Better channel fitting.** Fit QBC/FMCC parameters to *measured* modem error traces (not just BER/correlation—match burst-length distributions), then retrain the COVQ on the fitted model.

A Key formulas cheat sheet

$$\begin{aligned}
\text{MSE distortion: } D &= \mathbb{E}[(X - Q(X))^2], \quad \text{SNR} = 10 \log_{10}(\mathbb{E}[X^2]/D) \\
\text{Uniform quantizer: } D &\approx \Delta^2/12 \\
\text{High-res optimum: } D^*(N) &\approx \frac{1}{12N^2} \left(\int f^{1/3} \right)^3, \quad G'(x) \propto f(x)^{1/3} \\
6 \text{ dB/bit: } \text{SNR} &\approx 6.02R + \text{const} \\
\text{Bit allocation: } b_i &= \bar{b} + \frac{1}{2} \log_2(\sigma_i^2/\rho^2) + \frac{1}{2} \log_2(h_i/H) \\
\text{KLT gain: } D_{\text{PCM}}/D_{\text{tc}} &= \left(\frac{1}{k} \sum \lambda_i \right) / \left(\prod \lambda_i \right)^{1/k} \\
\text{COVQ encoder: } \mathcal{S}_i &= \{ \mathbf{x} : \sum_j P(j|i) \|\mathbf{x} - \mathbf{y}_j\|^2 \text{ minimal} \} \\
\text{COVQ decoder: } \mathbf{y}_j &= \mathbb{E}[\mathbf{X} \mid J = j] = \frac{\sum_i P(I=i)P(j|i)\mathbf{c}_i}{\sum_i P(I=i)P(j|i)} \\
\text{Fast encode: } i^* &= \arg \max_i \{ \mathbf{x}^t \tilde{\mathbf{y}}_i - \frac{1}{2} b_i \}, \quad \tilde{\mathbf{y}}_i = \sum_j P(j|i) \mathbf{y}_j \\
\text{Binary additive capacity: } C &= 1 - \bar{H}(Z) \\
\text{NEC capacity: } C &= (1 - \varepsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})] \\
\text{FMCC capacity: } C_M &= 1 - \sum_{k=0}^M \binom{M}{k} L_k h_b \left(\frac{\rho + k\delta}{1 + M\delta} \right) \\
\text{Pólya ML threshold: } d_0 &= \frac{n}{2} + \frac{1-2\rho}{2\delta}
\end{aligned}$$

Annotated bibliography

1. T. Linder, *Data Compression and Source Coding: Scalar Quantization*, lecture notes, Queen’s University. Lloyd–Max conditions, training-set implementation, uniform quantization, companding, the $f^{1/3}$ optimal point density (with full Hölder-inequality proof), Bucklew–Wise theorem, 6 dB/bit rule.
2. T. Linder, *Data Compression and Source Coding: Transform Coding*, lecture notes, Queen’s University. Bit-allocation problem and Lagrangian solution, orthogonal-transform linear algebra, KLT construction and high-resolution optimality (Hadamard’s inequality), DCT def-

inition and FFT-based computation, 2-D separable transforms, JPEG, subband and wavelet coding.

3. J. Cheng, *Channel Optimized Quantization of Images over Binary Channels with Memory*, M.Sc. thesis, Dept. of Mathematics and Statistics, Queen's University, Kingston, ON, Sep. 1997. COVQ design algorithm and initialization, transform coding with optimal bit allocation, the DCT image transmission system, fixed-bit-allocation tables, mismatch studies, and COVQ experimental results.
4. N. Farvardin and V. Vaishampayan, "On the performance and complexity of channel-optimized vector quantizers." End-to-end distortion objective; necessary encoder/decoder conditions; efficient implementation via effective codevectors; complexity analysis; the empty-encoding-region phenomenon (61/256 nonempty for Gauss–Markov, $k = 8$, $M = 256$, BSC 0.1).
5. T. Andersson and M. Skoglund, "A COVQ-based image coder for channels with bit errors and erasures." Binary symmetric erasure channel model; 3^L -pattern decoder codebook; erasures as soft information; multiple-description interpretation; 4-level 13-subband COVQ image coder with equal-slope allocation; graceful degradation and superiority over BCH-protected tandem VQ.
6. F. Alajaji and T. Fuja, "A communication channel modeled on contagion," *IEEE Trans. Inform. Theory*, vol. 40, no. 6, pp. 2035–2041, Nov. 1994. Pólya-urn additive-noise channel: stationarity without ergodicity, exchangeability, correlation $\delta/(1 + \delta)$; ML decoding as nearest-or-farthest with threshold d_0 ; averaged-channel representation via De Finetti; zero capacity and ϵ -capacity; finite-memory version with closed-form capacity C_M increasing in M .
7. Z. Zhong, F. Alajaji, and T. Takahara, "Queue-based channels" (2007). Finite-queue-plus-urn M th-order Markov noise; stationary ergodic binary additive channel; transparent block probabilities; capacity $1 - \bar{H}(Z)$; approximation of Gilbert–Elliott channels by BER/correlation matching with minimum KL-divergence rate; equivalence of the uniform QBC with the finite-memory Pólya channel.
8. Y. Song, F. Alajaji, and T. Linder, noise-erasure channels with feedback (2019). NEC model $Y_i = h(X_i, Z_i) / e$; quasi-symmetry and uniform-input optimality; capacity $(1 - \epsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})]$; $C_{\text{FB}} = C$ unconstrained but feedback improves the capacity–cost function under input costs; recursive entropy-rate bounds for hidden-Markov erasure indicators.
9. Fady's report: DCT-based image coder over bursty binary Markov channels. 8×8 DCT, zonal masking, zigzag scan, bank of channel-optimized scalar quantizers (Gaussian DC / Laplacian AC models); fixed vs. optimal bit allocation (~ 1 dB gap); gains up to ~ 5 dB (Baboon) / ~ 12 dB (Lena) at 1.19 bpp under noisy, highly correlated channels; mismatch robustness.

Design for the channel you have—not the channel you wish you had.

Part III — Formula Companion: Every Derivation Earned

`formula-companion.pdf` (16 pages)

The companion to Part II, meant to be read interleaved with it: every major formula of the core theory derived from first principles, with figures. Lloyd–Max conditions, the $\Delta^2/12$ quantization-noise rule, the $f^{1/3}$ point density (note the Hölder correction in this volume’s errata), the 6.02 dB/bit rule, optimal bit allocation, KLT/Hadamard optimality, the COVQ conditions and their linearization, the one-bit Gaussian COVQ shrinkage, Pólya decoding, and the noise-erasure channel capacity. If Part II tells you what is true, this part shows you why it must be true. Read a section here each time Part II asks you to take a formula on faith.

The Formulas Behind Joint Source–Channel Coding

Derivations, Intuition, and Visualizations

Companion to *Joint Source–Channel Coding for Image Transmission over Noisy Channels with Memory*

September 2026

Abstract

This companion takes every key formula in the main guide and answers three questions about it: *what* does it say, *where* does it come from (full derivation, no hand-waving), and *what* does each symbol mean. Every section ends with the intuition you should carry into implementation, and every derivation is paired with a figure that lets you *see* the mathematics working. Notation matches the main guide exactly.

Contents

1	The Lloyd–Max conditions	2
2	The $\Delta^2/12$ law	3
3	Optimal point density: why $f^{1/3}$	4
4	The 6 dB per bit rule	6
5	Optimal bit allocation	6
6	KLT optimality and the transform coding gain	8
7	COVQ: the channel-aware encoder and the posterior-mean decoder	10
8	Worked example: 1-bit Gaussian COVQ over a BSC	12
9	Pólya ML decoding: the d_0 threshold	12
10	Capacity: $C = 1 - \bar{H}(Z)$ and the NEC formula	14

1 The Lloyd–Max conditions

What the formulas say. For an N -level scalar quantizer with thresholds $t_0 < t_1 < \dots < t_N$ ($t_0 = -\infty$, $t_N = +\infty$) and reconstruction levels $y_1, \dots, y_N$, the MSE-optimal choice satisfies two coupled conditions:

$$t_i = \frac{y_i + y_{i+1}}{2} \quad (\text{nearest neighbor}), \quad (1)$$

$$y_i = \frac{\int_{t_{i-1}}^{t_i} x f(x) dx}{\int_{t_{i-1}}^{t_i} f(x) dx} = \mathbb{E}[X \mid t_{i-1} < X \leq t_i] \quad (\text{centroid}). \quad (2)$$

Where they come from. Write the distortion as a function of all $2N - 1$ free parameters:

$$D(\mathbf{t}, \mathbf{y}) = \sum_{i=1}^N \int_{t_{i-1}}^{t_i} (x - y_i)^2 f(x) dx. \quad (3)$$

Centroid condition. Differentiate under the integral sign with respect to y_i (only cell i depends on y_i):

$$\frac{\partial D}{\partial y_i} = \int_{t_{i-1}}^{t_i} 2(x - y_i)(-1) f(x) dx = -2 \int_{t_{i-1}}^{t_i} (x - y_i) f(x) dx. \quad (4)$$

Setting this to zero, $\int_{t_{i-1}}^{t_i} x f(x) dx = y_i \int_{t_{i-1}}^{t_i} f(x) dx$, which is exactly (2): each level must be the center of mass of its own cell.

Nearest-neighbor condition. The threshold t_i appears twice in (3): as the upper limit of cell i and the lower limit of cell $i + 1$. By Leibniz's rule,

$$\begin{aligned} \frac{dD}{dt_i} &= \underbrace{(t_i - y_i)^2 f(t_i)}_{\text{top of cell } i} + \underbrace{(-(t_i - y_{i+1})^2 f(t_i))}_{\text{bottom of cell } i+1} \\ &= f(t_i) [(t_i - y_i)^2 - (t_i - y_{i+1})^2]. \end{aligned} \quad (5)$$

Setting this to zero (and assuming $f(t_i) > 0$) gives $(t_i - y_i)^2 = (t_i - y_{i+1})^2$. The root $y_i = y_{i+1}$ is degenerate (two identical levels), so we take the other root:

$$t_i - y_i = -(t_i - y_{i+1}) \implies t_i = \frac{y_i + y_{i+1}}{2}. \quad (6)$$

Each threshold sits exactly halfway between its neighboring levels: every x is assigned to its nearest level.

Symbol glossary.

N	Number of quantization levels (rate $R = \log_2 N$ bits).
t_i	Decision thresholds; $t_0 = -\infty$, $t_N = +\infty$ fixed.
y_i	Reconstruction (output) levels.
$f(x)$	Source probability density.
D	Mean squared error $\mathbb{E}[(X - Q(X))^2]$.

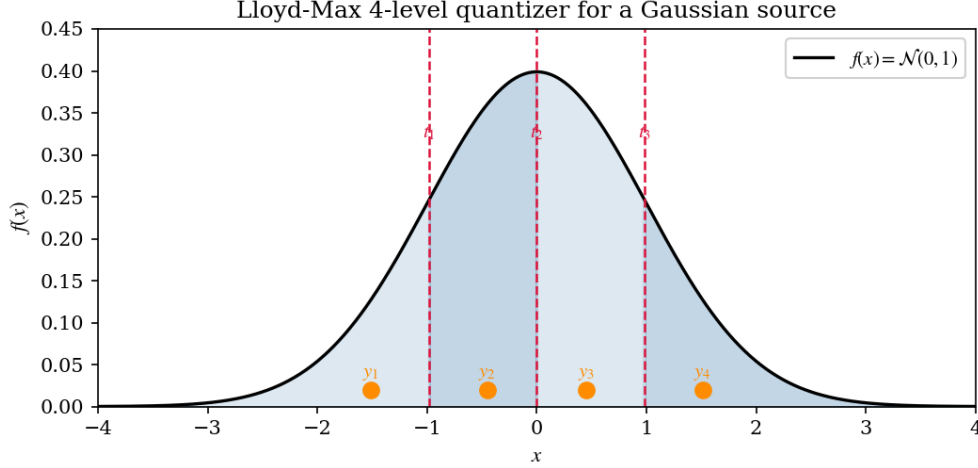


Figure 1: The two Lloyd–Max conditions in action on $\mathcal{N}(0, 1)$: thresholds t_i (dashed) at midpoints of levels y_i (dots), each y_i the centroid of its shaded cell. Levels crowd near $x = 0$ where the mass is—a preview of Section 3.

Intuition. The two conditions describe two players taking turns: the *partitioner* assigns each x to the nearest level (1), then the *centroider* moves each level to its cell’s center of mass (2). Each move can only lower D , so alternating them converges—but only to a *local* optimum (a fixed point of the pair). In code, this is the **repeat** loop of Lloyd–Max: nearest-neighbor assignment, then centroid update, until the distortion stops dropping.

2 The $\Delta^2/12$ law

What the formula says. A uniform quantizer with step size Δ has MSE

$$D \approx \frac{\Delta^2}{12}, \quad (7)$$

in the high-resolution limit (many levels, small Δ).

Where it comes from. Cell i is the interval $[y_i - \Delta/2, y_i + \Delta/2]$. Substitute $u = x - y_i$ in the distortion integral:

$$D = \sum_{i=1}^N \int_{-\Delta/2}^{\Delta/2} u^2 f(y_i + u) du. \quad (8)$$

The high-resolution assumption: Δ is small enough that f is nearly constant across one cell, $f(y_i + u) \approx f(y_i)$. Then

$$\int_{-\Delta/2}^{\Delta/2} u^2 f(y_i) du = f(y_i) \left[\frac{u^3}{3} \right]_{-\Delta/2}^{\Delta/2} = f(y_i) \frac{\Delta^3}{12}. \quad (9)$$

Summing over cells,

$$D \approx \frac{\Delta^2}{12} \sum_{i=1}^N f(y_i) \Delta \approx \frac{\Delta^2}{12} \int_{-\infty}^{\infty} f(x) dx = \frac{\Delta^2}{12}, \quad (10)$$

where $\sum_i f(y_i) \Delta$ is a Riemann sum for $\int f = 1$, using $f(y_i) \Delta \approx \mathbb{P}(X \text{ in cell } i)$.

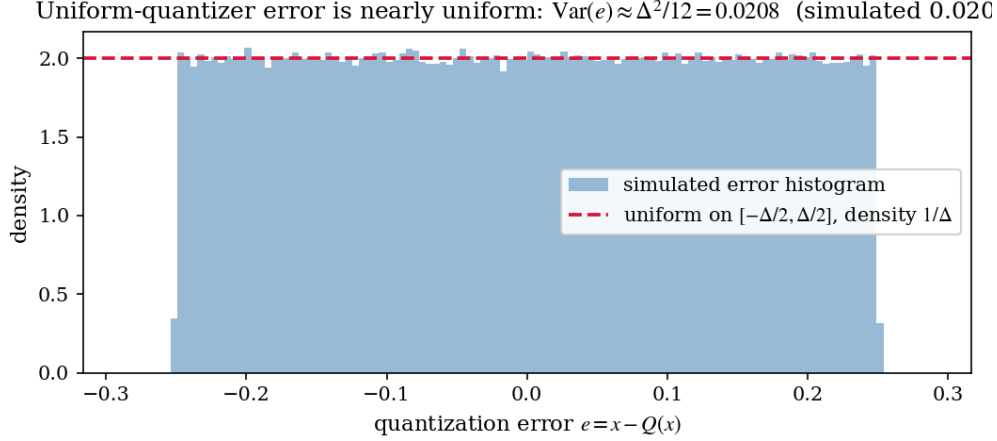


Figure 2: Simulated quantization error $e = X - Q(X)$ for a uniform quantizer ($\Delta = 0.5$) on Gaussian input: nearly uniform on $[-\Delta/2, \Delta/2]$. The variance of that uniform law is $\Delta^2/12$ —the entire content of the formula.

Symbol glossary.

Δ Quantizer step size (cell width); $N \approx \text{range}/\Delta$ levels.

$u = x - y_i$ Error within a cell, ranging over $[-\Delta/2, \Delta/2]$.

$f(y_i)\Delta$ Approximate probability of cell i (area of a thin rectangle under f).

Intuition. Inside one tiny cell the source looks uniform, so the error looks uniform on $[-\Delta/2, \Delta/2]$ —and the variance of a uniform $[a, b]$ variable is $(b - a)^2/12$. That is the whole story: $1/12$ is not a property of quantization, it is a property of the uniform distribution. Two caveats for implementers: (i) it ignores *overload* distortion (the unbounded outer cells), so it needs the range to cover the source’s effective support; (ii) it is the per-cell engine inside every high-resolution result that follows (Sections 3–4).

3 Optimal point density: why $f^{1/3}$

What the formula says. Let $\lambda(x) \geq 0$, $\int \lambda = 1$, be the *point density*: fraction of quantization levels per unit x . The distortion-minimizing density and the resulting minimum distortion are

$$\lambda^*(x) = \frac{f(x)^{1/3}}{\int f^{1/3}}, \quad D^*(N) \approx \frac{1}{12N^2} \left(\int_{-\infty}^{\infty} f(x)^{1/3} dx \right)^3. \quad (11)$$

Where it comes from. *Step 1: distortion in terms of λ .* Use a compander: map $x \mapsto y = G(x)$ with $G : \mathbb{R} \rightarrow [0, 1]$ strictly increasing, then apply a uniform N -level quantizer to y . The local x -step is $\Delta(x) = 1/(N G'(x))$ (one y -step of $1/N$ corresponds to $dx = dy/G'(x)$). The point density is $\lambda(x) = G'(x)$: levels per unit x , integrating to $G(\infty) - G(-\infty) = 1$. Applying $\Delta^2/12$ locally,

$$D \approx \int_{-\infty}^{\infty} f(x) \frac{\Delta(x)^2}{12} dx = \frac{1}{12N^2} \int_{-\infty}^{\infty} \frac{f(x)}{\lambda(x)^2} dx. \quad (12)$$

Step 2: the optimization problem. Minimize $J(\lambda) = \int f \lambda^{-2}$ subject to $\int \lambda = 1$, $\lambda \geq 0$.

Step 3: Hölder's inequality. Split the integrand cleverly:

$$f(x)^{1/3} = \underbrace{(f(x)\lambda(x)^{-2})^{1/3}}_{u(x)} \cdot \underbrace{\lambda(x)^{2/3}}_{v(x)}. \quad (13)$$

Hölder with exponents $p = 3$, $q = 3/2$ ($1/p + 1/q = 1$) gives

$$\int uv \leq \left(\int u^3 \right)^{1/3} \left(\int v^{3/2} \right)^{2/3}, \quad (14)$$

i.e.

$$\int f^{1/3} \leq \left(\int f\lambda^{-2} \right)^{1/3} \left(\int \lambda \right)^{2/3}. \quad (15)$$

Cubing and using $\int \lambda = 1$:

$$\left(\int f^{1/3} \right)^3 \leq \int f\lambda^{-2} = J(\lambda). \quad (16)$$

Step 4: equality condition. Hölder is an equality iff $u^3 \propto v^{3/2}$, i.e. $f\lambda^{-2} \propto \lambda^2$, i.e.

$$\lambda(x)^3 \propto f(x) \implies \lambda^*(x) \propto f(x)^{1/3}. \quad (17)$$

Normalizing ($\int \lambda^* = 1$) and substituting into (12) yields (11).

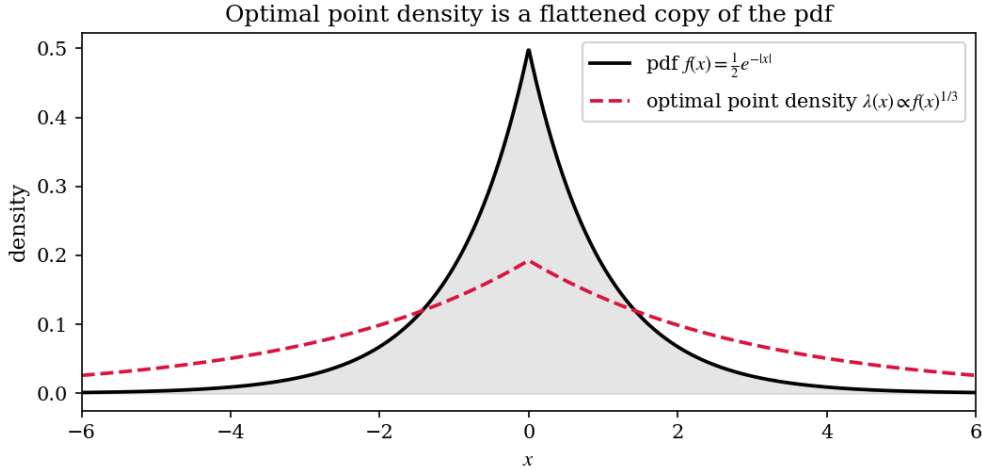


Figure 3: For a Laplacian pdf, the optimal point density $\lambda^* \propto f^{1/3}$ (dashed) is a *flattened* copy of f : levels concentrate where the mass is, but the cube root spreads them into the tails relative to f itself.

Symbol glossary.

$\lambda(x)$ Point density: levels per unit x near x .

$G(x)$ Compressor: increasing map $\mathbb{R} \rightarrow [0, 1]$; $\lambda = G'$.

$\Delta(x) = 1/(N\lambda(x))$
Local cell width at x .

$J(\lambda)$ Distortion functional being minimized.

$p = 3, q = 3/2$
Hölder conjugate exponents used to isolate $\int f^{1/3}$.

Intuition. Why a cube root and not f itself? The distortion contributed near x is $f(x)/\lambda(x)^2$: doubling the local density quarters the local distortion but *costs* density budget everywhere. The optimizer equalizes the *marginal* return of density across x , and the $1/\lambda^2$ dependence softens the response to f from linear to cube-root. Practical reading: put fine cells where the probability mass is, but not *as* aggressively as the mass dictates—tails deserve more levels than f alone suggests. For a Gaussian, $\int f^{1/3} = (2\pi\sigma^2)^{-1/6}(6\pi\sigma^2)^{1/2} \approx 3.198\sigma^{2/3}$, giving $D^* \approx 2.725\sigma^2/N^2$.

4 The 6 dB per bit rule

What the formula says. For an R -bit quantizer ($N = 2^R$ levels),

$$\text{SNR} \approx 6.02 R + \text{const} \quad (\text{dB}), \quad (18)$$

where the constant depends on the source shape but the slope 6.02 does not.

Where it comes from. Insert $N = 2^R$ into the high-resolution distortion $D^*(N) \approx C_0/N^2$ (with $C_0 = \frac{1}{12}(\int f^{1/3})^3$ from (11)):

$$\text{SNR} = 10 \log_{10} \frac{\sigma^2}{D^*} \approx 10 \log_{10} \frac{\sigma^2}{C_0} + 10 \log_{10}(2^{2R}). \quad (19)$$

The rate-dependent term is

$$10 \log_{10}(2^{2R}) = 20R \log_{10} 2 = 6.0206 R, \quad (20)$$

and everything else folds into $\text{const} = 10 \log_{10}(\sigma^2/C_0)$.

Why 6.02, exactly. One extra bit halves the step size: $\Delta \rightarrow \Delta/2$. By $\Delta^2/12$, distortion is quartered: $D \rightarrow D/4$. A factor of 4 in power is $10 \log_{10} 4 = 20 \log_{10} 2 = 6.0206$ dB. That is the entire origin of the number: *bits halve Δ ; squared error pays quadratically*.

Two worked constants.

Uniform source, unit variance. Exact (not asymptotic): $\Delta = A2^{-R}$, $\sigma^2 = A^2/12$, $D = \Delta^2/12$, so $\text{SNR} = 10 \log_{10}(2^{2R}) = 6.02R$ dB: $\text{const} = 0$.

Gaussian source. $C_0 = 2.725\sigma^2$, so $\text{const} = 10 \log_{10}(1/2.725) = -4.35$ dB: $\text{SNR} \approx 6.02R - 4.35$ dB.

Intuition. Every bit buys the same 6 dB because quantization error is a *relative* game: what matters is Δ relative to the signal scale, and bits control $\log \Delta$ linearly. Use it as a sanity check: if your measured SNR-vs-rate slope is far from 6 dB/bit, you are not in the high-resolution regime (too few levels, overload distortion, or a bug)—not witnessing a miracle.

5 Optimal bit allocation

What the formula says. For k coefficients with variances σ_i^2 and shape factors h_i , distributing B bits to minimize $\sum_i W_i(b_i)$ gives

$$b_i = \bar{b} + \frac{1}{2} \log_2 \frac{\sigma_i^2}{\rho^2} + \frac{1}{2} \log_2 \frac{h_i}{H}, \quad \bar{b} = \frac{B}{k}, \quad \rho^2 = \left(\prod_{i=1}^k \sigma_i^2 \right)^{1/k}, \quad H = \left(\prod_{i=1}^k h_i \right)^{1/k}, \quad (21)$$

with minimum distortion $D_{\text{opt}} = kH\rho^2 2^{-2\bar{b}}$.

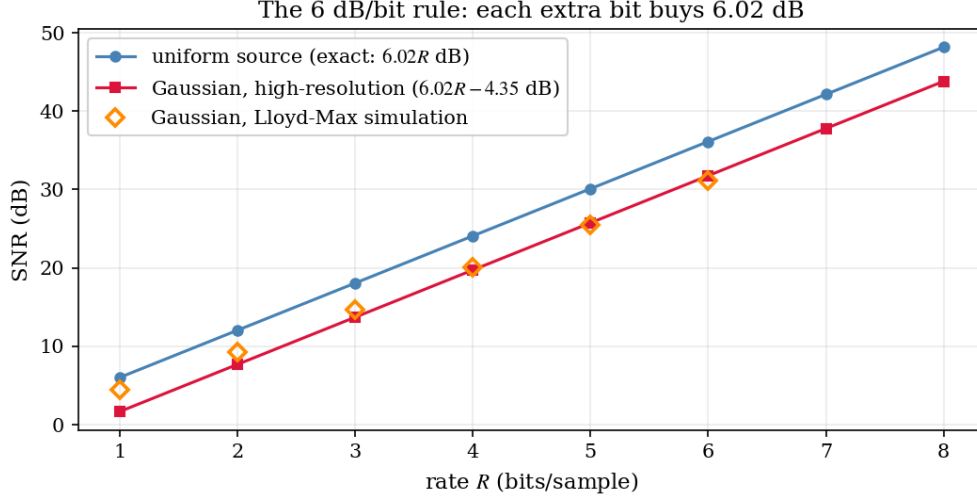


Figure 4: Measured SNR vs. rate: the uniform source follows $6.02R$ exactly; the Gaussian follows $6.02R - 4.35$ from $R \geq 3$, with Lloyd–Max simulations (diamonds) confirming the high-resolution line. Slope is universal; only the offset is source-dependent.

Where it comes from. *Step 1: the per-coefficient model.* From (11), the optimal b -bit distortion for a density with variance σ_i^2 and shape h_i is $W_i(b) = h_i \sigma_i^2 2^{-2b}$ (the shape factor h_i absorbs $(\int f_i^{1/3})^3/12$ after factoring out scale).

Step 2: Lagrangian. Minimize $D(\mathbf{b}) = \sum_i h_i \sigma_i^2 2^{-2b_i}$ subject to $\sum_i b_i = B$:

$$J(\mathbf{b}, \lambda) = \sum_{i=1}^k h_i \sigma_i^2 2^{-2b_i} + \lambda \left(\sum_{i=1}^k b_i - B \right). \quad (22)$$

Step 3: first-order condition.

$$\frac{\partial J}{\partial b_j} = h_j \sigma_j^2 \cdot 2^{-2b_j} (-2 \ln 2) + \lambda = 0 \implies 2^{-2b_j} = \frac{\lambda}{2 \ln 2 \cdot h_j \sigma_j^2}. \quad (23)$$

Take $\log_2$ of both sides:

$$-2b_j = \log_2 \left(\frac{\lambda}{2 \ln 2} \right) - \log_2(h_j \sigma_j^2), \quad (24)$$

so with $\mu = -\frac{1}{2} \log_2(\lambda/(2 \ln 2))$,

$$b_j = \frac{1}{2} \log_2(h_j \sigma_j^2) + \mu. \quad (25)$$

Step 4: enforce the constraint. Summing (25): $B = \frac{1}{2} \sum_j \log_2(h_j \sigma_j^2) + k\mu$, hence

$$\mu = \frac{B}{k} - \frac{1}{2k} \sum_{j=1}^k \log_2(h_j \sigma_j^2) = \bar{b} - \frac{1}{2} \log_2 \left[\left(\prod_j h_j \sigma_j^2 \right)^{1/k} \right] = \bar{b} - \frac{1}{2} \log_2(H \rho^2). \quad (26)$$

Substituting back into (25):

$$b_j = \bar{b} + \frac{1}{2} \log_2(h_j \sigma_j^2) - \frac{1}{2} \log_2(H \rho^2) = \bar{b} + \frac{1}{2} \log_2 \frac{\sigma_j^2}{\rho^2} + \frac{1}{2} \log_2 \frac{h_j}{H}, \quad (27)$$

which is (21). Strict convexity of D (sum of convex exponentials) makes the stationary point the global minimum. The minimum value follows by substitution: $D_{\text{opt}} = \sum_i h_i \sigma_i^2 2^{-2b_i} = k H \rho^2 2^{-2\bar{b}}$.

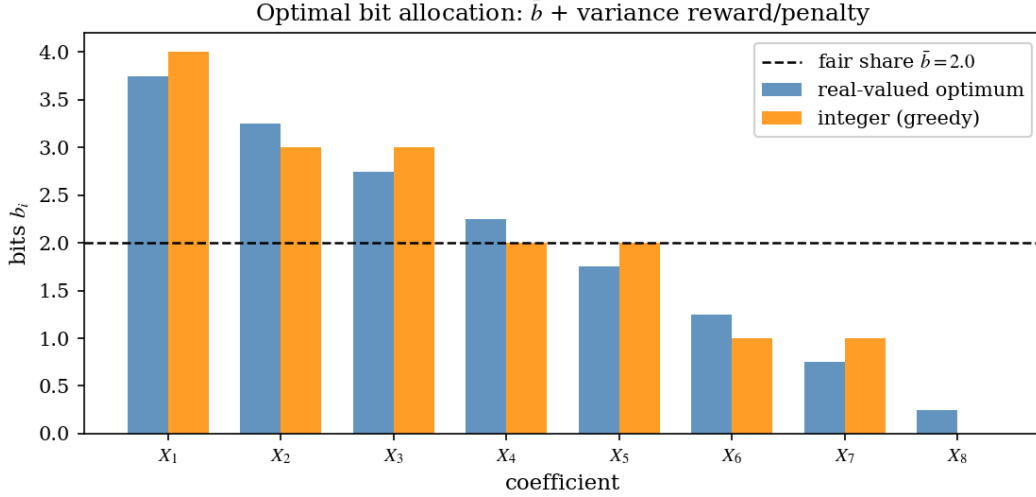


Figure 5: Optimal allocation for 8 Gaussian coefficients with geometrically decaying variances ($B = 16$): real-valued optimum (21) vs. greedy integer allocation. High-variance coefficients earn bits above the fair share $\bar{b} = 2$; low-variance ones are taxed. (Shape term is zero here: all h_i equal.)

Symbol glossary.

b_i	Bits assigned to coefficient i (real-valued in the formula; round/greedy-allocate in practice).
$B, \bar{b} = B/k$	Total bit budget; fair-share average.
σ_i^2	Variance of coefficient i ; ρ^2 its geometric mean.
h_i	Shape factor: distortion per unit variance per bit, shape-only (identical for all Gaussians).
H	Geometric mean of the shape factors.
$W_i(b)$	Best achievable b -bit distortion for coefficient i .

Intuition. Read (21) as *fair share plus two adjustments*: the variance term rewards high-energy coefficients (each doubling of σ_i^2 earns half a bit), the shape term rewards hard-to-quantize densities. The geometric means ρ^2, H are the “typical” references—a coefficient exactly at the geometric mean gets exactly $\bar{b}$ bits. In code, the formula can go negative for tiny σ_i^2 ; clamp at zero and re-solve, or use greedy marginal allocation (Figure 5, orange), which coincides with the formula when bits are plentiful.

6 KLT optimality and the transform coding gain

What the formulas say. For an orthogonal transform $\mathbf{Y} = \mathbf{A}\mathbf{X}$ with coefficient variances $d_i = [\mathbf{A}\mathbf{R}_x\mathbf{A}^t]_{ii}$, optimal bit allocation gives

$$D_{\text{tc}} \approx C \cdot \left(\prod_{i=1}^k d_i \right)^{1/k}, \quad (28)$$

(C independent of A), minimized by the KLT. The gain over direct (PCM) quantization is

$$\frac{D_{\text{PCM}}}{D_{\text{tc}}} = \frac{\frac{1}{k} \sum_i \lambda_i}{(\prod_i \lambda_i)^{1/k}} = \frac{\text{AM}(\lambda_i)}{\text{GM}(\lambda_i)} \geq 1, \quad (29)$$

where λ_i are the eigenvalues of R_x .

Where they come from. *Step 1: Hadamard's inequality.* For any positive-definite matrix M , $\det M \leq \prod_i M_{ii}$, with equality iff M is diagonal. Apply it to $R_y = AR_x A^t$:

$$\prod_{i=1}^k d_i \geq \det(R_y) = \det(R_x), \quad (30)$$

since $\det(AR_x A^t) = \det(R_x)$ for orthogonal A . The right side does not depend on A ; the left side is minimized (hence D_{tc} minimized) when equality holds, i.e. when R_y is diagonal.

Step 2: diagonalizing R_y . R_y diagonal means the transform coefficients are uncorrelated. The orthogonal A achieving this is the eigenvector matrix of R_x —the **KLT**. Its diagonal entries are then the eigenvalues λ_i .

Step 3: the gain. PCM quantizes the original (correlated) variables directly: with equal variances $\sigma^2 = \frac{1}{k} \sum \lambda_i = \frac{1}{k} \text{trace } R_x$, $D_{\text{PCM}} \approx C\sigma^2$. Transform coding with the KLT gives $D_{\text{tc}} \approx C(\prod_i \lambda_i)^{1/k}$. The ratio is (29): arithmetic mean over geometric mean of the eigenvalues, ≥ 1 by AM–GM.

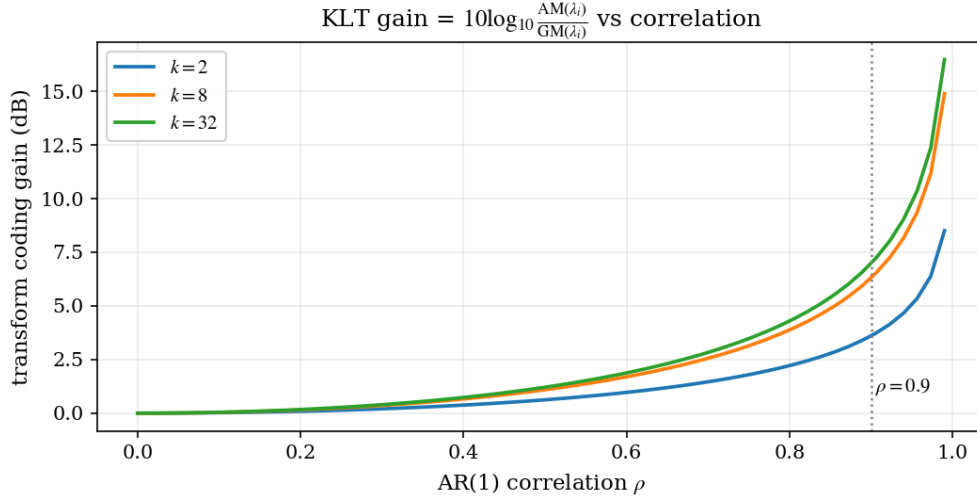


Figure 6: Transform coding gain vs. AR(1) correlation ρ : no correlation ($\rho = 0$) means equal eigenvalues and zero gain; strong correlation spreads the spectrum and the gain grows (6.3 dB at $\rho = 0.9$, $k = 8$). Larger blocks exploit correlation further.

Symbol glossary.

A	Orthogonal transform matrix; $\mathbf{Y} = A\mathbf{X}$.
R_x, R_y	Covariance matrices of input and coefficients.
d_i	Coefficient variances (diagonal of R_y).
λ_i	Eigenvalues of R_x (KLT coefficient variances).
AM, GM	Arithmetic and geometric means.

Intuition. Correlation is redundancy: if $X_1 \approx X_2$, PCM spends bits describing nearly the same thing twice. The KLT rotates into the basis where the redundancy becomes *energy concentration*—a few large- λ_i coefficients carry the signal, many tiny ones carry almost nothing—and the bit allocator (Section 5) then starves the tiny ones. The gain formula measures exactly this: AM/GM = 1 means flat spectrum (nothing to exploit); large AM/GM means spread spectrum (much to exploit). The DCT is used in practice because it approximates this rotation without depending on the signal.

7 COVQ: the channel-aware encoder and the posterior-mean decoder

What the formulas say. With channel transitions $P(j | i)$ from sent index i to received index j , the end-to-end distortion is

$$D = \sum_{i=1}^M \int_{\mathcal{S}_i} p_{\mathbf{X}}(\mathbf{x}) \sum_{j=1}^M P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 d\mathbf{x}, \quad (31)$$

minimized by alternating between

$$\mathcal{S}_i = \left\{ \mathbf{x} : \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 \leq \sum_j P(j | i') \|\mathbf{x} - \mathbf{y}_j\|^2 \quad \forall i' \right\} \quad (32)$$

and

$$\mathbf{y}_j = \mathbb{E}[\mathbf{X} | J = j] = \frac{\sum_i \mathbb{P}(I = i) P(j | i) \mathbf{c}_i}{\sum_i \mathbb{P}(I = i) P(j | i)}, \quad \mathbf{c}_i = \mathbb{E}[\mathbf{X} | I = i]. \quad (33)$$

Encoding can be linearized to

$$i^*(\mathbf{x}) = \arg \max_i \{ \mathbf{x}^t \tilde{\mathbf{y}}_i - \frac{1}{2} b_i \}, \quad \tilde{\mathbf{y}}_i = \sum_j P(j | i) \mathbf{y}_j, \quad b_i = \sum_j P(j | i) \|\mathbf{y}_j\|^2. \quad (34)$$

Where they come from. *Encoder (32): pointwise minimization.* Fix the decoder $\{\mathbf{y}_j\}$. The integrand of (31) at point $\mathbf{x}$ assigned to index i is $p_{\mathbf{X}}(\mathbf{x}) \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2$. Since $p_{\mathbf{X}}(\mathbf{x}) \geq 0$ is common to all choices, minimizing D means assigning each $\mathbf{x}$ to the i minimizing the bracket—exactly (32). (Compare the Lloyd–Max nearest-neighbor rule: the distance is now *averaged over the channel’s confusion*.)

Decoder (33): differentiate. Fix the partition. Rewrite (31) grouped by received index j :

$$D = \sum_{j=1}^M \sum_{i=1}^M P(j | i) \int_{\mathcal{S}_i} p_{\mathbf{X}}(\mathbf{x}) \|\mathbf{x} - \mathbf{y}_j\|^2 d\mathbf{x}. \quad (35)$$

Differentiating with respect to $\mathbf{y}_j$,

$$\frac{\partial D}{\partial \mathbf{y}_j} = -2 \sum_{i=1}^M P(j | i) \int_{\mathcal{S}_i} p_{\mathbf{X}}(\mathbf{x}) (\mathbf{x} - \mathbf{y}_j) d\mathbf{x} = \mathbf{0}. \quad (36)$$

Now $\int_{\mathcal{S}_i} p_{\mathbf{X}}(\mathbf{x}) d\mathbf{x} = \mathbb{P}(I = i)$ and $\int_{\mathcal{S}_i} \mathbf{x} p_{\mathbf{X}}(\mathbf{x}) d\mathbf{x} = \mathbb{P}(I = i) \mathbf{c}_i$. Substituting,

$$\sum_i P(j | i) \mathbb{P}(I = i) \mathbf{c}_i = \mathbf{y}_j \sum_i P(j | i) \mathbb{P}(I = i), \quad (37)$$

which is (33). By Bayes’ rule, $\mathbb{P}(I = i | J = j) = \mathbb{P}(I = i) P(j | i) / \mathbb{P}(J = j)$, so the right side is $\sum_i \mathbb{P}(I = i | J = j) \mathbf{c}_i = \mathbb{E}[\mathbf{X} | J = j]$: the **posterior mean**.

Linearization (34): expand the square. For fixed i ,

$$\begin{aligned} \sum_j P(j | i) \|\mathbf{x} - \mathbf{y}_j\|^2 &= \sum_j P(j | i) (\|\mathbf{x}\|^2 - 2\mathbf{x}^t \mathbf{y}_j + \|\mathbf{y}_j\|^2) \\ &= \underbrace{\|\mathbf{x}\|^2}_{\text{common to all } i} - 2\mathbf{x}^t \underbrace{\sum_j P(j | i) \mathbf{y}_j}_{\tilde{\mathbf{y}}_i} + \underbrace{\sum_j P(j | i) \|\mathbf{y}_j\|^2}_{b_i}. \end{aligned} \quad (38)$$

Dropping $\|\mathbf{x}\|^2$ and flipping min to max gives (34): encoding is M inner products against the **effective codevectors** $\tilde{\mathbf{y}}_i$ —the decoder codebook *as seen through the channel*—minus the offsets $b_i/2$.

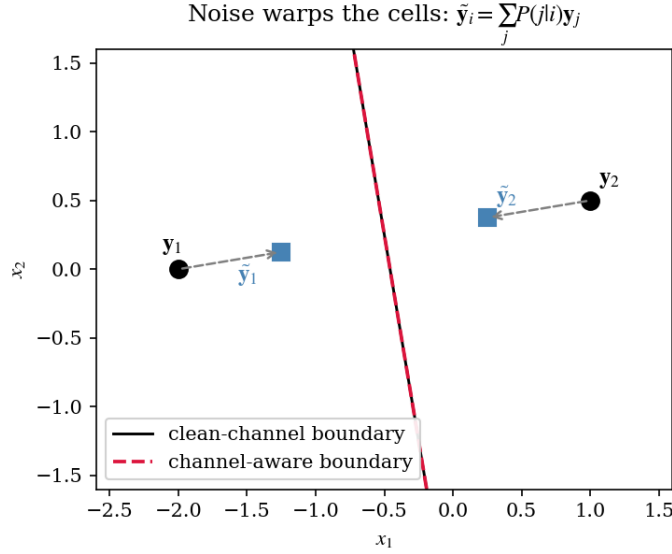


Figure 7: Two codevectors $\mathbf{y}_1, \mathbf{y}_2$ and their channel-averaged versions $\tilde{\mathbf{y}}_i$ (BSC, $\varepsilon = 0.25$) in 2-D. The effective codevectors are pulled toward each other, and the decision boundary shifts/rotates relative to the clean-channel perpendicular bisector: the encoder distrusts index 1 more than index 2.

Symbol glossary.

$\mathcal{S}_i$	Encoder cell: source vectors assigned index i .
$P(j i)$	Channel: probability of receiving j given i was sent (the $M \times M$ matrix the whole design hinges on).
$\mathbf{y}_j$	Decoder codebook: reconstruction upon receiving j .
$\mathbf{c}_i$	Centroid of cell $\mathcal{S}_i$ ($\mathbb{E}[\mathbf{X} I = i]$).
$\mathbb{P}(I = i)$	Probability the encoder emits i .
$\tilde{\mathbf{y}}_i$	Effective codevector: expected reconstruction when i is sent.
b_i	Offset: expected squared norm of that reconstruction.

Intuition. Two slogans. *Encoder*: “don’t ask which reconstruction is closest; ask which *distribution* of reconstructions is closest on average.” *Decoder*: “you didn’t receive index j cleanly—you received *evidence* j ; reconstruct the average source consistent with that evidence.” The linearization is the implementation payoff: channel awareness costs nothing at encode time beyond precomputing $(\tilde{\mathbf{y}}_i, b_i)$ once per training iteration.

8 Worked example: 1-bit Gaussian COVQ over a BSC

What the formulas say. For $X \sim \mathcal{N}(0, 1)$, one bit, $\text{BSC}(\varepsilon)$ with natural labeling, the optimal codevectors are $\pm y$ with

$$y = (1 - 2\varepsilon)\sqrt{\frac{2}{\pi}}, \quad D = 1 - (1 - 2\varepsilon)^2 \cdot \frac{2}{\pi}. \quad (39)$$

Where they come from. *Step 1: symmetry fixes the encoder.* By symmetry of source and channel, the optimal threshold is $t = 0$: $\mathcal{S}_1 = (-\infty, 0]$, $\mathcal{S}_2 = (0, \infty)$, each with $\mathbb{P}(I = i) = \frac{1}{2}$, and $\mathbf{c}_1 = -\sqrt{2/\pi}$, $\mathbf{c}_2 = +\sqrt{2/\pi}$ (mean of a half-normal).

Step 2: posterior-mean decoder. Apply (33) for $j = 1$ (by symmetry $y_2 = -y_1 =: y$):

$$\begin{aligned} y = \mathbb{E}[X \mid J = 1] &= \frac{\mathbb{P}(I = 1)P(J = 1 \mid I = 1)\mathbf{c}_1 + \mathbb{P}(I = 2)P(J = 1 \mid I = 2)\mathbf{c}_2}{\mathbb{P}(I = 1)P(J = 1 \mid I = 1) + \mathbb{P}(I = 2)P(J = 1 \mid I = 2)} \\ &= \frac{\frac{1}{2}(1 - \varepsilon)(-\sqrt{2/\pi}) + \frac{1}{2}\varepsilon(+\sqrt{2/\pi})}{\frac{1}{2}(1 - \varepsilon) + \frac{1}{2}\varepsilon} \\ &= [-(1 - \varepsilon) + \varepsilon]\sqrt{\frac{2}{\pi}} = -(1 - 2\varepsilon)\sqrt{\frac{2}{\pi}}. \end{aligned} \quad (40)$$

(The sign depends on labeling; $|y| = (1 - 2\varepsilon)\sqrt{2/\pi}$.) The clean-channel answer would be $\sqrt{2/\pi} \approx 0.798$; noise shrinks it by the factor $(1 - 2\varepsilon)$.

Step 3: distortion via the MMSE decomposition. For any estimator, $\mathbb{E}[(X - \mathbb{E}[X \mid J])^2] = \mathbb{E}[X^2] - \mathbb{E}[(\mathbb{E}[X \mid J])^2]$ (orthogonality principle). Here $\mathbb{E}[X \mid J] = \pm y$ with probability $\frac{1}{2}$ each, so $\mathbb{E}[(\mathbb{E}[X \mid J])^2] = y^2$, and with $\mathbb{E}[X^2] = 1$,

$$D = 1 - y^2 = 1 - (1 - 2\varepsilon)^2 \cdot \frac{2}{\pi}. \quad (41)$$

Intuition. At $\varepsilon = 0.1$, $y \approx 0.638$ instead of 0.798: the decoder hedges 20% toward zero because one received bit in ten lies. The distortion price is *quadratic* in $(1 - 2\varepsilon)$ —small noise costs little, but the cost accelerates. Every COVQ you will ever train is a high-dimensional version of this one line: *posterior means shrink toward the prior when the channel is unreliable*.

9 Pólya ML decoding: the d_0 threshold

What the formulas say. For block length n , initial red fraction ρ and contagion δ , the ML metric depends only on Hamming distance d through a log-convex $g(d)$ with unique minimum at

$$d_0 = \frac{n}{2} + \frac{1 - 2\rho}{2\delta}, \quad (42)$$

so ML decoding selects the codeword whose distance is **farthest from** d_0 —nearest or farthest, depending on which side the candidates fall.

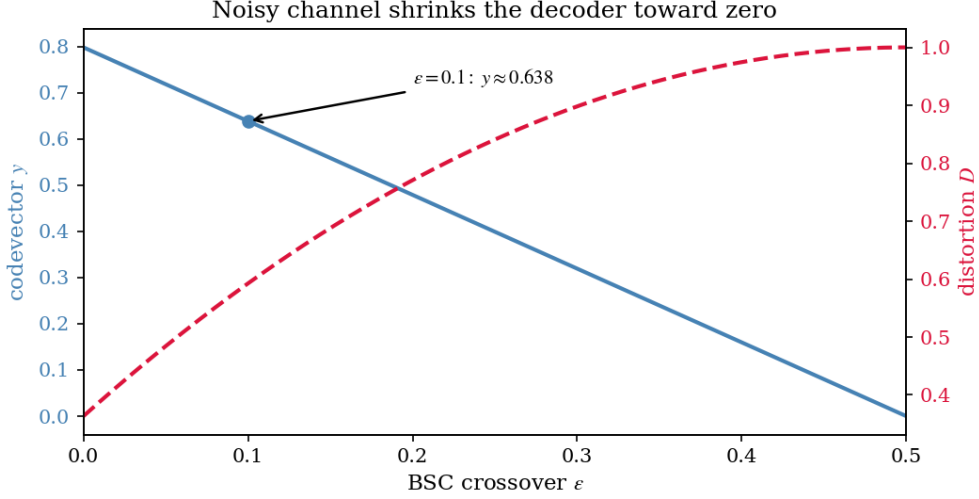


Figure 8: The shrinkage law (39): as $\varepsilon : 0 \rightarrow \frac{1}{2}$, the optimal codevector shrinks linearly to zero (the decoder trusts nothing) while distortion rises from $1 - 2/\pi \approx 0.36$ to 1 (the decoder outputs 0 always).

Where they come from. *Step 1: the gamma form.* The n -block transition probability (Main Guide, Sec. 6.4) is

$$g(d) = \frac{\prod_{a=0}^{d-1} (\rho + a\delta) \prod_{b=0}^{n-d-1} (\sigma + b\delta)}{\prod_{c=0}^{n-1} (1 + c\delta)}, \quad \sigma = 1 - \rho. \quad (43)$$

Factor δ out of each product: $\prod_{a=0}^{d-1} (\rho + a\delta) = \delta^d \Gamma(\rho/\delta + d)/\Gamma(\rho/\delta)$, giving

$$\log g(d) = \log \Gamma\left(\frac{\rho}{\delta} + d\right) + \log \Gamma\left(\frac{\sigma}{\delta} + n - d\right) + \text{const.} \quad (44)$$

Step 2: locate the minimum. Differentiate (digamma $\psi = \Gamma'/\Gamma$):

$$\frac{d}{dd} \log g(d) = \psi\left(\frac{\rho}{\delta} + d\right) - \psi\left(\frac{\sigma}{\delta} + n - d\right) = 0. \quad (45)$$

Since ψ is strictly increasing, this holds iff the arguments are equal:

$$\frac{\rho}{\delta} + d = \frac{\sigma}{\delta} + n - d \implies 2d = n + \frac{\sigma - \rho}{\delta} \implies d_0 = \frac{n}{2} + \frac{1 - 2\rho}{2\delta}. \quad (46)$$

Step 3: log-convexity. Differentiate again (trigamma $\psi' > 0$):

$$\frac{d^2}{dd^2} \log g(d) = \psi'\left(\frac{\rho}{\delta} + d\right) + \psi'\left(\frac{\sigma}{\delta} + n - d\right) > 0, \quad (47)$$

so $\log g$ is strictly convex: g is U-shaped with its unique minimum at d_0 .

Step 4: nearest-or-farthest. ML decoding maximizes $g(d)$ over the codeword distances $\{d(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in \mathcal{C}\}$. A convex U-shaped function attains its maximum over a finite set at an *extreme*—the point farthest from the minimizer d_0 . If all candidate distances lie below d_0 , that is the *smallest* distance (ordinary nearest-codeword decoding); if above, the *largest*. Sanity check: as $\delta \rightarrow 0$, $d_0 \rightarrow +\infty$, so every distance is below d_0 and we recover minimum-distance decoding, as we must for the BSC.

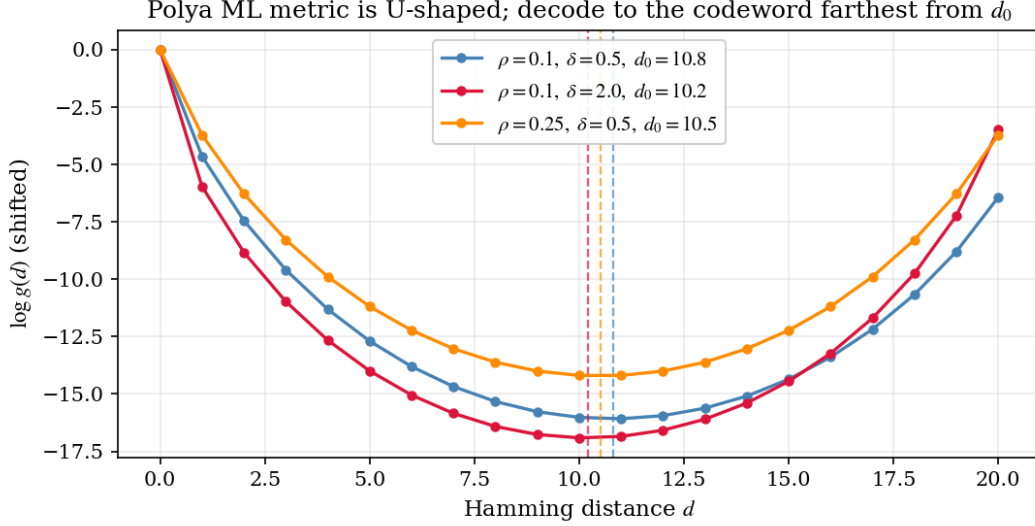


Figure 9: $\log g(d)$ for $n = 20$ and three (ρ, δ) sets: U-shaped with minimum at the predicted d_0 (dashed). ML decoding maximizes $g(d)$, i.e. picks the codeword distance farthest from d_0 . Larger contagion δ deepens the U (stronger memory).

Symbol glossary.

n	Block length.
ρ	Initial fraction of red balls (baseline error rate).
$\sigma = 1 - \rho$	Initial fraction of black balls.
δ	Contagion: extra balls per draw, scaled by urn size.
d	Hamming distance $d(\mathbf{x}, \mathbf{y})$.
d_0	Distance of minimum likelihood (the “least likely” number of errors).
ψ, ψ'	Digamma and trigamma functions.

Intuition. Under contagion, a *huge* Hamming distance is evidence of an “apparent epidemic”: the lot was bad, so the transmitted word was probably the one that looks *least* like what you received. The decoder therefore distrusts the middle ($d \approx d_0$ is the least likely outcome) and believes the extremes. This is the memory channel’s revenge on minimum-distance decoding: when errors come in clumps, closeness stops meaning correctness.

10 Capacity: $C = 1 - \bar{H}(Z)$ and the NEC formula

What the formulas say. For stationary binary additive noise $Y_i = X_i \oplus Z_i$,

$$C = 1 - \bar{H}(Z), \quad (48)$$

achieved by uniform i.i.d. inputs. For the noise-erasure channel (erasure probability ε , q -ary alphabet),

$$C = (1 - \varepsilon) \log q - [\bar{H}(Z) - \bar{H}(\tilde{Z})], \quad (49)$$

again with uniform inputs optimal. Feedback does not change (49) (unconstrained), but can improve the capacity–cost function under input–cost constraints.

Where (48) comes from. *Step 1: mutual information.* For block length n ,

$$\frac{1}{n}I(X^n; Y^n) = \frac{1}{n}H(Y^n) - \frac{1}{n}H(Y^n | X^n). \quad (50)$$

Given X^n , $Y^n = X^n \oplus Z^n$ is a deterministic function of Z^n , so $H(Y^n | X^n) = H(Z^n)$, and

$$\frac{1}{n}I(X^n; Y^n) = \frac{1}{n}H(Y^n) - \frac{1}{n}H(Z^n). \quad (51)$$

Step 2: maximize over inputs. Y_i is binary, so $\frac{1}{n}H(Y^n) \leq 1$, with equality iff Y^n is uniform i.i.d. Take X^n i.i.d. uniform and independent of Z^n : then $Y^n = X^n \oplus Z^n$ is i.i.d. uniform whatever Z^n is (the one-time-pad property), so $\frac{1}{n}H(Y^n) = 1$ exactly.

Step 3: the limit. Taking sup over input processes and $n \rightarrow \infty$ in (51),

$$C = 1 - \lim_{n \rightarrow \infty} \frac{1}{n}H(Z^n) = 1 - \bar{H}(Z). \quad (52)$$

The **entropy rate** $\bar{H}(Z)$ is the residual uncertainty per noise symbol *after* exploiting all of the noise’s memory—what no predictor, however clever, can remove. Uniform inputs are optimal by symmetry: the channel treats every input pattern identically ($Y | X$ depends only on $X \oplus Y$).

Reading (49) term by term.

$(1 - \varepsilon) \log q$ Raw throughput: each symbol survives with probability $1 - \varepsilon$, and a surviving q -ary symbol carries at most $\log q$ bits.

$\bar{H}(Z)$ Total noise uncertainty per symbol (as above).

$\bar{H}(\tilde{Z})$ Noise uncertainty *at the erased positions*. Formally $\tilde{Z}$ is the noise process masked to the erasure locations; the decoder knows *where* erasures happened, so noise at those positions costs it nothing.

difference $\bar{H}(Z) - \bar{H}(\tilde{Z})$ The noise uncertainty remaining at *unerased* positions—the true penalty. Checks: $\varepsilon = 0$ gives $\bar{H}(\tilde{Z}) = 0$ and $C = \log q - \bar{H}(Z)$ (noisy channel, no erasures); $Z \equiv 0$ gives $C = (1 - \varepsilon) \log q$ (pure erasure channel).

Memoryless binary check: $Z \sim \text{Bern}(p)$ i.i.d., erasures independent $\Rightarrow \bar{H}(Z) - \bar{H}(\tilde{Z}) = (1 - \varepsilon)h_b(p)$, so $C = (1 - \varepsilon)(1 - h_b(p))$ —the erasure channel’s throughput times the BSC’s per-symbol capacity.

What feedback does and doesn’t change. The converse for (49) bounds $H(Y^n)$ by the same $(1 - \varepsilon) \log q$ per symbol whether or not the encoder sees past outputs—feedback cannot create surviving symbols, so $C_{\text{FB}} = C$ unconstrained. Under an input-cost constraint, though, feedback tells the encoder *which* past symbols were erased, letting it concentrate its cost budget on symbols that survive: the capacity–cost function improves even though the raw capacity doesn’t.

Intuition. Both formulas say the same thing: *capacity equals the alphabet’s raw bit-carrying ability minus the noise’s irreducible uncertainty*. The entropy rate is the key conceptual leap from the memoryless case: with memory, the “noise entropy” is not $H(Z_1)$ but the limit after prediction—bursty noise has *less* entropy rate than its marginal suggests, which is exactly why exploiting memory (instead of interleaving it away) can win.

Every formula here is a machine with an input, a derivation, and a bill of materials. Read the derivation once, memorize the intuition forever.

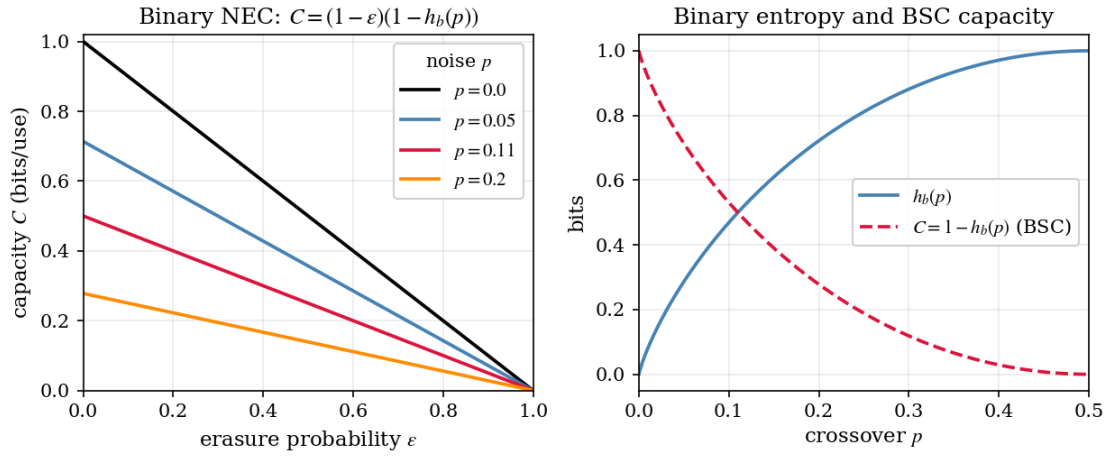


Figure 10: Left: binary NEC capacity $C = (1 - \epsilon)(1 - h_b(p))$ vs. erasure rate for several noise levels—erasures and noise multiply their damage. Right: the binary entropy $h_b(p)$ and the BSC capacity $1 - h_b(p)$: capacity is what remains after subtracting the noise’s irreducible uncertainty.

Part IV — Error-Correcting Codes: The Mathematics of Protection

`error-correcting-codes.pdf` (17 pages)

The other half of the channel story. Part II models channels and designs the joint system; this part shows how channel coding actually protects bits: finite fields built from scratch, linear block codes and syndrome decoding, the $d_{\min} \geq 2t + e + 1$ bound proved and demonstrated (including what happens when you violate it), the fundamental bounds, CRCs, Reed–Solomon codes decoded end to end (syndromes, Berlekamp–Massey, Chien search, Forney, erasures), convolutional codes and Viterbi, LDPC belief propagation, turbo and polar codes, fountain codes for erasures, and the Shannon limits. Two corrections from this volume’s errata apply while reading: the asymptotic Plotkin threshold is $\delta > 1/2$ (not $1/4$), and blank symbols should be read as $\mathbb{Z}$. Every number is verified by the companion script `ecc_demo.py`.

Error-Correcting Codes

The Mathematics of Fixing Errors and Erasures

A worked companion to the joint source–channel coding guides

September 2026

Abstract

This is the fifth guide in the set, and the most algebraic. It develops error-correcting codes from the ground up: the background mathematics each idea leans on (modular arithmetic, linear algebra over $\mathbb{F}_2$, counting in Hamming space, polynomial roots), the codes themselves (Hamming, cyclic/CRC, Reed–Solomon, convolutional, LDPC, turbo, polar, fountain), and the decoding algorithms that make them work. The running theme is the asymmetry between *errors* (a value arrives wrong) and *erasures* (a value never arrives): an erasure costs half as much redundancy as an error, because the decoder already knows *where* to look. Every worked example uses numbers produced by the companion script `ecc_demo.py` (pure Python, in the same folder) — run it and the printed values match this text exactly.

Contents

1	Introduction: two kinds of damage	3
2	Background: modular arithmetic, and why codes need fields	3
2.1	Arithmetic mod n	3
2.2	Why decoding needs division	3
3	Finite fields $\mathbb{F}_{2^m}$	4
3.1	Construction via a primitive polynomial	4
3.2	Worked arithmetic in $\mathbb{F}_{16}$	4
4	Background: linear algebra over $\mathbb{F}_2$	5
5	Linear block codes: what encoding really is	5
6	The Hamming(7,4) code, twice	5
6.1	Worked example 1	6
6.2	Worked example 2	6
7	The $d_{\min} \geq 2t + e + 1$ bound	6
7.1	Watching the bound fail	7
8	Background: counting in Hamming space	7
9	The fundamental bounds	8
10	Cyclic codes and CRCs	8
10.1	The polynomial view	8
10.2	Worked CRC example	9

11 Background: why polynomial evaluation gives distance	10
12 Reed–Solomon codes	10
12.1 Construction and parameters	10
12.2 Decoding: the pipeline	10
12.3 Worked example: 1 error + 2 erasures	11
12.4 Worked example 2: erasures only	12
13 Convolutional codes and the Viterbi algorithm	12
13.1 The trellis	12
13.2 Why Viterbi’s pruning is safe	12
13.3 Worked Viterbi decode	12
14 Modern codes: LDPC, turbo, polar	12
14.1 LDPC: sparse parity checks, belief propagation	13
14.2 Turbo codes: parallel concatenation	13
14.3 Polar codes: channel polarization	13
15 Erasures only: fountain codes	14
15.1 LT codes	14
15.2 Peeling walkthrough	15
15.3 When peeling stalls (second example)	15
16 Shannon limits and the road not taken	15
16.1 Separate coding vs. joint coding (COVQ)	15
A Cheat sheet	17

1 Introduction: two kinds of damage

A communication channel damages data in two fundamentally different ways. An **error** flips a symbol to a wrong value: you receive a 1 where a 0 was sent, and nothing tells you it happened. An **erasure** deletes a symbol: you receive a blank, and you know exactly which position is missing. Bit flips on a wire are errors; dropped packets on the internet are erasures.

This distinction drives the entire document, because it drives the central quantitative result of coding theory:

$$d_{\min} \geq 2t + e + 1 \quad (1)$$

is the condition for a code with minimum distance $d_{\min}$ to correct t errors *and* e erasures simultaneously. Read it as a price list: each error costs **two** units of distance, each erasure costs **one**. Erasures are half-price because the decoder knows *where* they are and only has to figure out *what* was there; errors force the decoder to solve both problems at once. We will prove (1) in Chapter 6, demonstrate it failing when violated, and spend Chapters 11–13 exploiting it.

How to read this guide. Each chapter opens with the background mathematics its machinery leans on — modular arithmetic before finite fields, linear algebra before syndromes, volume counting before the bounds, the polynomial root bound before Reed–Solomon decoding. Heavy derivations end with an *Intuition check* paragraph restating what just happened in plain language. Worked examples come in pairs wherever a single small case might look like a coincidence. Figures live in `figs/`; the script `ecc_demo.py` reproduces every number.

2 Background: modular arithmetic, and why codes need fields

2.1 Arithmetic mod n

Fix an integer $n \geq 2$ and declare two integers equivalent if they differ by a multiple of n :

$$a \equiv b \pmod{n} \iff n \mid (a - b). \quad (2)$$

The residues $\{0, 1, \dots, n-1\}$ with addition and multiplication mod n form the ring $\mathbb{Z}_n$. Addition and multiplication always work, but **division** — multiplicative inverses — may not exist. An element a has an inverse mod n iff $\gcd(a, n) = 1$. Mod 6, for instance, $5^{-1} \equiv 5$ (since $25 \equiv 1$), but 2 has no inverse: $2x \equiv 1 \pmod{6}$ would require $2x - 1$ divisible by 6, impossible since the left side is odd. Worse, $\mathbb{Z}_6$ has **zero divisors**: $2 \cdot 3 \equiv 0 \pmod{6}$ with neither factor zero.

2.2 Why decoding needs division

Decoding is, at its core, solving equations. Syndrome decoding solves $H\mathbf{e}^T = \mathbf{s}$ for the error pattern $\mathbf{e}$; Reed–Solomon erasure decoding solves a Vandermonde linear system for the missing values. Solving linear systems requires Gaussian elimination, and elimination requires dividing by pivot elements. In a ring with zero divisors, a “nonzero” pivot may still have no inverse, and elimination collapses. This is why coding theory works over **fields** — rings where every nonzero element has an inverse:

Definition 2.1 (Field). A field is a set with $+$, $\times$ where $(\mathbb{F}, +)$ is an abelian group, $(\mathbb{F} \setminus \{0\}, \times)$ is an abelian group, and multiplication distributes over addition.

$\mathbb{F}_p$ is a field iff p is prime. $\mathbb{F}_2 = \{0, 1\}$ with XOR as addition and AND as multiplication is the field everything binary is built on:

$+$	0	1	$\times$	0	1
0	0	1	0	0	0
1	1	0	1	0	1

Note that in characteristic 2, addition equals subtraction ($-a = a$), a fact we will use constantly: signs simply vanish.

Intuition check. *Ordinary integer arithmetic is too weak for code algebra because you cannot always divide. Decoding means solving equations, solving means eliminating, eliminating means dividing by pivots. Fields are exactly the number systems where division always works, so codes are built over fields — $\mathbb{F}_2$ for binary codes, larger fields when we need bigger alphabets.*

3 Finite fields $\mathbb{F}_{2^m}$

Binary codes live over $\mathbb{F}_2$, but Reed–Solomon codes need alphabets larger than 2 (a byte-sized symbol, say) while keeping field structure. There is no field with 6 elements, but there *is* one with 2^m elements for every m : the field $\mathbb{F}_{2^m}$.

3.1 Construction via a primitive polynomial

Represent elements as polynomials over $\mathbb{F}_2$ of degree $< m$:

$$a(x) = a_0 + a_1x + \cdots + a_{m-1}x^{m-1}, \quad a_i \in \mathbb{F}_2, \quad (3)$$

i.e. m -bit strings. Addition is XOR (coefficient-wise). For multiplication, multiply polynomials normally, then take the remainder upon division by a fixed **primitive polynomial** $p(x)$ of degree m — an irreducible polynomial whose root α generates all nonzero elements as powers. For $m = 4$ the standard choice is

$$p(x) = x^4 + x + 1 \quad (\text{binary } 10011 = 0\text{x}13), \quad (4)$$

with primitive element $\alpha = x$ (binary 0010). Since $p(\alpha) = 0$, we have the reduction rule

$$\alpha^4 = \alpha + 1, \quad (5)$$

which lets us reduce any power: $\alpha^5 = \alpha^2 + \alpha$, $\alpha^6 = \alpha^3 + \alpha^2$, and so on, cycling through all 15 nonzero elements before $\alpha^{15} = 1$.

3.2 Worked arithmetic in $\mathbb{F}_{16}$

Take $a = 1011_2$ ($x^3 + x + 1$) and $b = 0110_2$ ($x^2 + x$), using the values produced by `ecc_demo.py`:

$$\begin{aligned} a + b &= 1011 \oplus 0110 = 1101_2 && (\text{addition is XOR}) \\ a \cdot b &= (x^3 + x + 1)(x^2 + x) = x^5 + x^4 + x^3 + x^3 + x^2 + x \\ &= x^5 + x^4 + x^2 + x && (x^3 + x^3 = 0 \text{ in char. } 2) \\ &= (\alpha^2 + \alpha) + (\alpha + 1) + \alpha^2 + \alpha = \alpha^2 + \alpha + 1 = 1111_2 && (\text{reduce via } \alpha^4 = \alpha + 1). \end{aligned}$$

So $0\text{x}b \times 0\text{x}6 = 0\text{x}f$ in $\mathbb{F}_{16}$. Inverses come from the power representation: since $\alpha^{15} = 1$, $\alpha^{-j} = \alpha^{15-j}$, e.g. $\alpha^{-7} = \alpha^8$. In code this is a log/antilog table lookup; multiplication becomes *addition of logarithms*: $\alpha^{14} \cdot \alpha^3 = \alpha^{17} = \alpha^2$.

Intuition check. $\mathbb{F}_{2^m}$ keeps everything we like about ordinary arithmetic — especially division — while elements stay m bits wide. Think of it as “ m -bit strings with a multiplication that wraps around via the primitive polynomial instead of overflowing.” The power representation $\{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$ turns multiplication into addition mod $(2^m - 1)$, which is why hardware and software both use log tables.

4 Background: linear algebra over $\mathbb{F}_2$

A binary code of length n is a subset of $\mathbb{F}_2^n$. A **linear** code is a *subspace*: closed under addition (XOR). Everything in this chapter is ordinary linear algebra, except scalars come from $\mathbb{F}_2$.

The workhorse is the **null space**. Given an $r \times n$ matrix H over $\mathbb{F}_2$, its null space

$$\ker H = \{\mathbf{c} \in \mathbb{F}_2^n : H\mathbf{c}^T = \mathbf{0}\} \quad (6)$$

is a subspace of dimension $n - \text{rank } H$. We will *define* our code as $\ker H$ for a cleverly chosen H , the **parity-check matrix**. Each row of H is one parity equation the codeword must satisfy.

Now the key mechanical insight, the whole reason syndromes work. Send $\mathbf{c} \in \ker H$; receive $\mathbf{r} = \mathbf{c} + \mathbf{e}$ where $\mathbf{e}$ is the error pattern. Compute

$$\mathbf{s} = \mathbf{r}H^T = (\mathbf{c} + \mathbf{e})H^T = \underbrace{\mathbf{c}H^T}_{=\mathbf{0}} + \mathbf{e}H^T = \mathbf{e}H^T. \quad (7)$$

The codeword **annihilates itself** through H — the null-space property — so the syndrome $\mathbf{s}$ depends *only on the error*, never on which codeword was sent. Decoding becomes: given $\mathbf{s}$, find the “smallest” $\mathbf{e}$ with $\mathbf{e}H^T = \mathbf{s}$. The codeword has been projected away; what remains is pure error information.

Intuition check. H is a machine that outputs zero on every valid codeword. Adding an error moves the received word off the code space, and the syndrome measures how it moved — which is exactly the error pattern’s fingerprint, with the original message completely cancelled out.

5 Linear block codes: what encoding really is

An (n, k) binary linear block code maps k data bits to n codeword bits ($n > k$; the rate is $R = k/n$). Encoding is a **linear map** $\mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ given by a $k \times n$ **generator matrix** G of rank k :

$$\mathbf{c} = \mathbf{u}G, \quad \mathbf{u} \in \mathbb{F}_2^k. \quad (8)$$

What this *means*: the k rows of G are k linearly independent codewords forming a basis of the code; encoding writes the data vector $\mathbf{u}$ in that basis. The i th data bit selects whether the i th basis codeword (row) is XORed into the output. In **systematic** form $G = [I_k \mid P]$, the first k positions of $\mathbf{c}$ are just $\mathbf{u}$ itself (visible data), and the last $n - k$ are parity bits (linear combinations of data). The parity-check matrix is then $H = [P^T \mid I_{n-k}]$, and indeed $GH^T = \mathbf{0}$.

The **minimum distance**

$$d_{\min} = \min_{\mathbf{c} \neq \mathbf{c}'} d_H(\mathbf{c}, \mathbf{c}') \quad (9)$$

(Hamming distance d_H = number of differing positions) controls everything. For linear codes this equals the minimum *weight* (number of ones) of any nonzero codeword, since $d_H(\mathbf{c}, \mathbf{c}') = \text{wt}(\mathbf{c} + \mathbf{c}')$.

6 The Hamming(7,4) code, twice

The Hamming(7,4) code ($n = 7$, $k = 4$, $R = 4/7$) is the smallest single-error-correcting code. Its parity-check matrix is ingeniously simple: the columns are the binary numbers 1 through 7:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}, \quad \text{column } j = \text{binary}(j). \quad (10)$$

Why this works: a single error at position j gives syndrome $\mathbf{s} = \mathbf{e}H^T$ = the j th column of H = $\text{binary}(j)$. The syndrome *is* the error address. Positions 1, 2, 4 (powers of two) hold parity bits; positions 3, 5, 6, 7 hold data. Figure 1 shows the Tanner graph: 7 variable nodes, 3 check nodes, edges where H has ones.

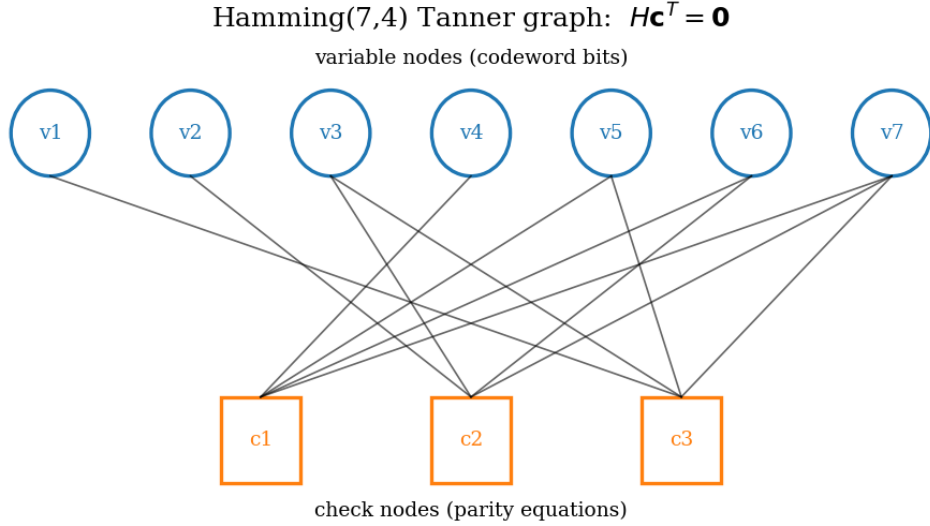


Figure 1: Tanner graph of the Hamming(7,4) code. Circles: codeword bits (positions 1–7); squares: the three parity checks (rows of H).

6.1 Worked example 1

Encode data $\mathbf{u} = [1, 0, 1, 1]$ (positions 3, 5, 6, 7). Each parity bit covers the positions whose index has that bit set:

$$p_1 = u_3 \oplus u_5 \oplus u_7 = 1 \oplus 0 \oplus 1 = 0,$$

$$p_2 = u_3 \oplus u_6 \oplus u_7 = 1 \oplus 1 \oplus 1 = 1,$$

$$p_4 = u_5 \oplus u_6 \oplus u_7 = 0 \oplus 1 \oplus 1 = 0.$$

Codeword (positions 1–7): $\mathbf{c} = [0, 1, 1, 0, 0, 1, 1]$. Flip position 5: $\mathbf{r} = [0, 1, 1, 0, 1, 1, 1]$. Syndrome:

$$\mathbf{s} = \mathbf{r}H^T = [1, 0, 1] = \text{binary}(5). \quad (11)$$

Flip position 5 back; read off data $[1, 0, 1, 1]$. (Script output: syndrome $[1, 0, 1] \rightarrow$ position 5, corrected, match: True.)

6.2 Worked example 2

Different data, different error, same machinery. Data $\mathbf{u} = [0, 1, 0, 1]$: $p_1 = 0 \oplus 1 \oplus 1 = 0$, $p_2 = 0 \oplus 0 \oplus 1 = 1$, $p_4 = 1 \oplus 0 \oplus 1 = 0$, so $\mathbf{c} = [0, 1, 0, 0, 1, 0, 1]$. Flip position 2: $\mathbf{r} = [0, 0, 0, 0, 1, 0, 1]$. Syndrome: $s_0 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0$, $s_1 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 1$, $s_2 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 0$, i.e. $[0, 1, 0] = \text{binary}(2)$. Flip position 2 back: done.

Intuition check. The syndrome doesn't “search” for the error — there is no search. Because H 's columns are position labels, the matrix multiplication $\mathbf{e}H^T$ simply reads out the column where the single 1 of $\mathbf{e}$ sits. Error correction here is table lookup disguised as linear algebra.

7 The $d_{\min} \geq 2t + e + 1$ bound

Theorem 7.1 (Errors-and-erasures correction). *A code with minimum distance $d_{\min}$ can correct any pattern of t errors and e erasures iff*

$$2t + e < d_{\min} \quad \text{i.e.} \quad d_{\min} \geq 2t + e + 1. \quad (12)$$

Proof. Erasures first. Suppose e positions are erased and the rest arrive clean. Two codewords $\mathbf{c} \neq \mathbf{c}'$ that agree on all unerased positions differ in at most e positions, so $d_H(\mathbf{c}, \mathbf{c}') \leq e$. If $e < d_{\min}$, this is impossible: at most one codeword matches what we received, so the decoder picks it. Erasures cost 1 unit of distance each.

Errors. With t errors and no erasures, the received word $\mathbf{r}$ lies within distance t of the true codeword. Every other codeword is at distance $\geq d_{\min}$ from the true one, hence at distance $\geq d_{\min} - t$ from $\mathbf{r}$ (triangle inequality). Nearest-neighbour decoding is correct iff the true codeword is strictly closest: $t < d_{\min} - t$, i.e. $2t < d_{\min}$. Errors cost 2 units each, because the decoder must both *locate* and *fix* them.

Combined. Erase e positions (puncture the code: distance drops by at most e), then correct t errors in the punctured code, which needs $2t < d_{\min} - e$. $\square$

Intuition check. An erasure says “something is missing here” — half the mystery is already solved, so it costs half the distance. An error says nothing; the decoder must discover both the location and the value, paying double. This is the single most useful inequality in practical coding: on a packet network where losses are flagged as erasures, you get twice the protection per parity symbol.

7.1 Watching the bound fail

Failure 1 (Hamming(7,4), $d_{\min} = 3$). Two errors need $2 \cdot 2 = 4 \geq 3$ — beyond the bound. Flipping positions 1 and 2 of $[0, 1, 1, 0, 0, 1, 1]$ gives syndrome $[0, 1, 1] = \text{binary}(3)$: the decoder “corrects” position 3, producing a *third* error. The script confirms: two flips $\rightarrow$ syndrome points at position 3 (miscorrects: $d_{\min} = 3$ fixes 1 only).

Failure 2 (RS(15,9), $d_{\min} = 7$). Four errors need $2 \cdot 4 = 8 > 7$. The script’s sweep: 40 random trials with $t = 4, e = 0$ — 0 of 40 decode correctly. Meanwhile every (t, e) with $2t + e \leq 6$ decodes 40/40. The bound is not conservative; it is tight.

8 Background: counting in Hamming space

Before the fundamental bounds, the right mental picture. $\mathbb{F}_2^n$ has 2^n points. The **Hamming ball** of radius t around $\mathbf{c}$,

$$B_t(\mathbf{c}) = \{\mathbf{x} : d_H(\mathbf{x}, \mathbf{c}) \leq t\}, \quad (13)$$

contains every word obtainable from $\mathbf{c}$ with $\leq t$ flips. Its volume (number of points) is

$$V(n, t) = \sum_{i=0}^t \binom{n}{i}, \quad (14)$$

independent of the center, by symmetry. A t -error-correcting code with M codewords needs M *disjoint* radius- t balls (else two codewords’ balls overlap and some received word is ambiguous). All balls live inside $\mathbb{F}_2^n$, so

$$M \cdot V(n, t) \leq 2^n. \quad (15)$$

That single inequality *is* the Hamming bound. Sphere-packing is literal: how many non-overlapping balls fit in the space?

Intuition check. Think of each codeword as owning a “territory” of all words within t flips. Decoding = finding whose territory you’re in. Territories can’t overlap (ambiguity) and can’t exceed the whole space (there are only 2^n words). Every upper bound in coding theory is some version of this volume argument.

9 The fundamental bounds

Let $A(n, d)$ be the maximum number of codewords in a binary code of length n and minimum distance d .

Theorem 9.1 (Hamming / sphere-packing bound).

$$A(n, d) \leq \frac{2^n}{\sum_{i=0}^t \binom{n}{i}}, \quad t = \left\lfloor \frac{d-1}{2} \right\rfloor. \quad (16)$$

Proof sketch. Radius- t balls around codewords are disjoint (if two balls shared a word $\mathbf{x}$, then $d_H(\mathbf{c}, \mathbf{c}') \leq d_H(\mathbf{c}, \mathbf{x}) + d_H(\mathbf{x}, \mathbf{c}') \leq 2t < d$, contradiction). Count points. $\square$

Theorem 9.2 (Singleton bound).

$$A(n, d) \leq 2^{n-d+1} \quad \text{i.e.} \quad k \leq n - d + 1. \quad (17)$$

Proof. Puncture (delete) any $d-1$ coordinates. Two codewords differing in $< d$ positions cannot exist, so all punctured codewords are distinct: $M \leq 2^{n-d+1}$. $\square$

Codes meeting it with equality are **MDS** (maximum distance separable) — Reed–Solomon codes are the prime example.

Theorem 9.3 (Plotkin bound). *If $d > n/2$ then $A(n, d) \leq 2 \left\lfloor \frac{d}{2d-n} \right\rfloor$. In particular, asymptotically, no positive-rate code has $\delta = d/n > 1/4$.*

Idea. Double-count pairs of codewords and their agreements: if all pairs are far apart ($> n/2$), the pigeonhole principle forces the code to be tiny. Summing d_H over all $\binom{M}{2}$ pairs and bounding each coordinate's contribution gives the result. $\square$

Theorem 9.4 (Gilbert–Varshamov bound). *There exist codes with*

$$\frac{k}{n} \geq 1 - h_2(\delta), \quad \delta = d/n, \quad (18)$$

where h_2 is the binary entropy function.

Idea – greedy packing. Build codewords one at a time: pick any word, delete its entire radius- $(d-1)$ ball (all words too close), repeat. Each step removes $< V(n, d-1)$ words, so we get $\geq 2^n / V(n, d-1)$ codewords; the asymptotic volume estimate $V \approx 2^{nh_2(\delta)}$ gives the bound. $\square$

Figure 2 plots the asymptotic forms: Singleton is the weakest upper bound, Hamming the strongest of the three shown, GV is a *lower* bound (existence), and Plotkin kills all positive rate beyond $\delta = 1/4$.

Intuition check. *Upper bounds (Hamming, Singleton, Plotkin) say how good codes can't be — usually volume arguments. GV says good codes exist — a greedy construction. The gap between Hamming and GV is where all of modern coding theory (LDPC, turbo, polar) operates.*

10 Cyclic codes and CRCs

10.1 The polynomial view

Write a codeword $\mathbf{c} = (c_0, \dots, c_{n-1})$ as a polynomial $c(x) = c_0 + c_1x + \dots + c_{n-1}x^{n-1}$ over $\mathbb{F}_2$. A code is **cyclic** if $(c_{n-1}, c_0, \dots, c_{n-2}) \in \mathcal{C}$ whenever $\mathbf{c} \in \mathcal{C}$ — rotation preserves codewords. In polynomial language, rotation is multiplication by $x \bmod (x^n + 1)$, and cyclic codes are

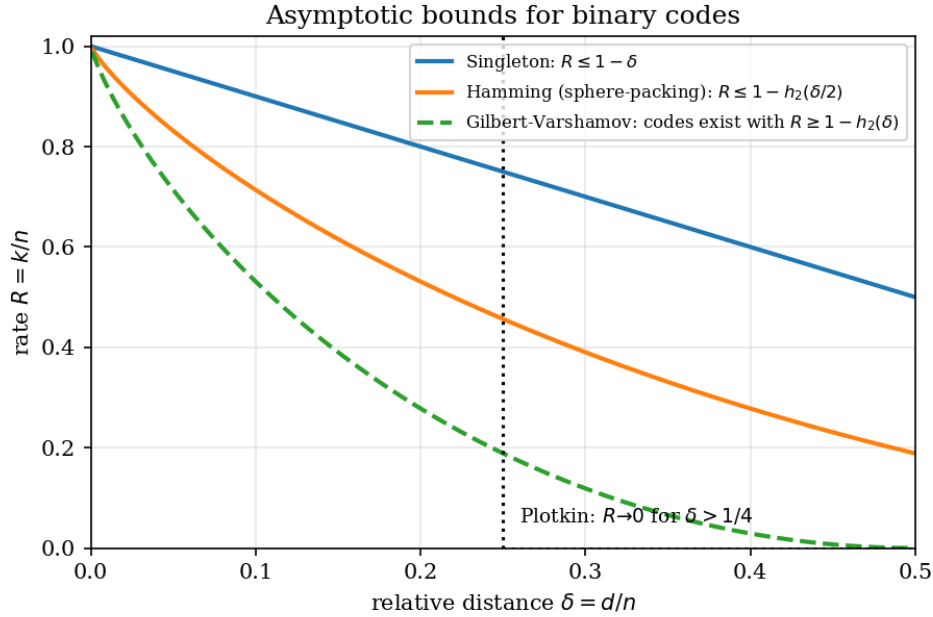


Figure 2: Asymptotic binary code bounds: rate R vs. relative distance δ . Upper bounds say “no code lives above”; GV says “some code lives above the dashed curve.”

exactly the **ideals** in the ring $\mathbb{F}_2[x]/(x^n + 1)$. Every ideal is principal, so every cyclic code has a **generator polynomial** $g(x)$ dividing $x^n + 1$:

$$\mathcal{C} = \{m(x)g(x) : \deg m < k\}, \quad n - k = \deg g. \quad (19)$$

Encoding = polynomial multiplication; the parity-check polynomial is $h(x) = (x^n + 1)/g(x)$. Cyclic structure gives shift-register encoders/decoders — the reason CRCs are everywhere in hardware.

10.2 Worked CRC example

A CRC *detects* (rather than corrects) using a degree- r generator. Data 11010011, generator $g(x) = x^3 + x + 1$ (1011): append $r = 3$ zeros and divide by $g(x)$ over $\mathbb{F}_2$:

```

11010011 000
1011      <- align, XOR
----
01100011 000
1011
----
0111011 000 ...

```

Carrying out the full long division (XOR at each 1), the remainder is **011**. Transmit the data bits followed by the remainder. The receiver divides the 11-bit word by 1011: remainder 000 means “no detected error.” Flip one bit (position 5): the remainder becomes 101 $\neq$ 000 — detected. (Verified by `ecc_demo.py`’s CRC check.)

A degree- r CRC detects all burst errors of length $\leq r$ (a burst of length $\leq r$ corresponds to $x^i b(x)$ with $\deg b < r$; since g has nonzero constant term, $g \nmid x^i b$), all odd-weight errors if g has an even number of terms, and any single-bit error. What it *cannot* do is correct: many error patterns share remainder 000.

Intuition check. A CRC is a divisibility test: valid words are multiples of $g(x)$. Noise adds an error polynomial $e(x)$; detection fails only if $e(x)$ happens to be a multiple of $g(x)$ too. Choosing g with the right factors makes the dangerous multiples (short bursts) impossible.

11 Background: why polynomial evaluation gives distance

Reed–Solomon codes evaluate polynomials. Their distance comes from one fact:

Theorem 11.1 (Root bound). *A nonzero polynomial of degree $< k$ over a field has fewer than k roots.*

Proof. Induction on k . For $k = 1$, a nonzero constant has no roots. Let $\deg p < k$ with $p(r) = 0$. By the factor theorem, $p(x) = (x - r)q(x)$ with $\deg q < k - 1$ (polynomial division works over a field — this is where §2 pays off). Every other root of p is a root of q (fields have no zero divisors: $(s - r)q(s) = 0$ forces $q(s) = 0$), and q has $< k - 1$ roots by induction. Total: $< k$. $\square$

Corollary: two distinct degree- $< k$ polynomials agree in $< k$ positions. So their n -point evaluations differ in $> n - k$ positions:

$$d_{\min} \geq n - k + 1. \quad (20)$$

Combined with Singleton ($d_{\min} \leq n - k + 1$), RS codes are MDS. The whole code is this one theorem plus the Singleton bound.

Intuition check. A low-degree polynomial is “rigid” — it can’t wiggle through many prescribed points. Two different messages give different polynomials, which must disagree almost everywhere. Evaluating at n points spreads each message across all n symbols, so damaging a few symbols can’t disguise one message as another.

12 Reed–Solomon codes

12.1 Construction and parameters

Fix $\mathbb{F}_{2^m}$, $n = 2^m - 1$, primitive α . Message $\mathbf{m} = (m_0, \dots, m_{k-1})$ defines $m(x) = m_0 + m_1x + \dots + m_{k-1}x^{k-1}$; the codeword is

$$\mathbf{c} = (m(1), m(\alpha), m(\alpha^2), \dots, m(\alpha^{n-1})). \quad (21)$$

Parameters: $(n, k, d_{\min}) = (2^m - 1, k, n - k + 1)$ — MDS by §10. Rate $R = k/n$. Figure 3 shows the structure. Each symbol is m bits, so RS corrects *byte*-level damage: a burst corrupting 10 consecutive bits hits at most 2–3 symbols, costing 2–3 units of distance instead of 10. This is why RS dominates storage (CDs, DVDs, QR codes) and packet erasure recovery: it is natively a **burst-error and erasure** code.

12.2 Decoding: the pipeline

Receive $\mathbf{r} = \mathbf{c} + \mathbf{e}$ with erasure set E known. Work over $\mathbb{F}_{2^m}$ with $X_j = \alpha^j$.

Step 1 – syndromes. $S_i = \sum_j r_j X_j^i$, $i = 1, \dots, n - k$. Codewords have $S_i = 0$ (they are evaluations of a degree- $< k$ polynomial; equivalently $1, \alpha, \dots$ are roots of the generator polynomial). Nonzero syndromes are pure error evidence.

Step 2 – erasure locator. $\Gamma(x) = \prod_{j \in E} (1 + X_j^{-1}x)$, with roots exactly at the erasure locations. Since the decoder *knows* E , this polynomial is built directly — no search needed. This is the half-price mechanism made mechanical.

Step 3 – modified syndromes. $T_i = \sum_l \gamma_l S_{i+l}$ for $i = 1, \dots, n - k - e$. Because $\Gamma(X_j) = 0$ on erasures, the erasure contributions vanish: T_i are syndromes of an *errors-only* problem with t unknowns.

RS(15,9) codeword: evaluations of $m(x) = m_0 + \dots + m_8x^8$ at $1, \alpha, \dots, \alpha^{14}$

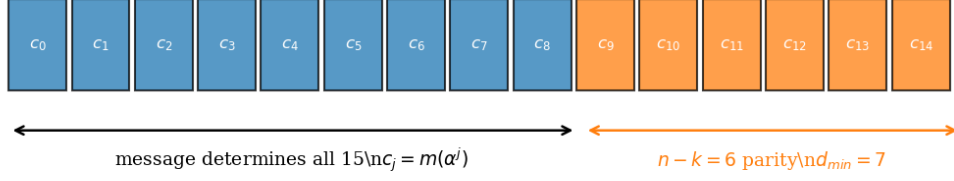


Figure 3: RS(15,9) codeword structure: 9 message symbols determine a degree-8 polynomial; all 15 evaluations are transmitted.

Step 4 – Berlekamp–Massey. Finds the shortest linear recurrence through $T_1, T_2, \dots$; its characteristic polynomial is the **error-locator** $\Lambda(x) = \prod (1 + X_j x)$ over the t unknown error positions. (Conceptually: the syndromes of t errors satisfy a linear recurrence of order t ; BM finds it.)

Step 5 – Chien search. Evaluate $\Lambda(\alpha^{-j})$ for $j = 0, \dots, n - 1$; zeros mark error positions. Exhaustive but tiny.

Step 6 – magnitudes. All corrupted positions are now known (t found + e given, total $\nu \leq n - k$). Solve the Vandermonde system $\sum_j e_j X_j^i = S_i$ ($i = 1, \dots, \nu$) — the “reduced linear system” for known locations. (Equivalently, Forney’s formula; the linear solve is the same mathematics made explicit.)

12.3 Worked example: 1 error + 2 erasures

RS(15,9) over $\mathbb{F}_{16}$ ($d_{\min} = 7$, budget $2t + e \leq 6$). Message $[1, 2, 4, 8, 3, 5, 6, 7, 9]$ encodes to

$$\mathbf{c} = [1, 3, 7, 13, 4, 5, 6, 15, 9, 9, 11, 6, 7, 15, 4].$$

Corrupt: position $3 \oplus 11$ ($13 \rightarrow 6$), erase positions 7, 11. Received:

$$[1, 3, 7, 6, 4, 5, 6, e, 9, 9, 11, e, 7, 15, 4].$$

1. Syndromes: $S_1, \dots, S_6 = [6, 12, 15, 7, 14, 3] \neq 0$ — damage confirmed.
2. Erasure locator ($X_7^{-1} = \alpha^{-7}$, $X_{11}^{-1} = \alpha^{-11}$): $\Gamma(x) = (1 + \alpha^{-7}x)(1 + \alpha^{-11}x) = 1 + 6x + 15x^2$.
3. Modified syndromes: $T_1, \dots, T_4 = [2, 3, 11, 7]$.
4. BM gives $\Lambda(x) = 1 + 8x$ (degree 1 = one error).
5. Chien: $\Lambda(\alpha^{-3}) = 0$ — error at position 3 ($X_3 = \alpha^3 = 8$).
6. Vandermonde solve on positions $\{3, 7, 11\}$:

$$\begin{pmatrix} 8 & 11 & 14 \\ 12 & 9 & 11 \\ 10 & 12 & 8 \end{pmatrix} \begin{pmatrix} e_3 \\ e_7 \\ e_{11} \end{pmatrix} = \begin{pmatrix} 6 \\ 12 \\ 15 \end{pmatrix} \implies (e_3, e_7, e_{11}) = (11, 15, 6), \quad (22)$$

exactly the injected error (11) and the true erased values ($c_7 = 15$, $c_{11} = 6$). Corrected codeword matches.

(All values from `ecc_demo.py`; $2 \cdot 1 + 2 = 4 \leq 6$.)

Intuition check. *Decoding peels the problem in layers: syndromes detect damage, Γ uses the free erasure-location information to delete those unknowns, BM+Chien hunt the remaining errors, and the final linear solve fills in values. Each stage converts one kind of ignorance into a smaller problem.*

12.4 Worked example 2: erasures only

Message $[5, 0, 11, 2, 7, 1, 13, 4, 10]$ encodes to

$$\mathbf{c} = [9, 14, 13, 10, 5, 0, 2, 12, 13, 13, 3, 9, 8, 7, 11].$$

Erase positions 2, 5, 9, 13 ($e = 4 \leq 6$). Syndromes $[15, 10, 7, 11, 9, 1]$. No BM needed ($t = 0$): solve directly

$$\begin{pmatrix} 4 & 6 & 10 & 13 \\ 3 & 7 & 8 & 14 \\ 12 & 1 & 15 & 10 \\ 5 & 6 & 12 & 11 \end{pmatrix} \begin{pmatrix} c_2 \\ c_5 \\ c_9 \\ c_{13} \end{pmatrix} = \begin{pmatrix} 15 \\ 10 \\ 7 \\ 11 \end{pmatrix} \implies (13, 0, 13, 7), \quad (23)$$

the true values. Four parity symbols recover four erased symbols — the $e \leq n - k$ erasure bound saturated exactly.

13 Convolutional codes and the Viterbi algorithm

13.1 The trellis

A convolutional code has **memory**: each output block depends on the current input bit *and* the previous ν input bits (the encoder state). The $(2, 1, 2)$ code with generators $(7, 5)$ (octal): state (s_1, s_2) , input u ,

$$o_1 = u \oplus s_1 \oplus s_2, \quad o_2 = u \oplus s_2, \quad \text{next state } (u, s_1). \quad (24)$$

Unrolling states over time gives the **trellis**: 2^ν states per stage, 2 edges leaving each state (input 0/1), edges labeled with the 2-bit output. The **free distance** d_{free} is the minimum weight of any nonzero trellis path diverging from and remerging with the all-zero path ($d_{\text{free}} = 5$ here) — the convolutional analogue of d_{min} .

13.2 Why Viterbi's pruning is safe

At stage t , many paths reach state s . Viterbi keeps only the **best** (minimum-metric) one per state and discards the rest. This is safe by **optimal substructure**: any continuation from (t, s) onward adds the same future cost regardless of how we reached (t, s) . So if path A beats path B at (t, s) , every extension of B is beaten by the same extension of A — B can never be part of the global optimum. Keeping 2^ν survivors instead of 2^t paths turns exponential search into linear time. (This is dynamic programming, the same principle as Dijkstra's algorithm.)

Intuition check. *The future doesn't care how you got here, only where you are. So at each state and time, among all paths that arrived, only the cheapest can possibly win — the rest are dead weight. Viterbi is Dijkstra on a layered graph.*

13.3 Worked Viterbi decode

Input $[1, 1, 0, 1]$ plus two flush zeros. Transmitted pairs: 11 01 01 00 10 11. Flip two bits (pairs 3 and 5): received 11 01 11 00 11 11. Branch metric = Hamming distance between received pair and edge label; path metric = accumulated min. The decoder's traceback yields

$[1, 1, 0, 1, 0, 0]$ — the two flush bits confirm termination, information $[1, 1, 0, 1]$ recovered despite 2 flips ($< d_{\text{free}}/2 = 2.5$). Final path metrics: state 00: 2, 01: 3, 10: 2, 11: 3; the 00 survivor is the ML path (Figure 4).

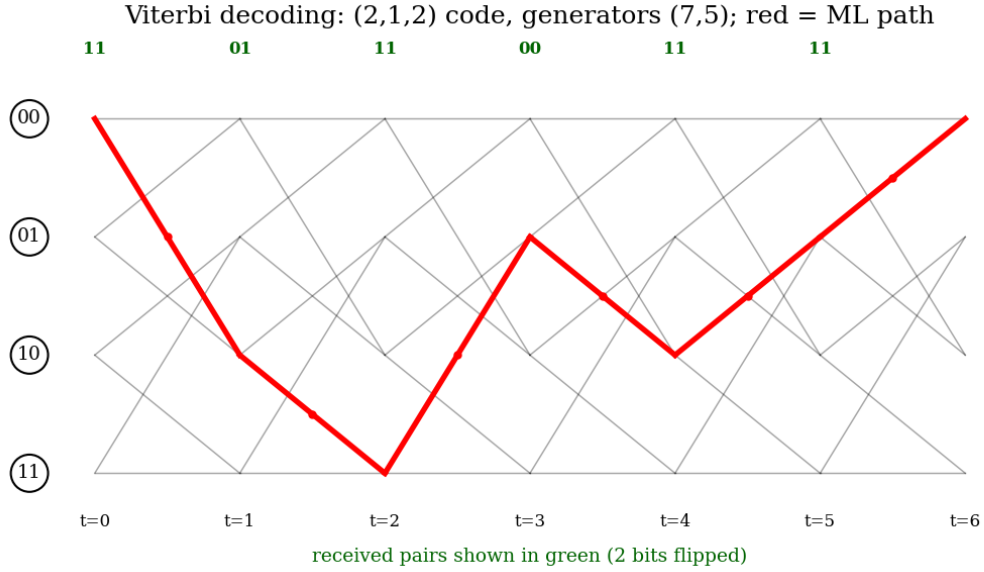


Figure 4: Viterbi decoding on the (2, 1, 2) trellis. Gray: all edges; red: maximum-likelihood survivor path. Green: received pairs.

14 Modern codes: LDPC, turbo, polar

Classical codes leave a gap between the Hamming and GV bounds. Three families (nearly) close it, all decoded **iteratively** with soft information.

14.1 LDPC: sparse parity checks, belief propagation

A low-density parity-check code has a parity-check matrix H that is **sparse** (few ones per row/column), drawn as a Tanner graph (Figure 1 shows the same idea for Hamming(7,4); LDPC graphs scale this up with no short cycles). Decoding passes **beliefs** (log-likelihood ratios) along edges:

$$\text{variable} \rightarrow \text{check: } L_{v \rightarrow c} = L_v^{\text{ch}} + \sum_{c' \neq c} L_{c' \rightarrow v}, \quad (25)$$

$$\text{check} \rightarrow \text{variable: } \tanh\left(\frac{1}{2}L_{c \rightarrow v}\right) = \prod_{v' \neq v} \tanh\left(\frac{1}{2}L_{v' \rightarrow c}\right). \quad (26)$$

The variable update is “add up independent evidence”; the check update is the LLR-domain form of “the parity of the other bits determines this one” (tanh rule). Iterate until all parity checks pass or a limit is hit. Why it works: on a tree, this computes exact posteriors; LDPC graphs are locally tree-like, and sparsity keeps iteration cheap ($O(n)$ per round).

Intuition check. *Each bit asks its parity checks “what do you think I am?”, each check answers using what the other bits told it, and everyone updates. Sparse connections mean each conversation is simple; many rounds let information diffuse across the whole block.*

14.2 Turbo codes: parallel concatenation

Encode the data twice: once directly, once interleaved, with two small convolutional encoders; transmit data + both parity streams. Decode iteratively: decoder 1 produces **extrinsic** LLRs (its opinion minus what it was told), interleaves them as *a priori* input to decoder 2, which returns its own extrinsic information, and so on. Each decoder sees the channel *plus* the other's independent view; the interleaver decorrelates their error patterns so they keep helping each other. Turbo codes were the first to operate within a fraction of a dB of the Shannon limit.

14.3 Polar codes: channel polarization

Apply the 2×2 kernel $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ recursively $n = 2^m$ times. As $m \rightarrow \infty$, the n synthetic channels **polarize**: each becomes either nearly noiseless or nearly useless, with the fraction of good ones $\rightarrow$ capacity C . Transmit data on the good channels, freeze the bad ones to 0 — achieving capacity with $O(n \log n)$ successive-cancellation decoding. The first codes *proved* to achieve capacity, now in 5G control channels.

15 Erasures only: fountain codes

On a packet network, packets either arrive perfectly (checksums) or vanish: the **binary erasure channel** at packet granularity. Fixed-rate codes must pick a rate for the *worst* expected loss; fountain codes are **rateless** — they generate a limitless stream of encoded symbols, and the receiver decodes once it collects slightly more than k of them, *whatever* the loss rate.

15.1 LT codes

Encode: to make one symbol, sample a degree d from a distribution μ , pick d random source symbols, XOR them. **Decode (peeling)**: find an encoded symbol of degree 1 (it *is* a source symbol); XOR it out of every symbol containing it (reducing their degrees); repeat. The process needs a steady supply of degree-1 symbols — the **ripple**.

The **robust soliton distribution** shapes μ to keep the ripple alive:

$$\rho(d) = \begin{cases} 1/k & d = 1 \\ 1/[d(d-1)] & d \geq 2 \end{cases}, \quad \tau(d) = \begin{cases} R/(dk) & 1 \leq d < k/R \\ R \ln(R/\delta)/k & d = k/R \\ 0 & \text{else} \end{cases} \quad (27)$$

$$\mu(d) = \frac{\rho(d) + \tau(d)}{\sum_{d'} (\rho(d') + \tau(d'))}, \quad R = c \ln(k/\delta) \sqrt{k}. \quad (28)$$

ρ is the ideal soliton (expected ripple size 1); τ boosts low degrees to keep the ripple $\approx R$ symbols wide so it never dies. Decoding succeeds with $k + O(\sqrt{k} \ln^2(k/\delta))$ received symbols. Figure 5 plots μ ; Figure 6 walks through peeling.

15.2 Peeling walkthrough

Source $S_1, \dots, S_4$; received $E_1=S_1$, $E_2=S_1 \oplus S_2$, $E_3=S_3$, $E_4=S_2 \oplus S_3 \oplus S_4$, $E_5=S_4$, $E_6=S_1 \oplus S_4$:

1. $E_1 \rightarrow S_1$, $E_3 \rightarrow S_3$ (degree-1 symbols).
2. $E_2 \oplus S_1 \rightarrow S_2$.
3. $E_4 \oplus S_2 \oplus S_3 \rightarrow S_4$ (E_5 confirms).

All four recovered from six symbols; any two could have been lost.

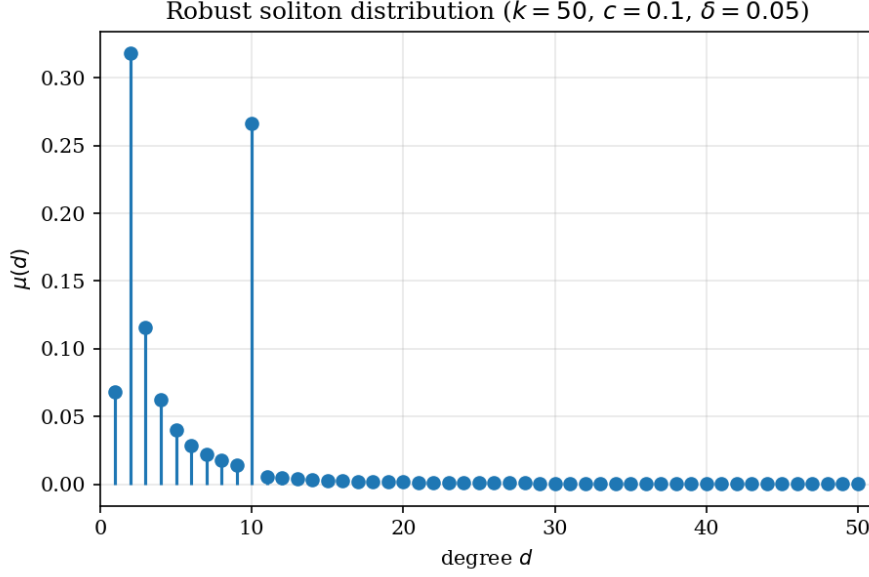


Figure 5: Robust soliton degree distribution ($k = 50$). The spike at $d = 1$ feeds the ripple; the heavy tail covers stragglers.

15.3 When peeling stalls (second example)

S_1, S_2, S_3 with $E_1 = S_1 \oplus S_2$, $E_2 = S_2 \oplus S_3$, $E_3 = S_1 \oplus S_2 \oplus S_3$: *every* symbol has degree ≥ 2 — no ripple, peeling cannot start, decoding fails despite having 3 equations for 3 unknowns. This is exactly why μ needs its degree-1 spike: the ripple is not a luxury, it is the ignition. **RaptorQ** (used in real multicast/broadcast standards) fixes this robustly by pre-coding with an LDPC layer, so peeling essentially never stalls.

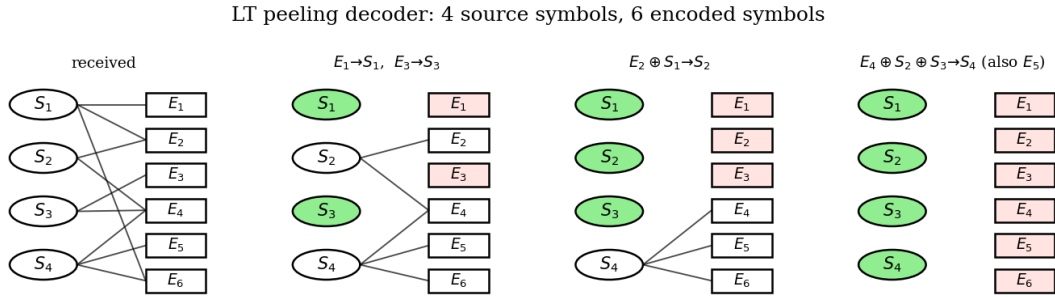


Figure 6: LT peeling decoder: green = recovered source symbols, pink = processed encoded symbols. Edges to recovered symbols are XORed away.

Intuition check. A fountain code never asks “how bad is the channel?” — it just keeps transmitting new linear combinations until the receiver says “got it.” Peeling works because degree-1 symbols are free gifts; the soliton distribution is engineered to keep those gifts coming until every source symbol is unwrapped.

16 Shannon limits and the road not taken

Figure 7 shows the two canonical capacities: the BSC needs $1 - h_2(p)$ bits per use (each flipped bit costs entropy), the BEC needs only $1 - \varepsilon$ (an erasure costs exactly its share — compare the

2:1 price ratio of §1). “Approaching capacity” means a code family whose rate R gets arbitrarily close to C with vanishing error probability as $n \rightarrow \infty$ — turbo, LDPC, and polar codes all do this; the plot is the target they aim at.

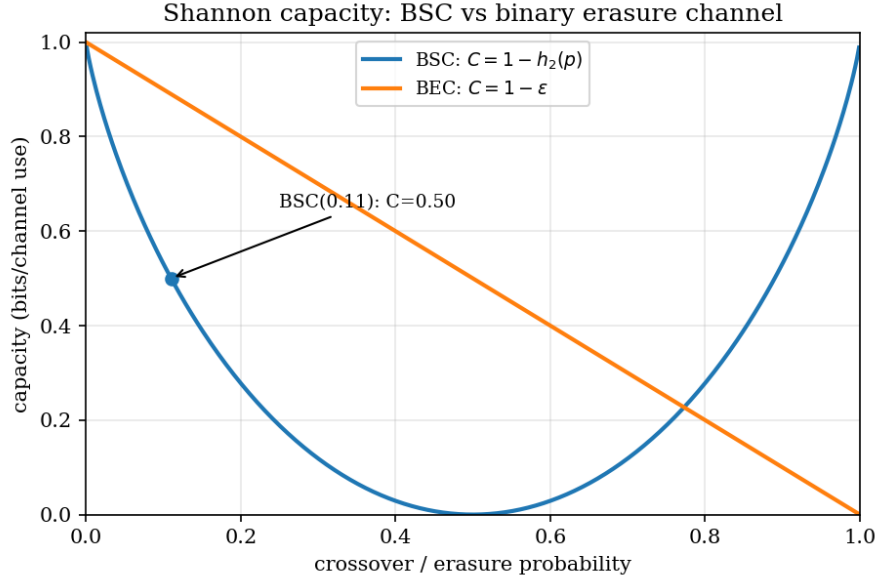


Figure 7: BSC capacity $1 - h_2(p)$ vs. BEC capacity $1 - \epsilon$. At $p = \epsilon = 0.11$, the BEC carries 0.89 bits/use while the BSC carries only 0.50 — knowing *where* is worth 0.39 bits.

16.1 Separate coding vs. joint coding (COVQ)

Everything in this guide is **separate** coding: compress the source, then add redundancy against a fixed channel model. Separation buys a clean guarantee — *any* source, *any* channel below capacity, vanishing error — at three prices: redundancy bits, delay (long blocks approach capacity), and the **cliff effect** (performance collapses past the design point). The COVQ philosophy of `jsc-guide.pdf` refuses the split: it designs the quantizer to minimize end-to-end distortion *over the noisy channel itself*, degrading gracefully where separate coding falls off a cliff. Neither dominates; the right choice depends on whether your channel is known and your delay budget finite. The erasure-decoding machinery here (3^L -pattern COVQ decoders, erasures as free soft information) is the bridge between the two worlds.

A Cheat sheet

Hamming ball volume:	$V(n, t) = \sum_{i=0}^t \binom{n}{i}$
Hamming bound:	$A(n, d) \leq 2^n / V(n, \lfloor \frac{d-1}{2} \rfloor)$
Singleton bound:	$k \leq n - d + 1$ (MDS = equality)
Errors + erasures:	$2t + e < d_{\min}$
Syndrome:	$\mathbf{s} = \mathbf{r}H^T = \mathbf{e}H^T$
CRC:	valid words are multiples of $g(x)$
RS:	$c_j = m(\alpha^j)$, $d_{\min} = n - k + 1$
RS syndromes:	$S_i = \sum_j r_j \alpha^{ij}$
Erasure locator:	$\Gamma(x) = \prod_{j \in E} (1 + \alpha^{-j}x)$
Modified syndromes:	$T_i = \sum_l \gamma_l S_{i+l}$
Error locator (BM):	$\Lambda(x) = \prod_{\text{errors}} (1 + X_j x)$
Chien:	$\Lambda(\alpha^{-j}) = 0 \Rightarrow \text{error at } j$
Viterbi:	keep best path per state (optimal substructure)
LDPC check update:	$\tanh(\frac{1}{2}L_{c \rightarrow v}) = \prod_{v' \neq v} \tanh(\frac{1}{2}L_{v' \rightarrow c})$
LT robust soliton:	$\mu \propto \rho + \tau$, $R = c \ln(k/\delta) \sqrt{k}$
BSC capacity:	$C = 1 - h_2(p)$
BEC capacity:	$C = 1 - \varepsilon$

Part V — Real Codecs: How Images Are Actually Compressed

`image-pipeline.pdf` (13 pages)

From mathematics to engineering: how production image formats really work. Raw storage, RGB/YCbCr, chroma subsampling, then JPEG's complete pipeline (markers, quantization, zigzag, Huffman, byte stuffing), PNG (filtering, DEFLATE, chunks and CRCs), and the modern formats, plus packetization, restart markers, and measured compression experiments on a deterministic test image. This is the practical half of the source-coding story from Parts II–III — and the baseline your channel-optimized coder will be compared against, so read it with the evaluation chapter of Part I (§15) in mind. Note the errata on Pillow's `optimize` default, Annex K tables, and PNG filter selection.

From Pixels to Bits to Pixels

How Images Are Stored, Compressed, Sent, and Decoded

A practical companion to the joint source–channel coding theory guide

September 2026

Abstract

This document is the practical sibling of the theory guide on joint source–channel coding. Instead of algorithms for noisy channels, it covers what actually happens to an image in the real world: how pixels are stored in memory, why we convert RGB to YCbCr and throw away half the color data, exactly how a JPEG file is built byte by byte (DCT, quantization tables, zigzag scan, Huffman coding, file markers) and taken apart again, how PNG’s filters and DEFLATE work, what WebP and AVIF changed, and how the compressed bits travel across a network—and what breaks when they don’t arrive intact. Every performance number is measured: a 512×512 test image was compressed at five JPEG qualities (plus PNG and WebP), and the file sizes, compression ratios, and PSNR values in the tables are the actual outputs. Read this alongside the theory guide: that document explains how to design for a noisy channel; this one explains the clean-pipe world those designs are trying to improve on.

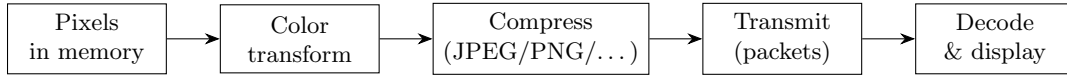
Contents

1	Introduction	3
2	How images are stored	3
2.1	Pixels, bit depth, and the size arithmetic	3
2.2	Color spaces: why we leave RGB	3
2.3	Chroma subsampling: throwing away half the color	4
3	JPEG, end to end	5
3.1	Level shift and the 2-D DCT	5
3.2	Quantization: where the loss happens	5
3.3	Zigzag scan	6
3.4	DC differential coding and AC run-length coding	6
3.5	Huffman coding: the last squeeze	7
3.6	The file: markers and byte layout	7
4	PNG: lossless via prediction + dictionary coding	8
4.1	The five filters	8
4.2	DEFLATE = LZ77 + Huffman	9
4.3	When PNG wins and when it loses	9
5	Modern codecs: WebP, HEIC, AVIF	9
6	Transmission: packets, errors, and why one bad bit hurts	10
6.1	From file to packets	10
6.2	Why a single flipped bit can destroy the rest of the image	10
6.3	Contrast with the theory guide	11

7	Measured experiments	11
7.1	Method	11
7.2	Results	11
8	Summary cheat sheet	12

1 Introduction

An image on your screen is an illusion built from numbers. A photo file is a much smaller pile of numbers plus instructions for rebuilding the illusion. Getting from one to the other—and back, after a trip across the internet—involves five distinct stages:



Each stage is engineered around a fact about human vision or about statistics, and each has failure modes the next stage must survive. This document walks the chain left to right, with the formulas, the byte layouts, and measured numbers.

How the numbers in this document were produced

All measurements use a synthetic 512×512 RGB test image (gradient sky, mountains, a red barn with sharp edges, text, and a color strip; deterministic seed) generated with Python PIL 10.2. JPEG encodes used 4:4:4 sampling (no chroma subsampling) to isolate the effect of the quality setting. PSNR is computed against the lossless reference over all three channels: $\text{PSNR} = 20 \log_{10}(255/\sqrt{\text{MSE}})$. Raw size is $512 \cdot 512 \cdot 3 = 786,432$ bytes.

2 How images are stored

2.1 Pixels, bit depth, and the size arithmetic

A digital image is a rectangular grid of **pixels**. In the standard 8-bit RGB representation each pixel is three bytes: one each for red, green, and blue intensity, 0–255. In memory (and in uncompressed formats like BMP) pixels are stored **row-major**: all of row 0 left to right, then row 1, and so on—a flat array of $W \cdot H \cdot 3$ bytes.

The size arithmetic is worth internalizing, because every compression ratio is measured against it:

$$\text{raw bytes} = W \times H \times \text{bytes per pixel}. \quad (1)$$

A 1920×1080 (1080p) photo at 3 bytes/pixel is $1920 \cdot 1080 \cdot 3 = 6,220,800$ bytes ≈ 5.93 MiB. Our 512×512 test image is 786,432 bytes. At 30 frames per second, raw 1080p video would need ≈ 187 MB/s—which is why nothing is ever stored or sent raw.

Bit depth is the other axis: 8 bits per channel is the default; 10–16 bits per channel (“deep color,” RAW photo formats) is used where banding in smooth gradients would be visible. More bits per pixel means proportionally more raw bytes, and proportionally more room for compression to work.

2.2 Color spaces: why we leave RGB

RGB matches the display (three colored subpixels), but it is a poor space to compress in: the three channels are highly correlated (a bright pixel is bright in all three), and the eye does not treat them equally. Human vision resolves **brightness detail** far better than **color detail**. So nearly every image and video codec converts to a luma/chroma space—**YCbCr**—where *Y* is brightness and *Cb*, *Cr* are blue-yellow and red-green color differences. The JPEG/JFIF conversion is a fixed linear map:

$$\begin{aligned} Y &= 0.299 R + 0.587 G + 0.114 B \\ Cb &= 128 - 0.168736 R - 0.331264 G + 0.5 B \\ Cr &= 128 + 0.5 R - 0.418688 G - 0.081312 B \end{aligned} \quad (2)$$

The weights (0.299, 0.587, 0.114) are the eye’s relative sensitivity to red, green, and blue—green dominates brightness perception. The inverse used by the decoder is

$$\begin{aligned} R &= Y + 1.402 (Cr - 128) \\ G &= Y - 0.344136 (Cb - 128) - 0.714136 (Cr - 128) \\ B &= Y + 1.772 (Cb - 128). \end{aligned} \tag{3}$$

What this buys: after the transform, almost all the structural detail lives in Y alone. Cb and Cr are smooth, low-energy signals—which means we can compress them brutally (next section) with little visible damage. In RGB, the same trick would smear brightness edges and look terrible.

2.3 Chroma subsampling: throwing away half the color

Because the eye barely notices color detail, JPEG/MPEG downsample the chroma channels. The notation is $J:a:b$: in each $J \times 2$ pixel block, a chroma samples are kept in the top row and b in the bottom row, per chroma channel:

Format	Chroma resolution	Raw bits/pixel	Data kept
4:4:4	full ($W \times H$)	24	everything
4:2:2	half horizontal ($W/2 \times H$)	16	2/3
4:2:0	half both ($W/2 \times H/2$)	12	1/2

4:2:0—the default for JPEG photos and all consumer video—stores one Cb and one Cr sample per 2×2 pixel quad: **half the total data** of 4:4:4 before compression even starts. The decoder upsamples (usually bilinear) back to full resolution.

Why the eye tolerates it: color edges get slightly soft, but brightness edges—which define perceived sharpness—are untouched. The failure case is saturated red/blue text or thin colored lines, where subsampling visibly bleeds. Figure 1 shows a measured demo: subsampling our test image’s chroma to 4:2:0 and back costs only the color edges, with whole-image PSNR of 30.21 dB for the subsampling step alone.

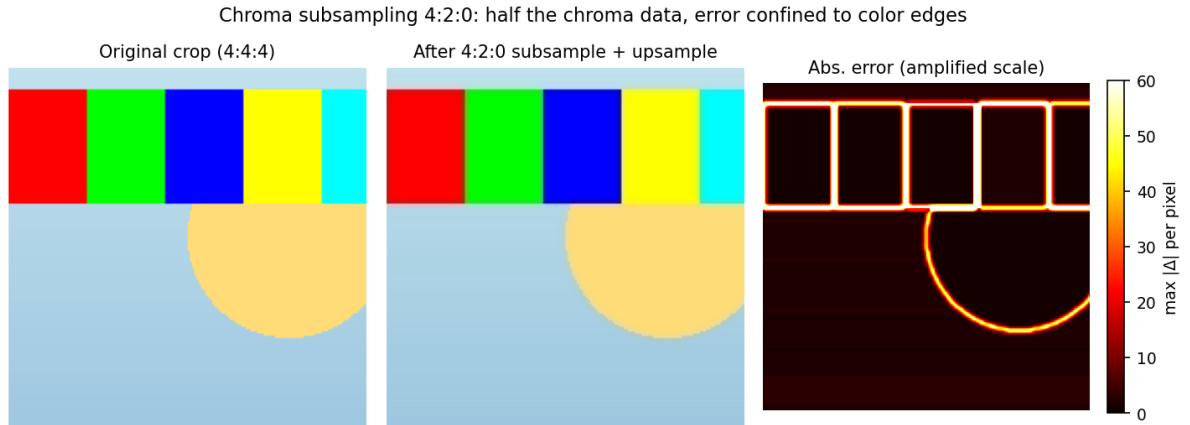
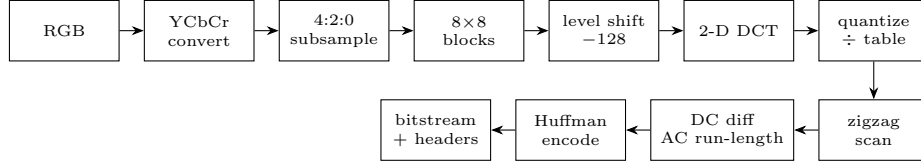


Figure 1: Measured 4:2:0 demo on the color-strip region. Left: original. Middle: Cb/Cr halved in each dimension, then bilinear-upsampled. Right: absolute error (note the scale)—error lives only at color boundaries; flat color areas are bit-identical.

3 JPEG, end to end

JPEG (baseline DCT mode, the one every decoder supports) is a pipeline of individually simple steps. Here is the whole encoder; the decoder is the mirror image.



3.1 Level shift and the 2-D DCT

Each 8×8 block of a component is level-shifted by -128 , mapping $[0, 255]$ to $[-128, 127]$. This centers the data so the DCT's DC coefficient measures the block's mean brightness rather than an arbitrary offset. Then the 2-D DCT-II:

$$F(u, v) = \frac{1}{4}C(u)C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}, \quad (4)$$

with $C(0) = 1/\sqrt{2}$, $C(k) = 1$ otherwise. The 64 basis images are shown in Figure 2: $F(0, 0)$ (top-left) is the flat DC term; moving right/down adds higher spatial frequencies. Natural image blocks concentrate energy in the top-left—that compaction is the entire reason the DCT is there (see the theory guide's KLT chapter for why this particular transform is near-optimal).

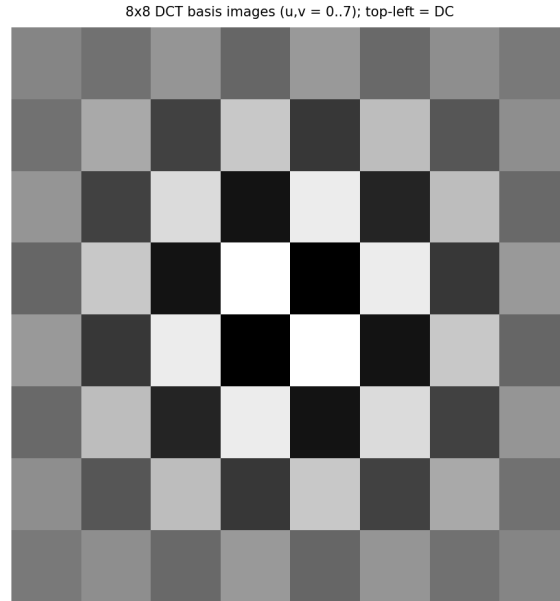


Figure 2: The 64 8×8 DCT basis images. Each block's 64 pixels are a weighted sum of these patterns; the weights are the DCT coefficients. Top-left is DC (mean brightness).

3.2 Quantization: where the loss happens

This is the only lossy step in baseline JPEG. Each coefficient is divided by a table entry and rounded:

$$F_q(u, v) = \text{round} \left(\frac{F(u, v)}{Q(u, v)} \right). \quad (5)$$

The standard **luminance quantization table** (quality 50) is shown in Figure 3. Read it as a sensitivity map: small entries (top-left, e.g. 16, 11, 10) quantize finely because the eye sees low-frequency errors clearly; large entries (bottom-right, up to 121) quantize coarsely because high-frequency errors are nearly invisible. The chrominance table is similar but uniformly coarser (the eye forgives color errors even more).

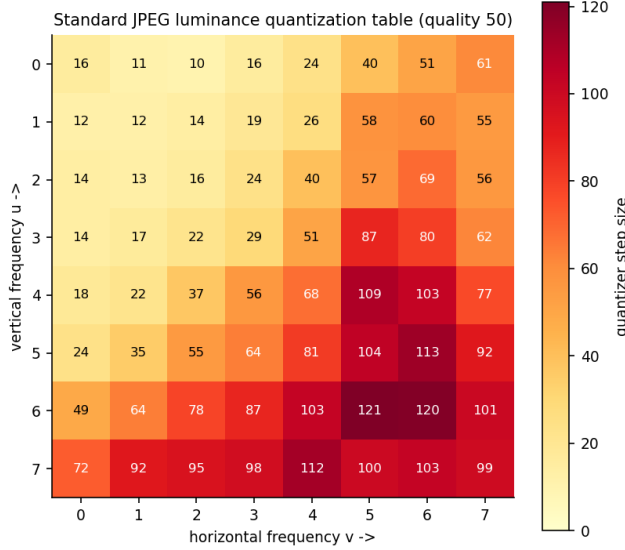


Figure 3: Standard JPEG luminance quantization table at quality 50. Each entry is the step size for that coefficient’s uniform quantizer: 16 for DC, growing to 121 for the highest frequencies. After division, most high-frequency coefficients round to zero.

What the “quality” slider does. It scales this table. The IJG (Independent JPEG Group) rule, used by nearly every encoder:

$$S = \begin{cases} 5000/Q & Q < 50 \\ 200 - 2Q & Q \geq 50 \end{cases}, \quad Q_{\text{scaled}}(u, v) = \text{clamp}\left(\text{round}\left(\frac{Q_{50}(u, v) \cdot S}{100}\right), 1, 255\right). \quad (6)$$

At $Q=75$, $S = 50$: every entry halves (DC step $16 \rightarrow 8$). At $Q=10$, $S = 500$: DC step $16 \rightarrow 80$ —only ~ 3 distinct DC levels span the whole brightness range, which is exactly why Figure 8 shows flat posterized blocks at $Q=10$. There is no universal meaning to “ $Q=75$ ” across encoders, but the scaling direction is the same: lower Q = bigger steps = more zeros = smaller file.

Decoder side: dequantization multiplies back, $\hat{F}(u, v) = F_q(u, v) \cdot Q(u, v)$. The rounding error is permanent—this is the distortion the theory guide analyzes.

3.3 Zigzag scan

The 64 quantized coefficients are reordered into a 1-D sequence by the zigzag scan (Figure 4): DC first, then increasing spatial frequency. *Why:* after quantization the sequence typically looks like [big, small, small, 0, 0, 0, ...]—a few nonzero low-frequency coefficients followed by a long tail of zeros. The scan turns the 2-D structure into the 1-D runs that the next step exploits.

3.4 DC differential coding and AC run-length coding

DC: neighboring blocks have similar mean brightness, so JPEG codes differences: $\text{DIFF}_i = DC_i - DC_{i-1}$. The symbol stored is $\text{SSSS} = \text{number of bits needed for } |\text{DIFF}|$ (its “size

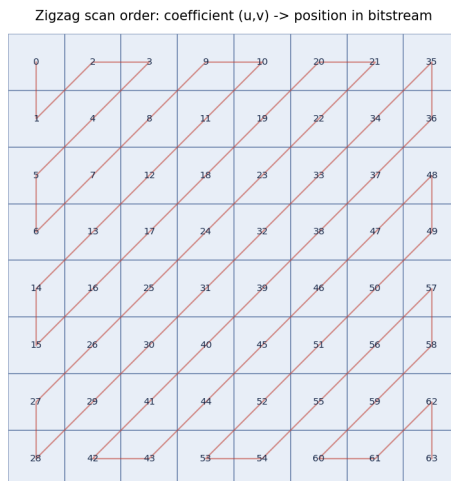


Figure 4: Zigzag scan order: coefficient (u,v) maps to the numbered position in the 1-D sequence. DC (top-left) goes first; the scan then visits coefficients in order of increasing spatial frequency, so the long runs of quantized zeros end up contiguous at the tail.

category”); the decoder reads SSSS additional bits to recover the exact difference and adds back the previous DC.

AC: each nonzero coefficient in the zigzag sequence becomes a symbol (RRRR,SSSS): RRRR = count of preceding zeros (0–15), SSSS = size category of the coefficient, followed by SSSS literal bits. Two special symbols: EOB = (0,0) ends the block (“everything after is zero”—this is where the compression really comes from), and ZRL = (15,0) means “16 zeros, keep going” for runs longer than 15.

Decoder side: it reads EOB and simply fills the rest of the block with zeros—no bits were ever spent on them.

3.5 Huffman coding: the last squeeze

The (RRRR,SSSS) and DC-SSSS symbols are coded with Huffman codes: frequent symbols (EOB, small coefficients) get short codes, rare ones get long codes. *How the tables work:* the DHT marker segment stores, for each of up to 4 tables, 16 bytes giving the **count of codes of each length 1–16**, followed by the symbol values in code-length order. From the counts alone the decoder reconstructs the canonical Huffman tree—the actual bit patterns are never stored. *Baseline vs. optimized:* baseline encoders use the fixed “standard” tables from the JPEG spec’s Annex K (no DHT needed in theory, though encoders usually still write them); **optimized Huffman** builds custom tables from the image’s actual symbol frequencies in a first pass and stores them in DHT—typically ~5% smaller files for the same pixels. PIL’s encoder writes optimized tables by default.

After Huffman coding, bytes of value 0xFF in the scan data are stuffed as 0xFF 0x00 so the decoder never mistakes data for a marker—a detail that matters in Section 6.

3.6 The file: markers and byte layout

A JPEG file is a sequence of **segments**, each starting with a 0xFF marker byte. Figure 5 shows the layout schematically; Table 1 gives the *actual* measured offsets from our Q=75 test file (25,124 bytes total).

Decoder walk-through: read SOI; parse APP0 (confirm JFIF); load the two quantization tables from DQT; read dimensions and component sampling from SOF0; build the four Huffman

JPEG file layout (schematic, widths ~ byte counts for a small baseline file)



Figure 5: Schematic JPEG layout. Headers (~600 bytes here) describe everything; the entropy-coded scan is the bulk. Real offsets for our Q=75 file are in Table 1.

Table 1: Measured marker layout of the 25,124-byte Q=75 test file.

Offset	Marker	Name	Length (B)	Contents
0	FFD8	SOI	2	start of image
2	FFE0	APP0	18	“JFIF” identifier, version, density
20	FFDB	DQT	69	luminance quantization table (64 B + header)
89	FFDB	DQT	69	chrominance quantization table
158	FFC0	SOF0	19	baseline frame: 512×512, 3 components, sampling
177	FFC4	DHT	33	Huffman table (DC luma)
210	FFC4	DHT	183	Huffman table (AC luma)
393	FFC4	DHT	33	Huffman table (DC chroma)
426	FFC4	DHT	183	Huffman table (AC chroma)
609	FFDA	SOS	14	start of scan: which tables each component uses
623	—	scan data	24,499	the Huffman-coded coefficients (97.5% of the file)
25122	FFD9	EOI	2	end of image

tables from DHT; at SOS, decode the scan—Huffman-decode each symbol, undo run-length and DC differences, de-zigzag into 8×8 blocks, dequantize (multiply by table), inverse DCT, add back 128, upsample chroma, YCbCr→RGB via (3), done. Every step is the exact inverse of the encoder except dequantization, which cannot undo the rounding.

4 PNG: lossless via prediction + dictionary coding

Where JPEG is lossy and frequency-domain, PNG is lossless and spatial-domain. Its pipeline is: **filter** each scanline (predict each pixel from its neighbors, store only the residual), then compress the residuals with **DEFLATE**.

4.1 The five filters

For pixel x with left neighbor a , above neighbor b , and upper-left c , the encoder stores $x - \text{pred}$:

Filter	Prediction
0: None	0
1: Sub	a
2: Up	b
3: Average	$\lfloor (a+b)/2 \rfloor$
4: Paeth	Paeth(a, b, c): the nearest of a, b, c to $a+b-c$

The decoder adds the prediction back—exactly, so nothing is lost. *Why it helps:* residuals cluster near zero (Figure 6, left), and near-zero values have far lower entropy than raw pixels. PNG tries all five filters **per row** and keeps the one with the smallest sum of absolute residuals—measured on our test image, the winner varies by row content (Up wins on the smooth sky gradient, $200\times$ reduction in $\sum |\text{res}|$; Average wins on textured rows, $\sim 15\times$), which is exactly why the choice is adaptive rather than fixed.

Why PNG filters: prediction residuals are cheaper to encode than raw pixels

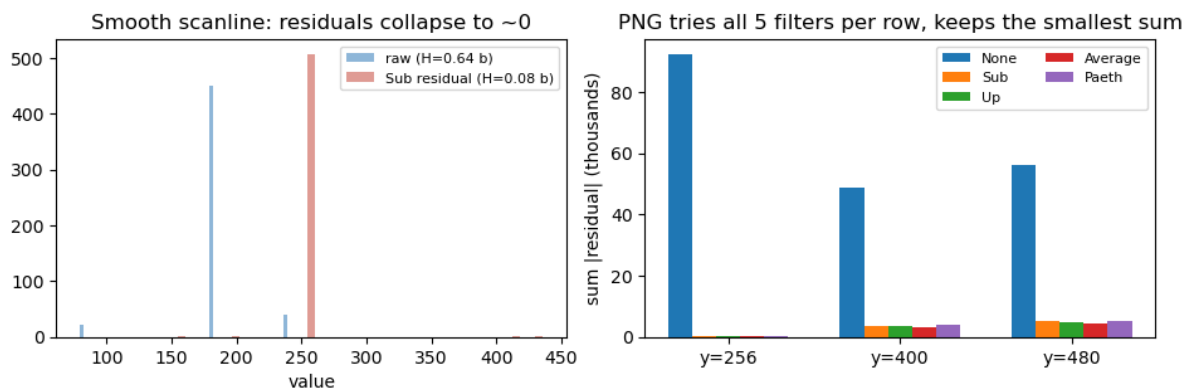


Figure 6: Left: on a smooth scanline, Sub-filter residuals collapse to ≈ 0 (entropy 0.64 $\rightarrow$ 0.08 bits/pixel). Right: measured $\sum |\text{residual}|$ per filter for three rows—the best filter depends on image content, so PNG selects per row.

4.2 DEFLATE = LZ77 + Huffman

The filtered byte stream is compressed with DEFLATE (the same algorithm as gzip/zip): **LZ77** replaces repeated byte strings with (length, backward-distance) references—long runs of near-zero residuals match constantly—and **Huffman** codes the literals and match descriptors by frequency. PNG’s compression lives almost entirely here; the filter’s job is to reshape the data so LZ77+Huffman have something easy to chew on.

4.3 When PNG wins and when it loses

Measured on our test image: PNG = 161,663 bytes (4.86:1, lossless) versus JPEG Q=95 = 76,323 bytes (10.3:1, 32.11 dB). The rule:

- **PNG wins** on graphics, screenshots, text, line art, and any image with large flat areas and sharp edges—content where prediction is nearly perfect and JPEG’s DCT would ring. And it is bit-exact, which matters for archival and editing.
- **JPEG wins** on photographs and anything with sensor noise or fine texture—content where prediction fails but the eye forgives frequency-domain coarsening. Typically $3\text{--}10\times$ smaller than PNG at visually transparent quality.

5 Modern codecs: WebP, HEIC, AVIF

JPEG’s 1992 design leaves compression on the table; newer codecs pick it up with three ideas: **better prediction** (predict each block from already-decoded neighbors instead of coding it blind), **better transforms** (larger, adaptive block sizes; directional modes), and **better entropy coding** (arithmetic/range coding instead of Huffman, with adaptive contexts).

Measured on our test image (PIL encoder, 4:4:4):

Codec	Setting	Bytes	Ratio	PSNR (dB)
JPEG	Q=75	25,124	31.3:1	30.20
WebP	Q=75	12,660	62.1:1	29.90
JPEG	Q=50	15,490	50.8:1	29.57
WebP	Q=50	6,262	125.6:1	29.45

At matched PSNR (≈ 30 dB), WebP needs roughly **half the bytes** of JPEG on this image—consistent with Google’s reported 25–35% savings on photographic corpora (ours is synthetic and prediction-friendly, so it does even better).

HEIC/AVIF go further: they are still images wrapped from video codecs (HEVC and AV1 intra frames), adding large coding-tree blocks (up to 128×128), dozens of directional intra-prediction modes, and context-adaptive binary arithmetic coding. Reported results (AO-Media/Netflix evaluations) put AVIF at roughly **50% smaller than JPEG at the same visual quality**—about another $2\times$ beyond WebP. The price is encode complexity (an order of magnitude slower than JPEG) and, for years, decoder support—now largely resolved in browsers.

6 Transmission: packets, errors, and why one bad bit hurts

6.1 From file to packets

A 25,124-byte JPEG does not travel as one piece. Over Ethernet/Wi-Fi the MTU is $\approx 1,500$ bytes, so with IP/TCP headers (≈ 40 bytes) each packet carries $\approx 1,460$ payload bytes: our Q=75 file needs $\lceil 25124/1460 \rceil = 18$ packets. Two regimes:

- **TCP** (web browsing, messaging): lost packets are retransmitted; the application receives the *exact* file. JPEG’s design assumes this—the decoder was never built to tolerate bit errors.
- **UDP/RTP** (video calls, live streaming): no retransmission; lost or corrupted packets are simply missing. Here the codec must cope, which is why video codecs have resync features JPEG mostly lacks.

6.2 Why a single flipped bit can destroy the rest of the image

JPEG’s scan data is a Huffman-coded symbol stream with **no built-in boundaries**: the decoder finds symbol edges by walking the Huffman tree bit by bit. Flip one bit and the decoder takes a wrong branch—it consumes the wrong number of bits, so *every subsequent symbol* is misparsed: wrong run lengths, wrong DC differences (which accumulate, since DC is differential), wrong block boundaries. The corruption typically runs to the end of the scan. (This is the exact failure mode the theory guide’s noisy-channel work is designed around: JPEG assumes a clean pipe and collapses without one.)

The band-aid is **restart intervals**: a DRI marker segment (FFD0) declares “insert RSTn markers every R_i MCUs,” and the encoder drops FFD0–FFD7 (cycling) into the scan. On detecting garbage, the decoder scans forward for the next RSTn marker, resynchronizes there,

and resets the DC predictors—so damage is contained to one interval instead of the whole image. Most encoders (including PIL’s default) don’t write restart markers; the web relies on TCP instead.

6.3 Contrast with the theory guide

The two documents describe opposite philosophies:

	This document (JPEG/PNG/WebP)	Theory guide (COVQ/JSCC)
Channel assumed	clean (TCP fixes errors underneath)	raw and noisy, with memory
Error strategy	prevent (retransmit) or contain (restart markers)	<i>design for</i> : minimize end-to-end distortion over the noisy channel
Graceful degradation	no—one bad bit can kill the scan	yes—posterior-mean decoder hedges
Cost	retransmission delay, or fragile files	channel-specific codebooks

Neither is “right”; they answer different questions. If you have a reliable transport, JPEG’s separation is near-optimal and far simpler. If you have a raw wireless link with no retransmission budget, the theory guide’s joint design is where the dBs come from.

7 Measured experiments

7.1 Method

Test image: synthetic 512×512 RGB (sky gradient, mountains, barn with sharp edges, overlaid text, color strip), generated deterministically with `numpy.random.default_rng(7)`. Reference stored lossless (PNG). JPEG encodes via PIL at qualities 95/75/50/25/10 with 4:4:4 subsampling (isolating quality from subsampling); WebP via PIL at 75/50. PSNR computed on all three channels vs. the lossless reference. Raw = 786,432 bytes.

7.2 Results

Table 2: Measured JPEG rate–distortion on the 512×512 test image (4:4:4).

Quality	File (bytes)	Bits/pixel	Ratio	PSNR (dB)
95	76,323	2.329	10.3:1	32.11
75	25,124	0.767	31.3:1	30.20
50	15,490	0.473	50.8:1	29.57
25	10,192	0.311	77.2:1	28.72
10	7,387	0.225	106.5:1	26.80

Reading the numbers: dropping from $Q=95$ to $Q=75$ cuts the file by $3 \times$ (76 KB $\rightarrow$ 25 KB) for only 1.9 dB PSNR—the knee of the curve, and the reason $Q \approx 75$ is the universal default. $Q=10$ reaches 106:1 but the image falls apart (26.8 dB, heavy blocking). For reference: PNG lossless = 161,663 bytes (4.86:1); 4:2:0 chroma subsampling alone (no JPEG) = 30.21 dB PSNR, i.e. most of JPEG’s visible quality budget at $Q=75$ is spent on quantization, not subsampling.

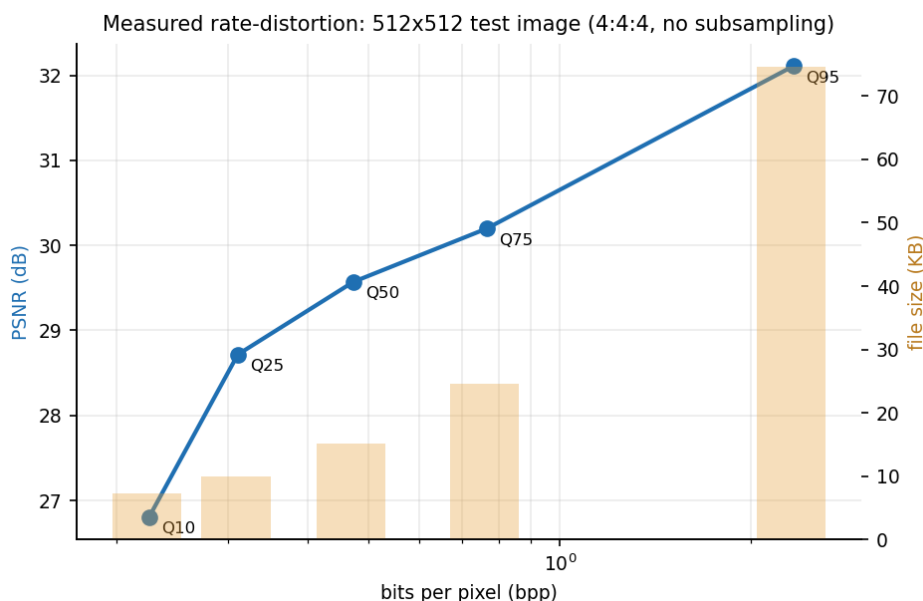


Figure 7: Measured rate-distortion curve (log bitrate axis). The knee sits near $Q=50-75$: below it, quality falls faster than the file shrinks—which is why $Q \approx 75$ is the web’s default.

8 Summary cheat sheet

Concept	One-line version
Raw storage	$W \times H \times 3$ bytes, row-major; 1080p ≈ 5.93 MiB
YCbCr	decorrelates brightness from color using eye-sensitivity weights (0.299/0.587/0.114)
4:2:0	half-resolution chroma; halves raw data; error only at color edges
Level shift	-128 centers pixels for the DCT
DCT	compacts block energy into top-left coefficients
Quant table	per-coefficient step sizes; the only lossy step; Q scales the table
Zigzag	orders coefficients low $\rightarrow$ high frequency for long zero runs
DC DPCM	code $DC_i - DC_{i-1}$ (neighbors are similar)
AC RLE	(zeros-run, size) symbols; EOB ends the block
Huffman	short codes for frequent symbols; tables stored in DHT
Markers	FFD8...FFD9; headers ~ 600 B, scan $\sim 97\%$ of file
PNG filters	5 predictors/row, keep $\min \sum \text{res} $; then LZ77+Huffman (DEFLATE)
PNG vs JPEG	PNG for graphics/lossless (4.86:1 measured); JPEG for photos (31:1 at 30 dB)
WebP/AVIF	better prediction + arithmetic coding; $\sim 2\times$ / $\sim 4\times$ vs JPEG
Bit errors	one flip desyncs Huffman stream; restart markers contain damage

References

1. ITU-T T.81 / ISO/IEC 10918-1, *Information technology – Digital compression and coding of continuous-tone still images* (JPEG), 1992. The baseline DCT spec: quantization, Huffman

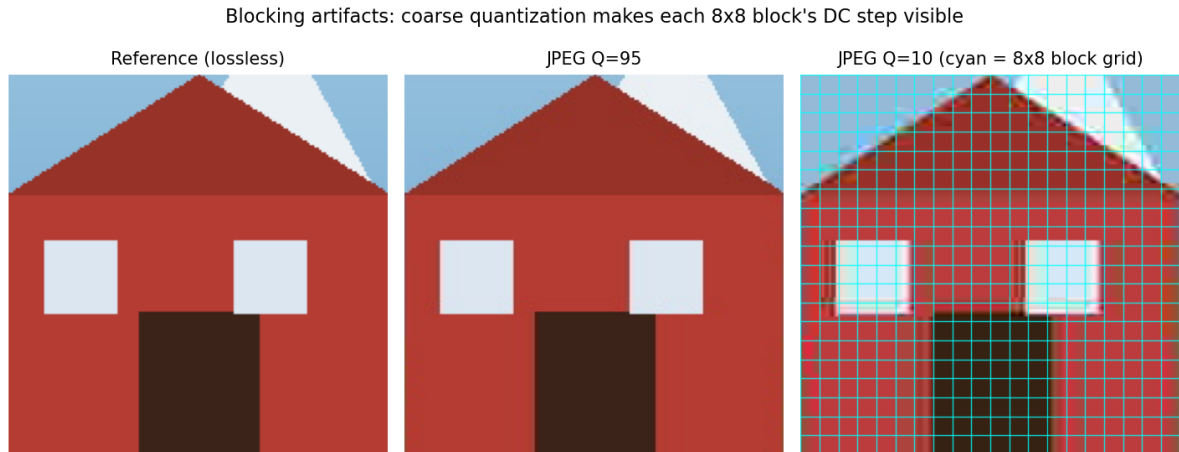


Figure 8: Measured crops (barn region). Q=95 is visually transparent; Q=10 shows blocking (each 8×8 block's coarse DC step becomes visible, cyan grid), ringing near edges, and posterized gradients.

tables, markers.

2. G. K. Wallace, “The JPEG still picture compression standard,” *IEEE Trans. Consumer Electronics*, 1991. The classic tutorial overview.
3. W3C, *Portable Network Graphics (PNG) Specification*, 2nd ed., 2003. Filtering (Ch. 6) and DEFLATE.
4. P. Deutsch, *DEFLATE Compressed Data Format Specification version 1.3*, RFC 1951, 1996. LZ77 + Huffman.
5. J. Alakuijala et al., WebP lossy bitstream specification, Google, 2010–. VP8 intra prediction, arithmetic entropy coding.
6. Alliance for Open Media, *AV1 Bitstream & Decoding Process Specification*, 2018; AVIF derived spec. The basis of AVIF's gains.
7. All measurements: Python 3, Pillow 10.2, numpy 1.26, on the synthetic 512×512 test image described in Section 7 (deterministic seed 7). Reproducible from the generation scripts.

Part VI — One Image, Every Number: A Complete Numerical Walkthrough

`image-walkthrough.pdf` (10 pages)

The concrete anchor for everything abstract so far. A single deterministic 16×16 image is carried through DCT, quantization, Huffman coding, and transmission — with every intermediate matrix printed — then three seeded bit flips corrupt it and you watch the desynchronization cascade destroy the reconstruction (PSNR 36.73 dB clean vs 12.22 dB corrupted). Read this after Parts II–V: it is the moment the quantization theory, the entropy coding, and the channel fragility all become one visible thing. The runnable script `walkthrough.py` regenerates every figure, so re-run it with your own bit-error patterns to build intuition for what your JSCC design must survive.

One Small Image, End to End

The Actual Numbers Behind Compression, Transmission, and Decoding

A worked companion to the joint source–channel coding guides

September 2026

Abstract

This document takes *one* deterministic 16×16 grayscale image and walks it through the entire pipeline with every number on the table: raw pixels, level shift, the 2-D DCT (implemented from the explicit orthogonal matrix), quantization with the standard JPEG luminance table at quality 50, zigzag scan, DC-differential and AC run-length symbols, Huffman coding with the standard tables (verified against a real encoder’s DHT segment), packetization, a seeded noisy channel that flips exactly 3 bits, and full decoding—clean and corrupted. Headline results: 256 raw bytes $\rightarrow$ 245 bits (31 bytes), 0.957 bits/pixel, 8.26:1 compression; clean decode PSNR 36.73 dB; 3 flipped bits $\rightarrow$ PSNR 12.22 dB. Every number was produced by running `walkthrough.py` (shipped alongside this document); the script’s printed output matches this text exactly.

Contents

1	The image	2
2	Encoding, block by block	2
2.1	Raw pixels	3
2.2	Level shift	3
2.3	The 2-D DCT	3
2.4	Quantization (JPEG luminance table, quality 50)	3
2.5	Zigzag scan	4
2.6	DC differential + AC run-length symbols	4
2.7	The bitstring	5
2.8	The other three blocks	5
3	Bitstream accounting	6
4	Transmission	6
4.1	Packets	6
4.2	The noisy channel	6
4.3	What the flips do	7
4.4	What real systems add	7
5	Decoding (clean)	8
6	Run it yourself	9
A	Original 16×16 pixel matrix	9
B	Reconstructed 16×16 matrix (clean decode)	10

1 The image

The test image is 16×16 grayscale, one byte per pixel, so 256 bytes raw. It is built from four 8×8 blocks, each chosen to stress a different part of the pipeline:

Block	Position	Content
0	top-left	Smooth gradient: $90 + 4i + 3j$ (values 90–139)
1	top-right	Sharp vertical edge: 50 for $j < 4$, 200 for $j \geq 4$
2	bottom-left	2×2 checkerboard texture: 80 / 170
3	bottom-right	Diagonal gradient: $120 + 2i + 2j$ (values 120–148)

Figure 1 shows all 256 pixel values. The full matrix is also tabulated in Appendix A.

Original image: all 256 pixel values

90	93	96	99	102	105	108	111	50	50	50	50	200	200	200	200
94	97	100	103	106	109	112	115	50	50	50	50	200	200	200	200
98	101	104	107	110	113	116	119	50	50	50	50	200	200	200	200
102	105	108	111	114	117	120	123	50	50	50	50	200	200	200	200
106	109	112	115	118	121	124	127	50	50	50	50	200	200	200	200
110	113	116	119	122	125	128	131	50	50	50	50	200	200	200	200
114	117	120	123	126	129	132	135	50	50	50	50	200	200	200	200
118	121	124	127	130	133	136	139	50	50	50	50	200	200	200	200
80	80	170	170	80	80	170	170	120	122	124	126	128	130	132	134
80	80	170	170	80	80	170	170	122	124	126	128	130	132	134	136
170	170	80	80	170	170	80	80	124	126	128	130	132	134	136	138
170	170	80	80	170	170	80	80	126	128	130	132	134	136	138	140
80	80	170	170	80	80	170	170	128	130	132	134	136	138	140	142
80	80	170	170	80	80	170	170	130	132	134	136	138	140	142	144
170	170	80	80	170	170	80	80	132	134	136	138	140	142	144	146
170	170	80	80	170	170	80	80	134	136	138	140	142	144	146	148

Figure 1: The original 16×16 image: every pixel value.

2 Encoding, block by block

We now encode block 0 with every intermediate matrix shown. Blocks 1–3 are summarized in Section 2.8.

2.1 Raw pixels

$$X_0 = \begin{matrix} & 90 & 93 & 96 & 99 & 102 & 105 & 108 & 111 \\ & 94 & 97 & 100 & 103 & 106 & 109 & 112 & 115 \\ & 98 & 101 & 104 & 107 & 110 & 113 & 116 & 119 \\ 102 & 105 & 108 & 111 & 114 & 117 & 120 & 123 \\ 106 & 109 & 112 & 115 & 118 & 121 & 124 & 127 \\ 110 & 113 & 116 & 119 & 122 & 125 & 128 & 131 \\ 114 & 117 & 120 & 123 & 126 & 129 & 132 & 135 \\ 118 & 121 & 124 & 127 & 130 & 133 & 136 & 139 \end{matrix} \quad (1)$$

2.2 Level shift

JPEG shifts unsigned pixels to a signed range before the transform: $X_s = X_0 - 128$:

$$X_s = \begin{matrix} -38 & -35 & -32 & -29 & -26 & -23 & -20 & -17 \\ -34 & -31 & -28 & -25 & -22 & -19 & -16 & -13 \\ -30 & -27 & -24 & -21 & -18 & -15 & -12 & -9 \\ -26 & -23 & -20 & -17 & -14 & -11 & -8 & -5 \\ -22 & -19 & -16 & -13 & -10 & -7 & -4 & -1 \\ -18 & -15 & -12 & -9 & -6 & -3 & 0 & 3 \\ -14 & -11 & -8 & -5 & -2 & 1 & 4 & 7 \\ -10 & -7 & -4 & -1 & 2 & 5 & 8 & 11 \end{matrix} \quad (2)$$

2.3 The 2-D DCT

With the orthogonal DCT matrix A ($A_{0n} = \sqrt{1/8}$, $A_{mn} = \sqrt{2/8} \cos(\pi(2n+1)m/16)$, $m \geq 1$; verified $\max |AA^T - I| = 1.40 \times 10^{-15}$),

$$Y = AX_sA^T = \begin{matrix} -108.0 & -54.7 & 0 & -5.7 & 0 & -1.7 & 0 & -0.4 \\ -72.9 & & 0 & 0 & & 0 & 0 & 0 \\ 0 & & 0 & 0 & & 0 & 0 & 0 \\ -7.6 & & 0 & 0 & & 0 & 0 & 0 \\ 0 & & 0 & 0 & & 0 & 0 & 0 \\ -2.3 & & 0 & 0 & & 0 & 0 & 0 \\ 0 & & 0 & 0 & & 0 & 0 & 0 \\ -0.6 & & 0 & 0 & & 0 & 0 & 0 \end{matrix} \quad (3)$$

(rounded to one decimal; exact zeros are shown as 0). The smooth gradient's energy sits almost entirely in the DC term (-108.0) and the first horizontal/vertical AC terms (-54.7 , -72.9)—this is energy compaction made visible. Figure 2 shows the same matrix as a symlog heatmap.

2.4 Quantization (JPEG luminance table, quality 50)

Divide by the standard table Q and round: $\hat{Y} = \text{round}(Y/Q)$.

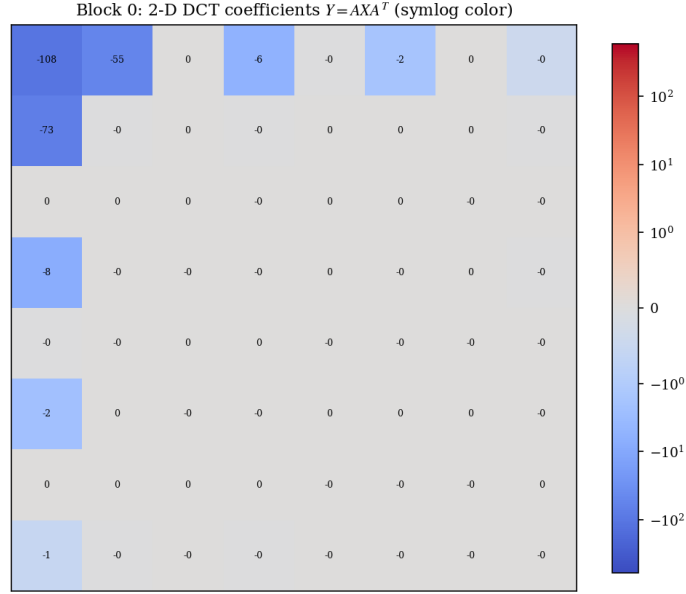


Figure 2: Block 0 DCT coefficients: nearly all energy in three low-frequency terms.

$$\begin{array}{cccccccc}
 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\
 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\
 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\
 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\
 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\
 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\
 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\
 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99
 \end{array}
 \hat{Y}_0 = \begin{array}{cccccccc}
 -7 & -5 & 0 & 0 & 0 & 0 & 0 & 0 \\
 -6 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array} \quad (4)$$

Check two entries: $-108.0/16 = -6.75 \rightarrow -7$; $-54.7/11 = -4.97 \rightarrow -5$. Only **4 of 64** coefficients survive—quantization is where the compression actually happens (Figure 3).

2.5 Zigzag scan

Reading $\hat{Y}_0$ in zigzag order (Figure 4) gives the 64-value sequence

$$-7, -5, -6, \underbrace{0, 0, 0, 0, 0, 0}_{6 \text{ zeros}}, -1, \underbrace{0, \dots, 0}_{54 \text{ zeros}}. \quad (5)$$

2.6 DC differential + AC run-length symbols

The DC term is differentially coded against the previous block's DC (0 here): $\text{diff} = -7 - 0 = -7$. Each value is described by its **category** = $\text{bit-length}(|v|)$, plus **amplitude bits** (v in binary if $v > 0$; $v + 2^{\text{cat}} - 1$ if $v < 0$):

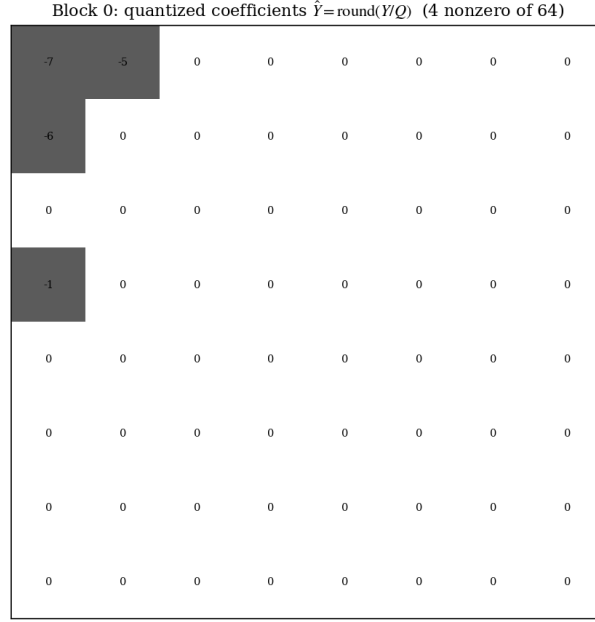


Figure 3: Block 0 quantized coefficients: 4 nonzero out of 64.

Symbol	Meaning	Category	Huffman code	Amplitude bits
DC	diff = -7	3	100	$-7 + 7 = 0 \rightarrow 000$
(0/3)	value -5 after 0 zeros	3	100	$-5 + 7 = 2 \rightarrow 010$
(0/3)	value -6 after 0 zeros	3	100	$-6 + 7 = 1 \rightarrow 001$
(6/1)	value -1 after 6 zeros	1	1111011	$-1 + 1 = 0 \rightarrow 0$
EOB	end of block	—	1010	—

2.7 The bitstring

Concatenating code + amplitude bits:

$$\underbrace{100\ 000}_{\text{DC}} \underbrace{100\ 010}_{-5} \underbrace{100\ 001}_{-6} \underbrace{1111011\ 0}_{-1} \underbrace{1010}_{\text{EOB}} \quad (6)$$

i.e. 100000100010100001111101101010 — **30 bits** for the whole 8×8 block (64 pixels $\rightarrow$ 0.47 bits/pixel). The Huffman tables are the standard JPEG Annex K tables, and the script verifies them code-for-code against the DHT segment of a real encoder's output.

2.8 The other three blocks

Block	Character	Quantized DC	Nonzero / 64	Bits
0	smooth gradient	-7	4	30
1	sharp edge	-1	5	78
2	checkerboard	-2	10	119
3	diagonal gradient	3	3	18

The pattern is exactly what theory predicts: smooth blocks quantize to almost nothing (18–30 bits); the sharp edge needs high-frequency coefficients (78 bits); the checkerboard—pure high frequency—is the most expensive (119 bits, 10 nonzero coefficients).

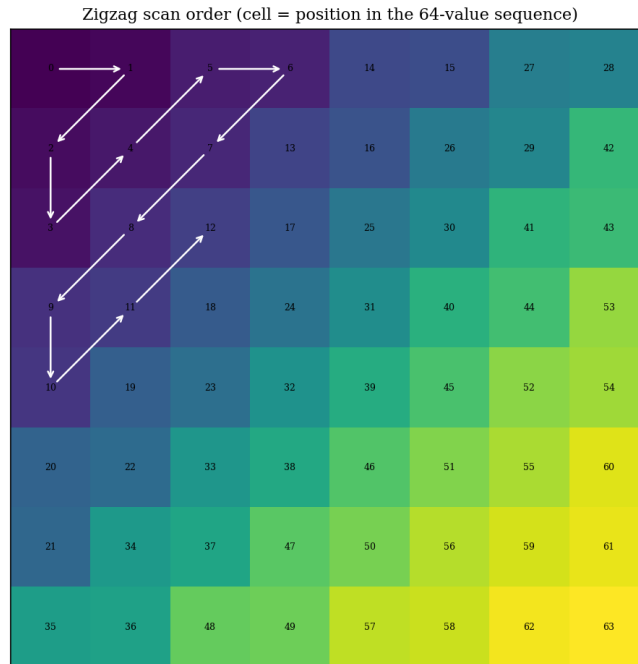


Figure 4: Zigzag scan order; the surviving coefficients sit at the start of the sequence.

3 Bitstream accounting

Quantity	Value
Total entropy-coded bits	245
Bytes ($\lceil 245/8 \rceil$)	31
Raw image	256 bytes
Bits per pixel	$245/256 = 0.957$
Compression ratio	$256/31 = 8.26:1$

(Header/marker overhead excluded—this is the entropy-coded scan data, the part the channel actually corrupts.)

4 Transmission

4.1 Packets

The 245-bit stream is split into 4 packets (62/61/61/61 bits). Each carries an 8-bit sequence number and an 8-bit length header (Figure 5).

4.2 The noisy channel

A seeded RNG (`random.Random(7)`) flips exactly 3 bits of the 245-bit stream:

Position	Packet	Block region	0/1 flip
38	0	block 1 (bits 30–107)	1 $\rightarrow$ 0
82	1	block 1	0 $\rightarrow$ 1
242	3	block 3 (bits 227–244)	0 $\rightarrow$ 1

Bitstream: 245 bits in 4 packets (headers = 8-bit seq + 8-bit length); red x = flipped bits

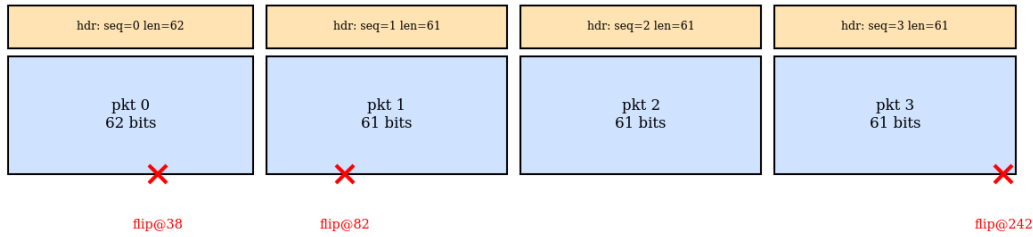


Figure 5: Packetization of the 245-bit stream; red crosses mark the 3 flipped bits.

4.3 What the flips do

Decoding the corrupted stream with the *same* Huffman parser gives these measured bit spans per block:

Block	Clean bits	Corrupted bits	Decoded DC (clean $\rightarrow$ corrupt)
0	0–29	0–29	$-7 \rightarrow -7$ (identical)
1	30–107	30–105	$-1 \rightarrow -1$, ACs differ
2	108–226	106–226	$-2 \rightarrow 3$
3	227–244	227–240	$3 \rightarrow 8$

Reading it:

- Block 0 (bits 0–29): untouched — decodes bit-identical.
- Block 1: the flips at bits 38 and 82 land inside codewords, so wrong symbols decode; the parse consumes 76 bits instead of 78. That 2-bit framing shift is the killer.
- Block 2: starts parsing at bit 106, two bits inside block 1’s true data. Every symbol mis-parses — DC decodes as 3 instead of -2 , AC coefficients are garbage, the checkerboard is destroyed. This is the cascade: one wrong codeword *length* destroys the framing of everything downstream.
- Block 3: the parse happens to land back on the true boundary (bit 227), but the flip at 242 corrupts its symbols (DC 8 instead of 3); the block is recognizable but shifted ≈ 10 too high, and 4 tail bits go unparsed.

This is the classic fragility of variable-length entropy coding: one wrong bit doesn’t corrupt one pixel, it corrupts the *framing* of everything after it. Corrupted decode: MSE 3896.01, PSNR **12.22 dB** — against 36.73 dB clean (Figure 6).

4.4 What real systems add

A toy stream has no defenses; real systems do (covered in depth in [image-pipeline.pdf](#)): **forward error correction** (redundant bits that fix flips), **retransmission** (TCP/ARQ re-sends lost packets), **restart markers** (periodic resynchronization points that stop the cascade—JPEG’s RSTn), and **error concealment** (the decoder guesses missing blocks from neighbors). The joint source–channel coding in [jscc-guide.pdf](#) takes the further step of making the *quantizer itself* expect the noise.

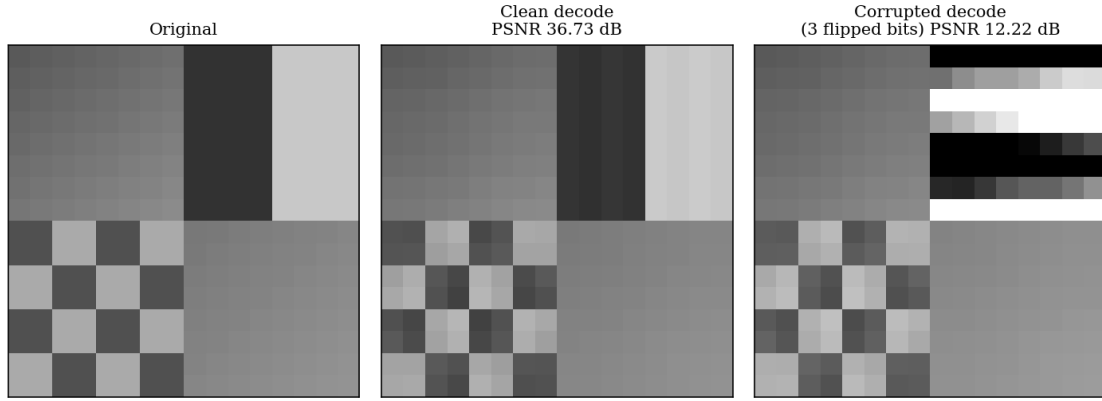


Figure 6: Original, clean decode, and corrupted decode (3 flipped bits).

5 Decoding (clean)

The decoder inverts every step: parse Huffman symbols $\rightarrow$ restore DC by cumulative sum $\rightarrow$ inverse zigzag $\rightarrow$ multiply by $Q \rightarrow$ IDCT ($X = A^T(Y \cdot Q)A$) $\rightarrow$ add 128 $\rightarrow$ round and clip to $[0, 255]$. Figure 7 shows the result beside the original; Figure 8 shows the error matrix (original $-$ reconstructed).

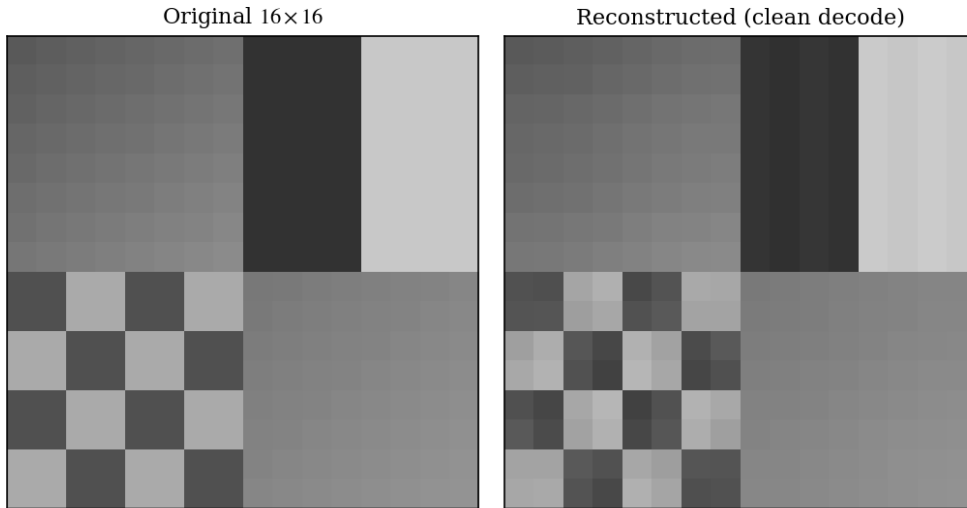


Figure 7: Original vs. clean reconstruction.

$$\text{MSE} = \frac{1}{256} \sum_{i,j} (x_{ij} - \hat{x}_{ij})^2 = 13.80, \quad \text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}} = \mathbf{36.73 \text{ dB}}. \quad (7)$$

The error concentrates where quantization killed real signal: the sharp edge (block 1, up to ± 4) and the checkerboard (block 2, up to ± 14), while the smooth gradients are nearly exact (max error 3 and 2). That is the rate–distortion tradeoff made concrete: 8.26:1 compression costs 36.73 dB, and the cost is paid in high frequencies.

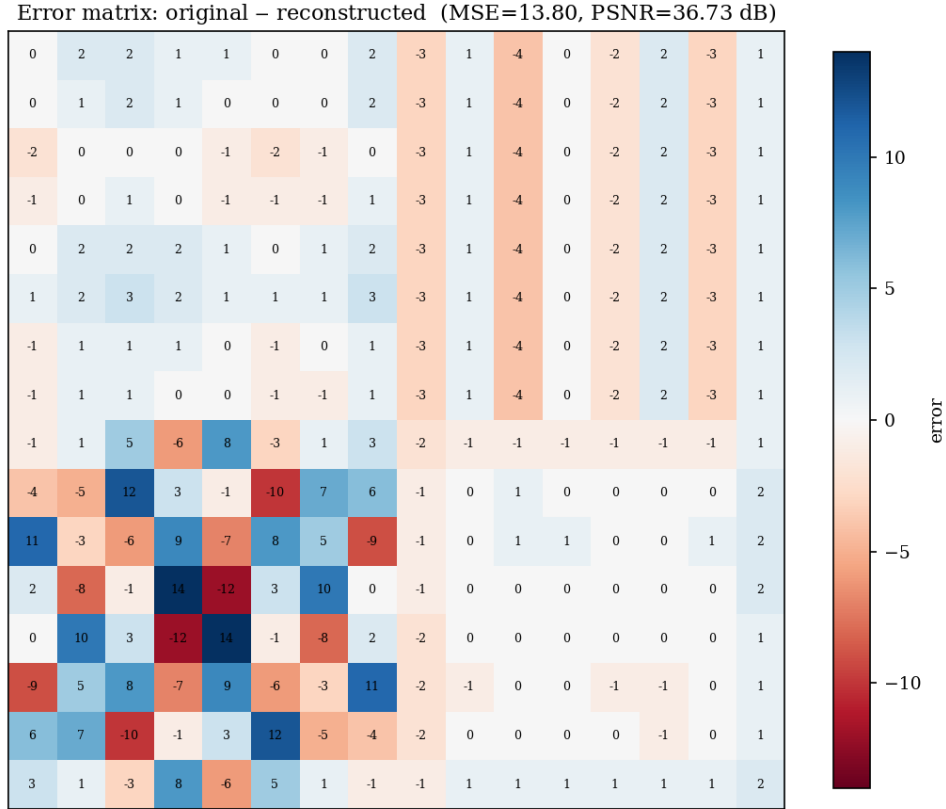


Figure 8: Error matrix (original – reconstructed).

6 Run it yourself

walkthrough.py (in ~/workspace/jscg-guide/) implements the entire pipeline and prints every intermediate matrix shown above, plus a JSON summary (walkthrough_summary.json) with the headline numbers. Run:

```
1 python3 ~/workspace/jscg-guide/walkthrough.py
```

It also regenerates all eight figures under figs/. The DCT uses the explicit orthogonal matrix (no library DCT call); the Huffman tables are the standard JPEG tables, verified code-for-code against the DHT segment emitted by a real encoder.

A Original 16 × 16 pixel matrix

90	93	96	99	102	105	108	111	50	50	50	50	200	200	200	200
94	97	100	103	106	109	112	115	50	50	50	50	200	200	200	200
98	101	104	107	110	113	116	119	50	50	50	50	200	200	200	200
102	105	108	111	114	117	120	123	50	50	50	50	200	200	200	200
106	109	112	115	118	121	124	127	50	50	50	50	200	200	200	200
110	113	116	119	122	125	128	131	50	50	50	50	200	200	200	200
114	117	120	123	126	129	132	135	50	50	50	50	200	200	200	200
118	121	124	127	130	133	136	139	50	50	50	50	200	200	200	200
80	80	170	170	80	80	170	170	120	122	124	126	128	130	132	134
80	80	170	170	80	80	170	170	122	124	126	128	130	132	134	136
170	170	80	80	170	170	80	80	124	126	128	130	132	134	136	138
170	170	80	80	170	170	80	80	126	128	130	132	134	136	138	140
80	80	170	170	80	80	170	170	128	130	132	134	136	138	140	142
80	80	170	170	80	80	170	170	130	132	134	136	138	140	142	144
170	170	80	80	170	170	80	80	132	134	136	138	140	142	144	146
170	170	80	80	170	170	80	80	134	136	138	140	142	144	146	148

B Reconstructed 16×16 matrix (clean decode)

90	91	94	98	101	105	108	109	53	49	54	50	202	198	203	199
94	96	98	102	106	109	112	113	53	49	54	50	202	198	203	199
100	101	104	107	111	115	117	119	53	49	54	50	202	198	203	199
103	105	107	111	115	118	121	122	53	49	54	50	202	198	203	199
106	107	110	113	117	121	123	125	53	49	54	50	202	198	203	199
109	111	113	117	121	124	127	128	53	49	54	50	202	198	203	199
115	116	119	122	126	130	132	134	53	49	54	50	202	198	203	199
119	120	123	127	130	134	137	138	53	49	54	50	202	198	203	199
81	79	165	176	72	83	169	167	122	123	125	127	129	131	133	133
84	85	158	167	81	90	163	164	123	124	125	128	130	132	134	134
159	173	86	71	177	162	75	89	125	126	127	129	132	134	135	136
168	178	81	66	182	167	70	80	127	128	130	132	134	136	138	138
80	70	167	182	66	81	178	168	130	130	132	134	136	138	140	141
89	75	162	177	71	86	173	159	132	133	134	136	139	141	142	143
164	163	90	81	167	158	85	84	134	134	136	138	140	143	144	145
167	169	83	72	176	165	79	81	135	135	137	139	141	143	145	146

Part VII — Deep Learning for Image Classification

`deep-learning-guide.pdf` (18 pages)

The final component of the project: automatically classifying the decoded images. This part develops deep learning from the ground up — the multivariate chain rule, maximum likelihood to cross-entropy, backpropagation with a fully worked numerical example, CNNs with a worked convolution, regularization, the classification pipeline with metrics, transfer learning — then implements it in PyTorch and applies it to the project’s question: does classification survive compression and channel noise? It builds directly on Part I’s ML chapter (§13) and connects back to Part II’s optional deep-JSCC extension. The key experimental result to internalize: *train the classifier on degraded images*, because matched train/test conditions recover most of the accuracy lost to compression artifacts. Note the errata on the SmallCNN input-size assumption.

Deep Learning for Image Classification

The Math, the Implementation, and the Build Plan

A learning companion to the joint source–channel coding guides

September 2026

Abstract

This is the seventh guide in the set, and the most hands-on: everything you need to *learn* deep learning for the image-classification component of the JSCC project. It develops the mathematics from the ground up (multivariate chain rule, maximum likelihood, back-propagation with a fully worked numerical example), explains convolutional networks layer by layer, covers regularization, metrics, and transfer learning, shows how it is implemented in PyTorch, and ends with a concrete technical build-out plan: the tools, the compute, and a phased path from “gradient by hand” to “classifier running on channel-degraded images.” Nothing here is trained or built for you — this is the learning portion, with diagrams and numbers at every step.

Contents

1	How to use this guide	3
1.1	Minimum math checklist	3
2	Math foundations as deep learning uses them	3
2.1	Vectors, matrices, tensors	3
2.2	The chain rule, multivariate form	3
2.3	Probability view: maximum likelihood to cross-entropy	4
3	The neuron and the multilayer perceptron	4
3.1	Forward-pass equations	4
3.2	Activation functions	5
4	Training: losses, gradient descent, backpropagation	5
4.1	Loss functions	5
4.2	Gradient descent, SGD, mini-batches	6
4.3	Backpropagation, derived in full	7
4.4	Worked numerical example (verified)	7
4.5	Optimizers: momentum and Adam	8
5	Convolutional neural networks	8
5.1	The convolution operation	8
5.2	Stride, padding, output size	9
5.3	Pooling	10
5.4	The classification CNN, end to end	10
5.5	Residual connections	10
5.6	Architecture tour, one paragraph each	10

6	Regularization and generalization	10
6.1	Overfitting and bias–variance	10
6.2	The toolkit	10
7	The classification pipeline end to end	11
7.1	Metrics, done right	12
7.2	Transfer learning	12
8	Implementation in PyTorch	13
8.1	Tensors and autograd	13
8.2	A small CNN as an <code>nn.Module</code>	13
8.3	Data, the training loop, evaluation	13
9	The project’s classification component	14
9.1	System diagram	14
9.2	Experimental design	15
9.3	Datasets	15
9.4	Generating degraded training data	15
10	Technical build-out plan	16
10.1	Tool verdicts	16
10.2	Compute: honest numbers	16
10.3	Phased plan	16
A	Notation glossary	17
B	Complete minimal CIFAR-10 training script (template)	17

1 How to use this guide

Each chapter follows the same pattern: **what it is** $\rightarrow$ **the math, fully shown** $\rightarrow$ **intuition** $\rightarrow$ **diagram or worked numbers**. Two rules of the road:

- **Read Chapters 2–4 in order.** Everything else builds on the chain rule and backpropagation. If backprop feels shaky, do not proceed to CNNs — go back and redo the worked example by hand.
- **Type the code, don’t just read it.** The PyTorch snippets in Chapter 8 are short enough to retype. The CIFAR-10 script in Appendix B is marked as an untested template: getting it to run is Phase 2 of the build plan.

1.1 Minimum math checklist

Before each chapter, you should be comfortable with:

Chapter	Prerequisite comfort
2 (foundations)	Vectors, matrices, matrix–vector products; basic probability (random variables, expectation)
3 (neurons)	Chapter 2; function composition
4 (training)	Chapter 3; partial derivatives; the chain rule from calculus
5 (CNNs)	Chapter 4; 2-D arrays; the output-size formula (derived inside)
6 (regularization)	Chapters 4–5
7 (pipeline/metrics)	Chapter 5; basic statistics (mean, fractions)
8 (PyTorch)	Chapters 4–7; basic Python
9 (project component)	Chapters 5–8; the channel models from the JSCC guides
10 (build plan)	Everything above, lightly

2 Math foundations as deep learning uses them

2.1 Vectors, matrices, tensors

Deep learning is linear algebra plus the chain rule, repeated at scale. The objects:

- **Vector** $\mathbf{x} \in \mathbb{R}^n$: an ordered list of n numbers. In ML it is a data point (e.g. 3 pixel values), a weight vector, or a gradient.
- **Matrix** $W \in \mathbb{R}^{m \times n}$: a linear map $\mathbb{R}^n \rightarrow \mathbb{R}^m$. A layer’s weights are a matrix; applying the layer is the matrix–vector product $W\mathbf{x}$.
- **Tensor**: the generic word for an N -dimensional array. A color image batch is a 4-D tensor: (batch, channel, height, width), e.g. (64, 3, 32, 32).

Two facts you will use constantly: $(AB)^t = B^t A^t$, and for a scalar function of a vector, the **gradient** $\nabla_{\mathbf{x}} f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)^t$ points in the direction of steepest increase of f .

2.2 The chain rule, multivariate form

If $z = f(\mathbf{y})$ and $\mathbf{y} = g(\mathbf{x})$, then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}, \quad \text{i.e.} \quad \nabla_{\mathbf{x}} z = J_g(\mathbf{x})^t \nabla_{\mathbf{y}} z, \quad (1)$$

where J_g is the **Jacobian** matrix $(J_g)_{ji} = \partial y_j / \partial x_i$. *Intuition:* to know how x_i affects z , add up its influence through every intermediate variable y_j it touches. Backpropagation is this formula applied mechanically, layer by layer, from the loss backwards.

Worked gradient. Let $f(\mathbf{x}) = (2x_1 + x_2)^2 + x_1x_2$ at $\mathbf{x} = (1, 3)$. Set $u = 2x_1 + x_2$ (so $u = 5$), $f = u^2 + x_1x_2$:

$$\begin{aligned}\frac{\partial f}{\partial x_1} &= \frac{\partial f}{\partial u} \frac{\partial u}{\partial x_1} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial x_1} = 2u \cdot 2 + x_2 = 2 \cdot 5 \cdot 2 + 3 = 23, \\ \frac{\partial f}{\partial x_2} &= \frac{\partial f}{\partial u} \frac{\partial u}{\partial x_2} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial x_2} = 2u \cdot 1 + x_1 = 10 + 1 = 11.\end{aligned}$$

So $\nabla f(1, 3) = (23, 11)$. Every backprop step you will ever do is this computation with more variables.

2.3 Probability view: maximum likelihood to cross-entropy

A classifier outputs a *probability distribution* over classes. Given data $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ and a model $p(y | \mathbf{x}; \boldsymbol{\theta})$, the **likelihood** is

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^N p(y_i | \mathbf{x}_i; \boldsymbol{\theta}), \quad (2)$$

and **maximum likelihood estimation (MLE)** picks $\boldsymbol{\theta}$ maximizing it — equivalently minimizing the **negative log-likelihood (NLL)**:

$$\text{NLL}(\boldsymbol{\theta}) = - \sum_{i=1}^N \log p(y_i | \mathbf{x}_i; \boldsymbol{\theta}). \quad (3)$$

For binary labels $y_i \in \{0, 1\}$ with predicted $\hat{y}_i = p(y = 1 | \mathbf{x}_i; \boldsymbol{\theta})$, this is exactly the **binary cross-entropy**:

$$L_{\text{BCE}} = - \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]. \quad (4)$$

What just happened: “cross-entropy loss” is not an arbitrary choice — it *is* maximum likelihood for probabilistic classifiers. Training a classifier = fitting a probability model by MLE.

The **KL divergence** $D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$ measures how far q is from p . Since $D_{\text{KL}}(p||q) = H(p, q) - H(p)$ where $H(p, q)$ is cross-entropy and $H(p)$ doesn’t depend on the model, minimizing cross-entropy $\equiv$ minimizing KL divergence to the true label distribution. For one-hot labels, $H(p) = 0$ and they coincide exactly.

3 The neuron and the multilayer perceptron

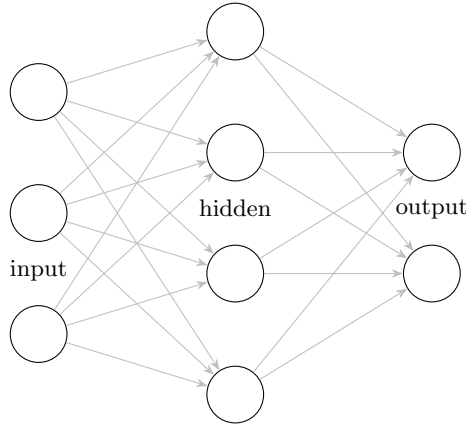
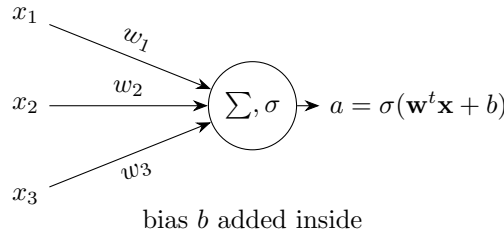
3.1 Forward-pass equations

A neuron computes a weighted sum, adds a bias, and applies a nonlinearity:

$$z = \mathbf{w}^t \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b, \quad a = \sigma(z). \quad (5)$$

A layer of m neurons is $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, $\mathbf{a} = \sigma(\mathbf{z})$ applied elementwise. An L -layer **MLP** composes these:

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad l = 1, \dots, L. \quad (6)$$



3.2 Activation functions

Function	Formula	Derivative
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma'(x) = \sigma(x)(1 - \sigma(x))$
tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - \tanh^2(x)$
ReLU	$\max(0, x)$	1 if $x > 0$ else 0
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \approx 0.01$ – 0.1	1 if $x > 0$ else α
Softmax	$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	Jacobian $\hat{y}_i(\delta_{ij} - \hat{y}_j)$ (rarely used directly)

Why ReLU won. Sigmoid and tanh **saturate**: for large $|x|$ their derivatives are ≈ 0 , so gradients vanish in deep stacks and early layers stop learning. ReLU’s derivative is exactly 1 whenever the unit is active — gradients flow undiminished — and it is one max operation to compute. The price (“dying ReLU”, units stuck at 0) is handled in practice by initialization and variants like leaky ReLU.

Why softmax pairs with cross-entropy. Softmax turns raw scores $\mathbf{z}$ into a probability distribution. With $L = -\sum_i y_i \log \hat{y}_i$ (one-hot $\mathbf{y}$), the gradient collapses beautifully:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i, \quad (7)$$

i.e. “prediction minus target.” The log undoes the exp, the normalization undoes itself in the derivative — the pair is designed to make learning clean. (Proof: differentiate $-\log(e^{z_k} / \sum_j e^{z_j})$ w.r.t. z_i ; the two terms combine to $\hat{y}_i - y_i$.)

4 Training: losses, gradient descent, backpropagation

4.1 Loss functions

- **Mean squared error:** $L_{\text{MSE}} = \frac{1}{N} \sum_i \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2$. The right loss for *regression* (predicting continuous values).

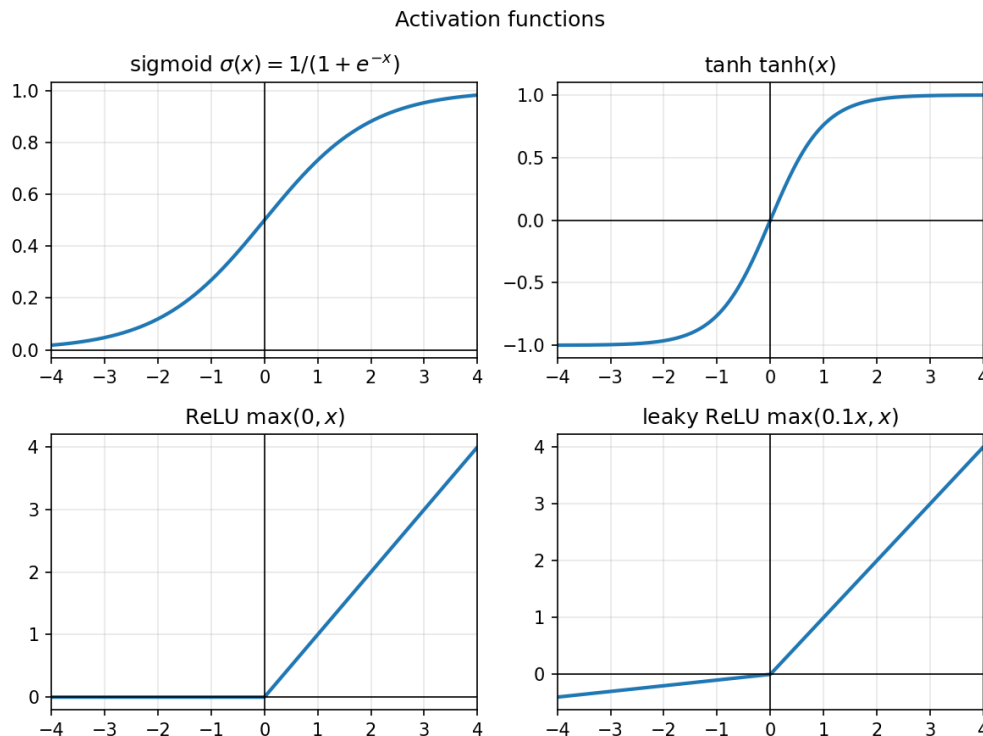


Figure 1: The four standard activations. Sigmoid/tanh saturate ($\sigma' \rightarrow 0$ for large $|x|$), which chokes gradients; ReLU's derivative is 1 for all active units, which is why it won.

- **Cross-entropy:** $L_{CE} = -\frac{1}{N} \sum_i \sum_c y_{ic} \log \hat{y}_{ic}$. The right loss for *classification*, because it is MLE (Chapter 2). Using MSE for classification works poorly: with sigmoid/softmax outputs the gradient vanishes when the model is confidently wrong, exactly when you need large updates.

4.2 Gradient descent, SGD, mini-batches

Training minimizes $L(\theta)$ over all parameters. The update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta), \quad (8)$$

with learning rate η . Computing ∇L over all N samples is expensive, so in practice:

- **SGD:** estimate the gradient from *one* random sample (or a few). Noisy but fast, and the noise helps escape bad local minima.
- **Mini-batch:** average over B samples ($B = 32$ – 256 typical):

$$\theta \leftarrow \theta - \eta \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\theta} L_i(\theta). \quad (9)$$

The standard compromise: vectorizes well on GPUs, gradient estimate stable enough.

Intuition: the gradient is a compass pointing uphill; you walk downhill in small steps. Too large an η and you bounce out of the valley (divergence); too small and training takes forever.

4.3 Backpropagation, derived in full

Consider a 2-layer net: $\mathbf{z}_1 = W_1 \mathbf{x} + \mathbf{b}_1$, $\mathbf{a}_1 = \sigma_1(\mathbf{z}_1)$, $z_2 = \mathbf{w}_2^t \mathbf{a}_1 + b_2$, $\hat{y} = \sigma_2(z_2)$, loss $L(\hat{y}, y)$. Define the **error signals** $\delta = \partial L / \partial \mathbf{z}$:

1. **Output layer.** Chain rule through loss and activation:

$$\delta_2 = \frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2}. \quad (10)$$

For BCE + sigmoid this collapses to $\delta_2 = \hat{y} - y$ (the $\hat{y}(1 - \hat{y})$ from σ' cancels the denominator from $\partial L / \partial \hat{y}$ — the pairing from Chapter 3 again).

2. **Output weights.** $z_2 = \mathbf{w}_2^t \mathbf{a}_1 + b_2$ gives

$$\frac{\partial L}{\partial \mathbf{w}_2} = \delta_2 \mathbf{a}_1, \quad \frac{\partial L}{\partial b_2} = \delta_2. \quad (11)$$

3. **Backpropagate to hidden layer.**

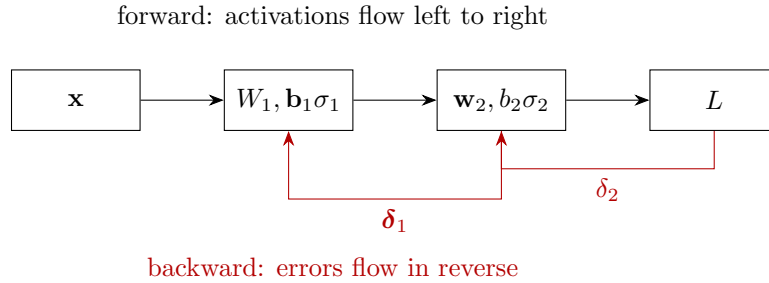
$$\delta_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (W_2^t \delta_2) \odot \sigma'_1(\mathbf{z}_1), \quad (12)$$

where $\odot$ is elementwise multiplication: the error flows back through the weights ($W_2^t \delta_2$) and then through the hidden activation's derivative.

4. **Hidden weights.**

$$\frac{\partial L}{\partial W_1} = \delta_1 \mathbf{x}^t, \quad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1. \quad (13)$$

That is the whole algorithm: forward pass caches $(\mathbf{z}_l, \mathbf{a}_l)$; backward pass computes δ_l from δ_{l+1} and reads off every gradient. Cost is the same order as the forward pass.



4.4 Worked numerical example (verified)

Network: 2 inputs $\rightarrow$ 2 hidden (ReLU) $\rightarrow$ 1 output (sigmoid), BCE loss.

$$\mathbf{x} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 0.4 & -0.2 \\ 0.7 & 0.1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}, \quad \mathbf{w}_2 = [0.5 \quad -0.6], \quad b_2 = 0.2, \quad y = 1.$$

Forward pass.

$$\mathbf{z}_1 = W_1 \mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 0.4(0.5) + (-0.2)(-0.3) + 0.1 \\ 0.7(0.5) + 0.1(-0.3) - 0.1 \end{bmatrix} = \begin{bmatrix} 0.36 \\ 0.22 \end{bmatrix},$$

$$\mathbf{a}_1 = \text{ReLU}(\mathbf{z}_1) = [0.36, 0.22]^t,$$

$$z_2 = 0.5(0.36) + (-0.6)(0.22) + 0.2 = 0.248,$$

$$\hat{y} = \sigma(0.248) = 0.561684,$$

$$L = -\log(0.561684) = 0.576816.$$

Backward pass.

$$\begin{aligned}
\delta_2 &= \hat{y} - y = -0.438316, \\
\frac{\partial L}{\partial \mathbf{w}_2} &= \delta_2 \mathbf{a}_1 = [-0.157794, -0.096429], & \frac{\partial L}{\partial b_2} &= -0.438316, \\
\frac{\partial L}{\partial \mathbf{a}_1} &= \mathbf{w}_2^t \delta_2 = [-0.219158, 0.262989], \\
\delta_1 &= \frac{\partial L}{\partial \mathbf{a}_1} \odot \text{ReLU}'(\mathbf{z}_1) = [-0.219158, 0.262989] \quad (\text{both } z_1 \text{ entries} > 0), \\
\frac{\partial L}{\partial \mathbf{W}_1} &= \delta_1 \mathbf{x}^t = \begin{bmatrix} -0.109579 & 0.065747 \\ 0.131495 & -0.078897 \end{bmatrix}, & \frac{\partial L}{\partial \mathbf{b}_1} &= [-0.219158, 0.262989].
\end{aligned}$$

One gradient step ($\eta = 0.5$): $\mathbf{W}_1 \leftarrow \begin{bmatrix} 0.454789 & -0.232874 \\ 0.634253 & 0.139448 \end{bmatrix}$, $\mathbf{b}_1 \leftarrow [0.209579, -0.231495]^t$, $\mathbf{w}_2 \leftarrow [0.578897, -0.551785]$, $b_2 \leftarrow 0.419158$, and the loss falls $0.576816 \rightarrow 0.407051$. Every number above was reproduced by an independent NumPy script (`dl_backprop_check.py`); analytic gradients match central finite differences to 8.2×10^{-10} .

Intuition check: $\delta_2 < 0$ says “ $\hat{y}$ is too low, push z_2 up.” That push propagates back: $w_{2,1}$ grows (its hidden unit was active and positive), $w_{2,2}$ shrinks toward zero (its unit was pulling the wrong way). Backprop is just blame assignment, quantified.

4.5 Optimizers: momentum and Adam

Plain SGD uses one global η . Two upgrades dominate practice:

Momentum. Keep a velocity $\mathbf{v}$:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \nabla_{\boldsymbol{\theta}} L, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v}, \quad \beta \approx 0.9. \quad (14)$$

What it does: averages recent gradients, so consistent directions accelerate (ravines) while oscillation cancels. Physically a heavy ball rolling downhill.

Adam. Per-parameter adaptive rates from gradient moments:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (15)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (16)$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t), \quad (17)$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \quad (18)$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. *What each term does:* $\mathbf{m}_t$ is momentum (direction); $\mathbf{v}_t$ tracks per-parameter gradient variance, so parameters with huge gradients get smaller effective steps and quiet ones get larger; the hats correct the zero-initialization bias early in training. Adam is the default starting optimizer for most new projects.

5 Convolutional neural networks

5.1 The convolution operation

An MLP treats every pixel as an independent input, which is wasteful: a 256×256 image has 65,536 inputs, and pixels next to each other are strongly correlated. A **convolutional layer** instead slides a small kernel over the image, reusing the same weights at every position. (Deep-learning libraries implement *cross-correlation*, kernel not flipped; everyone still calls it convolution.)

For input I and kernel K of size $k \times k$:

$$O[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I[i+u, j+v] K[u, v] + b. \quad (19)$$

Each output is a weighted sum of a local patch — the network learns *local patterns* (edges, textures) and detects them anywhere in the image (**translation equivariance**). With C_{in} input channels and C_{out} kernels, the layer has $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ weights, independent of image size.

Worked example. 5×5 input, 3×3 kernel:

$$I = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 4 & 0 & 1 & 2 & 1 \\ 2 & 1 & 3 & 0 & 2 \\ 0 & 2 & 1 & 1 & 0 \\ 1 & 0 & 2 & 3 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}.$$

Top-left output ($i = j = 0$): align K over the top-left patch,

$$O[0, 0] = 1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) + 4 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) + 1 \cdot 0 + 3 \cdot 1 = 1 + 0 + 0 + 0 + 0 + 0 - 2 + 0 + 3 = 2.$$

Sliding over all 9 positions (valid mode):

$$O = \begin{bmatrix} 2 & 1 & -2 \\ 5 & 0 & -1 \\ 2 & 5 & 1 \end{bmatrix}.$$

(Verified by direct summation in the companion script.)

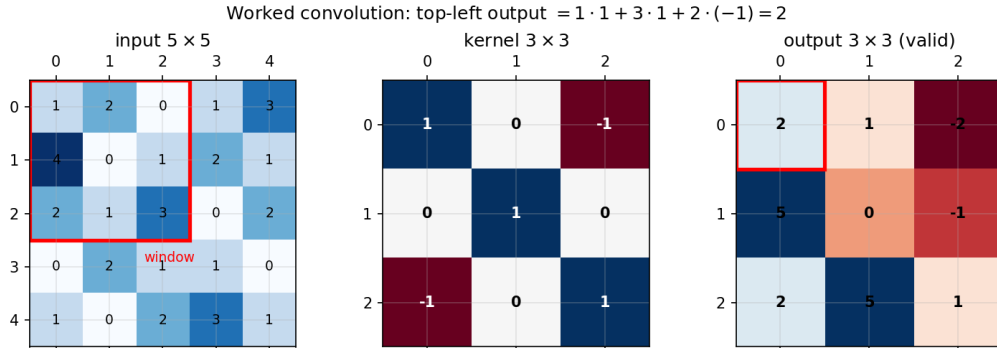


Figure 2: The worked convolution: the red window is the patch producing the red output element. Note this kernel responds to diagonal structure — learned kernels do the same for edges, corners, textures.

5.2 Stride, padding, output size

With input size n , kernel k , padding p , stride s :

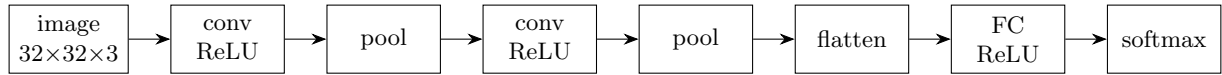
$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \quad (20)$$

Reading it: $n + 2p - k$ is how far the kernel's top-left corner can travel; dividing by s counts steps; $+1$ counts the starting position. Two standard choices: **valid** ($p = 0$, output shrinks) and **same** (p chosen so $n_{\text{out}} = n$ at $s = 1$, i.e. $p = (k - 1)/2$ for odd k). Stride $s = 2$ halves the spatial size — the cheap way to downsample.

5.3 Pooling

Max-pooling takes the max over each $p \times p$ window (usually 2×2 , stride 2): it downsamples by 2 while keeping the strongest activation in each region, adding local translation invariance. No learnable parameters. (Average-pooling exists; max-pooling dominates in classifiers.)

5.4 The classification CNN, end to end



The pattern: early layers learn edges/textures at high resolution; pooling shrinks the map while channels grow (e.g. $32 \times 32 \times 16 \rightarrow 16 \times 16 \times 32 \rightarrow 8 \times 8 \times 64$); flatten turns the last feature map into a vector; fully-connected layers combine global evidence; softmax outputs class probabilities. The **receptive field** of a deep unit — the input region that can influence it — grows with depth, which is why late layers “see” whole objects.

5.5 Residual connections

Very deep plain CNNs suffer **vanishing gradients**: products of many small derivatives kill the signal to early layers. A residual block rewrites the layer as

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}, \quad (21)$$

learning only the *residual* $\mathcal{F}(\mathbf{x})$. *Why it fixes training*: the gradient has a direct path, $\partial \mathbf{y} / \partial \mathbf{x} = \partial \mathcal{F} / \partial \mathbf{x} + I$; even if $\partial \mathcal{F} / \partial \mathbf{x}$ vanishes, the identity term carries the gradient through untouched. It is also trivial for the net to learn the identity (set $\mathcal{F} = 0$), so adding layers can never hurt in principle.

5.6 Architecture tour, one paragraph each

LeNet-5 (1998): conv→pool→conv→pool→FC — the template everything follows; digit recognition. **AlexNet** (2012): the same idea scaled up with ReLUs, dropout, and GPUs — won ImageNet and started the deep-learning era. **VGG** (2014): showed depth itself wins; stacks of tiny 3×3 convolutions instead of large kernels. **ResNet** (2015): residual connections enabled 100+ layer networks; the backbone still used for transfer learning today.

6 Regularization and generalization

6.1 Overfitting and bias–variance

A model with far more parameters than training samples can *memorize* the training set: training loss $\rightarrow 0$ while test performance is poor. This is **overfitting**. The **bias–variance** lens: expected test error decomposes into bias² (error from wrong assumptions — too-simple model) + variance (error from sensitivity to the particular training sample — too-flexible model) + irreducible noise. Regularization methods below all trade a little bias for a lot less variance.

6.2 The toolkit

L2 (weight decay). Add $\frac{\lambda}{2} \|\mathbf{w}\|^2$ to the loss; the gradient gains $+\lambda \mathbf{w}$, so weights decay toward zero unless the data insists. Penalizes large weights $\rightarrow$ smoother functions. **L1** adds $\lambda \|\mathbf{w}\|_1$ instead, which drives weights *exactly* to zero (sparsity).

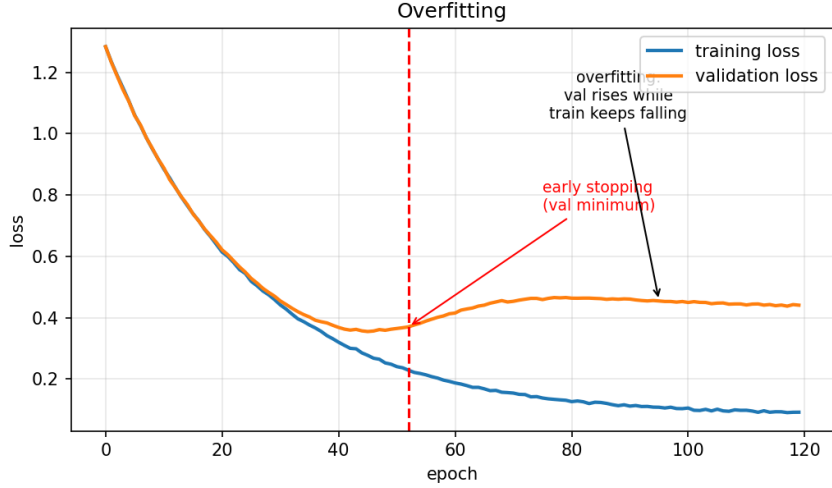


Figure 3: The signature of overfitting: training loss keeps falling while validation loss bottoms out and rises. Stop training at the validation minimum (**early stopping**).

Dropout. During training, zero each activation independently with probability p (typical $p = 0.2-0.5$), and scale survivors by $1/(1-p)$ (**inverted dropout**) so the expected sum is unchanged:

$$\tilde{a}_i = \begin{cases} a_i/(1-p) & \text{with prob. } 1-p, \\ 0 & \text{with prob. } p, \end{cases} \quad \mathbb{E}[\tilde{a}_i] = a_i. \quad (22)$$

At test time dropout is off. *Intuition:* the network can't rely on any single unit, so it learns redundant representations — like training an ensemble of thinned networks.

Batch normalization. For each mini-batch, normalize then re-scale with learned parameters:

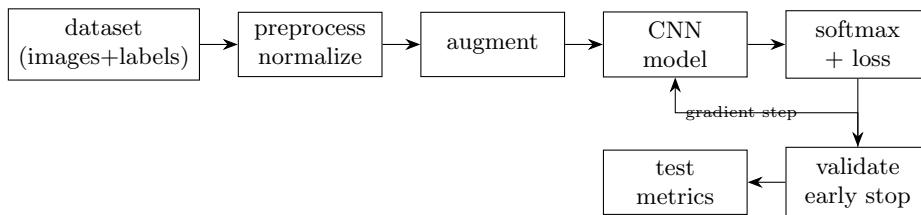
$$\mu_B = \frac{1}{m} \sum x_i, \quad \sigma_B^2 = \frac{1}{m} \sum (x_i - \mu_B)^2, \quad \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta. \quad (23)$$

Why it helps: keeps layer inputs in the activation's healthy range, allows larger learning rates, and adds slight noise (batch statistics vary) that regularizes. γ, β let the network undo the normalization if it wants to.

Data augmentation. Train on randomly transformed copies (crops, flips, rotations, color jitter): artificially multiplies the dataset and teaches invariance. For images this is the single highest value-per-effort regularizer.

Early stopping. Track validation loss; keep the weights from its minimum (Figure 3). Free, and almost always used.

7 The classification pipeline end to end



Preprocessing: scale pixels to $[0, 1]$ (divide by 255), then standardize per channel with dataset mean/std ($x \leftarrow (x - \mu)/\sigma$) so inputs sit in the network’s comfortable range. Compute μ, σ on the *training* set only; reuse for val/test.

7.1 Metrics, done right

Raw accuracy lies when classes are imbalanced (99% “no fire” accuracy with zero fires detected). Use the **confusion matrix**.

Worked 3-class example. 300 test images (fire / smoke / no-fire), rows = predicted, columns = actual:

	actual fire	actual smoke	actual no-fire
pred. fire	82	9	4
pred. smoke	6	70	10
pred. no-fire	2	11	106

Accuracy = $(82 + 70 + 106)/300 = 86.0\%$. Per class ($P = TP/(TP + FP)$, $R = TP/(TP + FN)$, $F_1 = 2PR/(P + R)$):

Class	TP	Precision	Recall	F_1
fire	82	$82/95 = 0.863$	$82/90 = 0.911$	0.887
smoke	70	$70/86 = 0.814$	$70/90 = 0.778$	0.796
no-fire	106	$106/119 = 0.891$	$106/120 = 0.883$	0.887

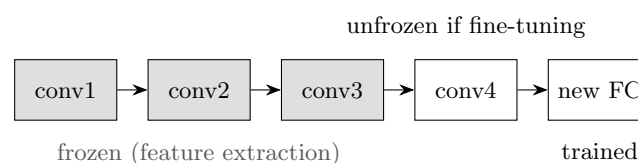
Macro- F_1 = $(0.887 + 0.796 + 0.887)/3 = 0.857$ averages the per-class scores, so the weak smoke class can’t hide. For fire detection, *recall on fire* (fraction of real fires caught) is the safety-critical number; precision (fraction of alarms that are real) controls false-alarm fatigue. **ROC/AUC** plots true-positive rate vs. false-positive rate as the decision threshold sweeps, and AUC summarizes it in one number — useful when you will tune the alarm threshold later.

7.2 Transfer learning

Training a deep CNN from scratch needs millions of labeled images. **Transfer learning** reuses a network pretrained on ImageNet (1.2M images, 1000 classes):

- **Feature extraction:** freeze all pretrained layers, train only a new final classifier on your data. Fast; works when your dataset is small.
- **Fine-tuning:** unfreeze later layers too and train everything with a small learning rate. Better when you have more data or the domain differs (aerial fire imagery $\neq$ ImageNet photos).

Why it works: early layers learn generic features (edges, textures, color blobs) useful for *any* vision task; only late layers are ImageNet-specific. For a small fire dataset, fine-tuning a ResNet-18 beats training from scratch by a wide margin.



8 Implementation in PyTorch

PyTorch is the standard for research and for this project’s classification component (the PyTorch-vs-TensorFlow verdict is in Chapter 10). The mental model: **tensors** are NumPy arrays that can live on GPU and record operations for **autograd**.

8.1 Tensors and autograd

```
1 import torch
2
3 x = torch.tensor([0.5, -0.3], requires_grad=True)
4 W = torch.tensor([[0.4, -0.2], [0.7, 0.1]], requires_grad=True)
5 b = torch.tensor([0.1, -0.1], requires_grad=True)
6
7 z = W @ x + b          # every op is recorded on a graph
8 L = (z ** 2).sum()      # some scalar loss
9 L.backward()           # chain rule, done for you
10 print(W.grad)          # dL/dW -- compare with Chapter 4 by hand
```

Listing 1: Autograd: gradients without hand-derived backprop.

What’s happening: `requires_grad=True` tells PyTorch to track operations; `.backward()` walks the graph applying the chain rule (exactly Chapter 4’s δ ’s); `.grad` holds the result. You will never hand-derive gradients in the project — but Chapter 4 is why you trust these numbers.

8.2 A small CNN as an `nn.Module`

```
1 import torch.nn as nn
2
3 class SmallCNN(nn.Module):
4     def __init__(self, n_classes=2):
5         super().__init__()
6         self.features = nn.Sequential(
7             nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
8             # 32->16
9             nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
10            # 16->8
11        )
12        self.classifier = nn.Sequential(
13            nn.Flatten(),
14            nn.Linear(32*8*8, 128), nn.ReLU(), nn.Dropout(0.3),
15            nn.Linear(128, n_classes),
16        )
17    def forward(self, x):
18        return self.classifier(self.features(x)) # raw scores (logits)
```

Listing 2: A minimal classifier CNN.

Notes: `Conv2d(3,16,3)` = 3 input channels, 16 kernels, 3×3 ; the model outputs *logits* — apply `nn.CrossEntropyLoss`, which bundles softmax + NLL (never apply softmax yourself before it).

8.3 Data, the training loop, evaluation

```
1 from torchvision import datasets, transforms
2
```

```

3 train_tf = transforms.Compose([
4     transforms.RandomResizedCrop(224),
5     transforms.RandomHorizontalFlip(),
6     transforms.ToTensor(),                      # [0,1]
7     transforms.Normalize(mean=[0.485,0.456,0.406],# ImageNet stats
8                             std=[0.229,0.224,0.225])),
9 ])
10 train_dl = torch.utils.data.DataLoader(
11     datasets.ImageFolder("data/train", transform=train_tf),
12     batch_size=32, shuffle=True, num_workers=4)

```

Listing 3: DataLoader with preprocessing + augmentation.

```

1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 model = SmallCNN(n_classes=2).to(device)
3 opt = torch.optim.Adam(model.parameters(), lr=1e-3)
4 loss_fn = nn.CrossEntropyLoss()
5
6 for epoch in range(20):
7     model.train()
8     for xb, yb in train_dl:
9         xb, yb = xb.to(device), yb.to(device)
10        opt.zero_grad()          # clear last step's gradients
11        loss = loss_fn(model(xb), yb)
12        loss.backward()          # backprop (Chapter 4, automated)
13        opt.step()              # Adam update

```

Listing 4: The canonical training loop.

```

1 model.eval()
2 correct = total = 0
3 with torch.no_grad():          # disables autograd: faster, less
4     memory                     # memory
5     for xb, yb in test_dl:
6         pred = model(xb.to(device)).argmax(dim=1)
7         correct += (pred == yb.to(device)).sum().item()
8         total += len(yb)
9 print(f"accuracy: {correct/total:.3f}")

```

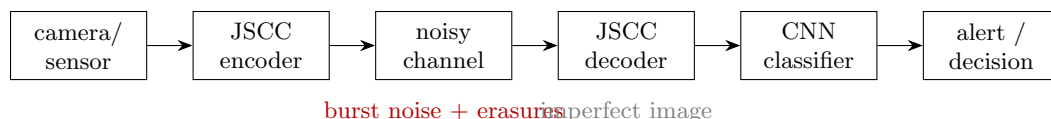
Listing 5: Evaluation: no gradients, collect predictions.

Saving/loading: `torch.save(model.state_dict(), "cnn.pt")` stores the weights; recreate the model class and `load_state_dict` to restore. Save the *best validation* checkpoint during training, not the last epoch (early stopping, Chapter 6).

GPU: `.to(device)` moves model and batches; without a GPU everything still runs on CPU, just slower (Chapter 10 has the numbers). Mixed precision (`torch.cuda.amp`) halves memory and speeds up training on modern GPUs — a one-line-ish change worth knowing exists.

9 The project's classification component

9.1 System diagram



The classifier never sees the clean image — it sees the JSCC decoder’s *reconstruction*, complete with quantization and channel artifacts. The research question: **does classification survive compression + noise, and how far can you push the channel before it doesn’t?**

9.2 Experimental design

1. **Baseline:** train a classifier on *clean* images; measure accuracy on clean test images. This is the ceiling.
2. **Degraded test:** run the test set through your JSCC pipeline at several channel qualities (BER / erasure rate sweep); measure accuracy on the *reconstructions*. Plot **accuracy vs. channel quality** — expect graceful decline, not a cliff (the COVQ philosophy from the JSCC guides).
3. **Matched training:** retrain (or fine-tune) the classifier on *degraded* training images at the target channel quality. Chapter 7’s rule — match train and test distributions — predicts most of the lost accuracy comes back, because the network learns to ignore quantization/blocking artifacts.
4. **Ablations:** clean-train/degraded-test vs. degraded-train/degraded-test vs. clean-train/clean-test; which artifacts hurt most (try: quantization-only, errors-only, erasures-only degradations).

Report confusion matrices and per-class recall (for fire: recall on the fire class is the safety metric), not just accuracy.

9.3 Datasets

- **CIFAR-10** for method debugging: 60,000 tiny 32×32 color images, 10 classes. Too small for real fire work, but the standard sandbox for getting the pipeline right fast.
- **FLAME** (verified: Shamsoshoara et al., arXiv:2012.14036, “Fire Luminosity Airborne-based Machine learning Evaluation”): drone imagery of prescribed pile burns in Northern Arizona; **47,992 images labeled for classification at 254×254** , plus thermal palettes and segmentation masks. This is the natural dataset for the project’s fire-detection application.
- Literature-reported alternatives (verify before committing): **FireNet**, the **Corsican Fire Database**, and **DeepFire** — all used in published fire-classification work; sizes and access conditions vary.

9.4 Generating degraded training data

You already own the degradation pipeline: the JSCC encoder/decoder from the project (or the walkthrough script’s JPEG-style chain as a stand-in). Procedure: take each training image → encode → simulate the noisy channel at the target operating point → decode → save the reconstruction with the *original label*. Labels survive because degradation doesn’t change what’s in the picture. Generate one degraded copy per channel operating point you want to study; keep the clean originals too, for the baseline.

10 Technical build-out plan

10.1 Tool verdicts

Tool	Role	Verdict
Python 3.10+	Everything glues through it	The working language; all ML tooling targets it
PyTorch (+torchvision)	Model building, training	Use this. Research standard; dynamic graphs; best tutorial/debug ecosystem. The classification component lives here.
TensorFlow/Keras	Alternative framework	Skip unless your lab requires it; no advantage for this project
NumPy / Matplotlib	Numerics, plots	Required; the backprop-from-scratch phase is pure NumPy
Jupyter	Experimentation	Notebooks for exploration; convert working code to scripts for reproducibility
TensorBoard / W&B	Experiment tracking	Log loss curves, learning rates, checkpoints; W&B is nicer, TensorBoard is free and local
Git	Version control	Non-negotiable; commit code, configs, and the scripts that generate figures

10.2 Compute: honest numbers

Training cost scales with (dataset size $\times$ model size $\times$ epochs). Ballparks, single GPU unless noted:

- Small CNN (like Chapter 8's) on CIFAR-10, 20 epochs: **$\sim 5\text{--}10$ minutes** on a Colab T4 GPU; **$\sim 45\text{--}90$ minutes** on a modern laptop CPU. Phase 2 is doable on a laptop; a GPU just makes iteration pleasant.
- Fine-tuning ResNet-18 on $\sim 5,000$ fire images, 10 epochs: **a few minutes** on any GPU from the last 5 years.
- Training ResNet-50 from scratch on ImageNet: days on 8 GPUs — *this is why transfer learning exists* (Chapter 7).

Free GPUs: Google Colab and Kaggle both offer free GPU notebooks (time-limited sessions) — enough for every phase below. For longer runs, a university cluster or a single rented GPU suffices; you do not need to buy hardware for this project.

Dataset storage is trivial (CIFAR-10 ≈ 170 MB; FLAME images at 254×254 are a few GB) — any laptop disk works.

10.3 Phased plan

1. **Phase 1 — backprop from scratch (NumPy).** Reimplement Chapter 4's worked example as a tiny MLP trained on a toy dataset (e.g. XOR or 2-D blobs). Goal: *understanding*, not performance. ~ 1 week.
2. **Phase 2 — PyTorch CNN on CIFAR-10.** Get Appendix B's script running; reach $\sim 70\text{--}75\%$ accuracy with the small CNN; add augmentation and watch it climb. Goal: the full pipeline (data $\rightarrow$ train $\rightarrow$ evaluate) working. $\sim 1\text{--}2$ weeks.

3. **Phase 3 — transfer learning.** Fine-tune a pretrained ResNet-18 (torchvision, one line to load) on a fire dataset; compare frozen vs. fine-tuned. Goal: $\geq 90\%$ + accuracy on clean fire images. $\sim 1\text{--}2$ weeks.
4. **Phase 4 — robustness study (the project contribution).** Generate degraded datasets with the JSCC pipeline (Chapter 9); run the clean-vs-degraded experimental design; produce accuracy-vs-channel-quality curves and confusion matrices. This is the classification chapter of the project writeup. $\sim 2\text{--}3$ weeks.

Total: roughly 5–8 weeks of the ML component alongside the JSCC work, with each phase producing a working artifact before the next begins.

A Notation glossary

Symbol	Meaning
$\mathbf{x}, \mathbf{w}, \boldsymbol{\theta}$	Input vector, weight vector, parameter vector
$W^{(l)}, \mathbf{b}^{(l)}$	Weight matrix / bias of layer l
$\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$	Pre-activation / activation of layer l
σ, ReLU	Activation function
$\hat{\mathbf{y}}, \mathbf{y}$	Predicted / true labels (one-hot for classification)
L, ℓ	Loss (total / per-sample)
η	Learning rate
$\nabla_{\boldsymbol{\theta}} L$	Gradient of loss w.r.t. parameters
$\boldsymbol{\delta}^{(l)}$	Backprop error signal $\partial L / \partial \mathbf{z}^{(l)}$
$D_{\text{KL}}(p \ q)$	KL divergence
K (conv)	Convolution kernel; s, p stride/padding
γ, β	Batch-norm scale/shift; also Adam's are β_1, β_2
m_t, v_t	Adam's first/second moment estimates
TP/FP/TN/FN	True/false positives/negatives

B Complete minimal CIFAR-10 training script (template)

```

1 import torch, torch.nn as nn
2 from torchvision import datasets, transforms, models
3
4 # --- data ---
5 tf = transforms.Compose([transforms.RandomCrop(32, padding=4),
6                          transforms.RandomHorizontalFlip(),
7                          transforms.ToTensor(),
8                          transforms.Normalize((0.4914, 0.4822, 0.4465),
9                                              (0.2470, 0.2435, 0.2616))])
10 train_dl = torch.utils.data.DataLoader(
11     datasets.CIFAR10("data", train=True, download=True, transform=tf),
12     batch_size=128, shuffle=True, num_workers=4)
13 test_dl = torch.utils.data.DataLoader(
14     datasets.CIFAR10("data", train=False, download=True,
15                      transform=transforms.Compose([
16                          transforms.ToTensor(),
17                          transforms.Normalize((0.4914, 0.4822, 0.4465),
18                                              (0.2470, 0.2435, 0.2616))])),
19     batch_size=256, num_workers=4)

```

```

20
21 # --- model: small CNN, or swap in a pretrained ResNet for Phase 3 ---
22 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
23 model = models.resnet18(weights="IMAGENET1K_V1") # transfer learning
24 model.fc = nn.Linear(model.fc.in_features, 10) # new head for CIFAR
    -10
25 model = model.to(device)
26
27 opt = torch.optim.Adam(model.parameters(), lr=1e-3)
28 sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=20)
29 loss_fn = nn.CrossEntropyLoss()
30 best = 0.0
31
32 for epoch in range(20):
33     model.train()
34     for xb, yb in train_dl:
35         xb, yb = xb.to(device), yb.to(device)
36         opt.zero_grad()
37         loss = loss_fn(model(xb), yb)
38         loss.backward(); opt.step()
39     sched.step()
40     # validate
41     model.eval(); correct = total = 0
42     with torch.no_grad():
43         for xb, yb in test_dl:
44             pred = model(xb.to(device)).argmax(1)
45             correct += (pred == yb.to(device)).sum().item(); total +=
                len(yb)
46     acc = correct/total
47     print(f"epoch_{epoch}: test_acc_{acc:.3f}")
48     if acc > best: # keep best
49         checkpoint
        best = acc; torch.save(model.state_dict(), "best_cnn.pt")
50 print("best:", best)

```

Listing 6: Untested learning template — getting this running is Phase 2.

Learn the math by hand once; then let autograd do it forever.

Part VIII — Supplement: Four Gap-Fill Chapters

New material written for this volume

The audit found four topics the project needs that none of the seven guides covers: how bits become waveforms and return as soft decisions (the physical justification for the burst models of Part II and the soft decoders of Part IV), how to fit those burst models to measured channel data, the statistics needed to run honest experiments, and the deployment tradeoffs for the sensor networks in the project description. Each chapter shows the math, not just the conclusion.

1 Supplement S1: From bits to waveforms and back

Every discrete channel model in Part II is an abstraction of a physical process: bits are mapped to waveforms, the waveforms pass through a noisy, fading medium, and the receiver converts the received signal back into (bit, confidence) pairs. Understanding this round trip explains where bursts come from and why modern decoders want *soft* information.

Modulation. The simplest map is BPSK: bit 0 $\mapsto x = +1$, bit 1 $\mapsto x = -1$ (or vice versa), transmitted as x times a carrier pulse. QPSK packs two bits per symbol using four constellation points $(\pm 1 \pm j)/\sqrt{2}$, with Gray mapping so adjacent points differ in one bit. The receiver observes $y = hx + n$, where h is the channel gain and n is noise, typically $n \sim \mathcal{N}(0, \sigma^2)$ per real dimension (AWGN).

Hard vs soft decisions. A hard demodulator outputs the single most likely bit. A soft demodulator outputs a log-likelihood ratio (LLR):

$$L(y) = \log \frac{P(y | x = +1)}{P(y | x = -1)}, \quad (1)$$

whose sign is the hard decision and whose magnitude is the confidence. For BPSK in AWGN with $h = 1$,

$$L(y) = \log \frac{e^{-(y-1)^2/2\sigma^2}}{e^{-(y+1)^2/2\sigma^2}} = \frac{2y}{\sigma^2}.$$

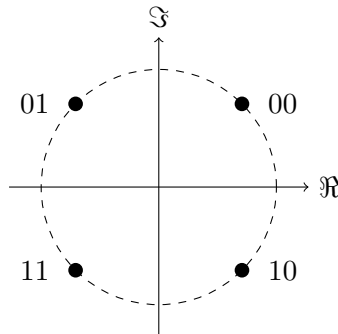
Worked example: $\sigma^2 = 0.5$, received $y = 0.7$. Then $L = 2(0.7)/0.5 = 2.8$, and $P(x = +1 | y) = 1/(1 + e^{-2.8}) \approx 0.943$. A hard decision would report only “bit 0”; the LLR additionally reports “94% sure” — and a competing observation $y = 0.05$ gives $L = 0.2$, i.e. “55% sure,” which a good decoder treats almost as an erasure.

Why soft information helps. Every iterative decoder in Part IV (LDPC belief propagation, turbo decoding, even soft-output Viterbi) consumes LLRs, not bits: weak observations are down-weighted rather than trusted. Quantizing LLRs to ± 1 (hard decisions) costs roughly 2 dB of SNR on the AWGN channel — a permanent, avoidable tax. In a Viterbi decoder the LLRs enter directly as branch metrics: for hypothesized bits $\hat{b}_k \in \{\pm 1\}$ the branch metric is $-\sum_k \hat{b}_k L_k/2$, so confident observations dominate the path decision automatically.

QPSK. Two bits per symbol, Gray-mapped so neighbors differ in one bit:

bits $b_1 b_2$	symbol x
00	$(+1 + j)/\sqrt{2}$
01	$(-1 + j)/\sqrt{2}$
11	$(-1 - j)/\sqrt{2}$
10	$(+1 - j)/\sqrt{2}$

Each bit gets its own LLR by marginalizing over the symbols (the max-log approximation keeps the two nearest constellation points). The same machinery scales to 16-QAM and beyond; only the constellation changes.



QPSK constellation: Gray mapping, unit energy ($|x| = 1$).

Fading and bursts. In wireless links the gain h varies with time (multipath fading). When $|h|$ is large the demodulator is confident and correct; when a fade drives $|h| \approx 0$ the LLRs collapse toward zero and errors arrive in *runs*. This is the physical origin of burst noise, and the reason Part II models the channel as a Markov chain over quality states rather than a memoryless BSC: the hidden state is, physically, the fading envelope.

2 Supplement S2: Fitting burst models to measured data

A channel model is only useful if its parameters match the real link. Given a measured binary error trace $o_1, \dots, o_T$ (1 = error), we fit a hidden Markov model with states $\{G, B\}$ (good/bad), transition matrix $A = (a_{ij})$, emission probabilities $b_j(o)$, and initial distribution π .

Method of moments (quick sketch for Gilbert–Elliott). Match model moments to empirical ones: the mean error rate fixes $\pi_B \varepsilon_B + \pi_G \varepsilon_G$ with stationary $\pi_B = a_{GB}/(a_{GB} + a_{BG})$; the lag-1 autocorrelation of the error process fixes the memory $1 - a_{GB} - a_{BG}$; the error-free run distribution (geometric with mean $1/a_{GB}$) fixes the remaining degree of freedom. Fast and transparent, but statistically inefficient.

Baum–Welch (EM for HMMs). The principled fit. E-step, forward quantities $\alpha_t(i) = P(o_1 \dots o_t, s_t = i)$:

$$\alpha_1(i) = \pi_i b_i(o_1), \quad \alpha_{t+1}(j) = b_j(o_{t+1}) \sum_i \alpha_t(i) a_{ij}, \quad (2)$$

and backward quantities $\beta_T(i) = 1$,

$$\beta_t(i) = \sum_j a_{ij} b_j(o_{t+1}) \beta_{t+1}(j). \quad (3)$$

State posteriors $\gamma_t(i) = \alpha_t(i) \beta_t(i) / \sum_k \alpha_t(k) \beta_t(k)$ and pair posteriors

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(o_1 \dots o_T)}.$$

M-step re-estimation:

$$\pi_i = \gamma_1(i), \quad a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \quad b_j(k) = \frac{\sum_{t: o_t=k} \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)}. \quad (4)$$

Iterate E/M to convergence. **Pitfalls:** EM finds local maxima — run from several initializations; states are only identified up to label permutation (“label switching”), so pin e.g. $\varepsilon_G < \varepsilon_B$; scale α_t, β_t by $c_t = \sum_i \alpha_t(i)$ each step to avoid underflow, i.e. work with $\bar{\alpha}_t(i) = \alpha_t(i)/c_t$ (then $\log P(o_1 \dots o_T) = \sum_t \log c_t$ comes for free as the log-likelihood); short traces cannot resolve slow state dynamics.

A validation sketch. From a fitted GE model with $a_{GB} = 0.02$ the error-free runs are geometric with mean $1/0.02 = 50$. Bin the measured error-free runs into a histogram P and the geometric prediction into Q over the same bins; the fit quality is $D_{\text{KL}}(P||Q) = \sum_k P_k \log(P_k/Q_k)$. Values $\lesssim 0.05$ nats indicate the two-state memory captures the burst structure; a heavy empirical tail (large KL) means the channel has longer memory than one bad state can express.

Validating the fit. Compute the model’s stationary distribution $\pi^* = \pi^* A$ and check it against the empirical state occupancy; compare the empirical error-free/burst *run-length* histograms against the model’s geometric predictions via KL divergence. If the tail disagrees, the two-state model is too coarse — add states or move to the richer models of Part II.

3 Supplement S3: Statistics for experiments

A rate–distortion point measured once is anecdote. Report intervals.

Confidence interval for a mean. From n runs with sample mean $\bar{x}$ and sample standard deviation s ,

$$\bar{x} \pm t_{n-1, 1-\alpha/2} \frac{s}{\sqrt{n}}, \quad (5)$$

with t the Student quantile. *Worked example:* five PSNR runs 30.1, 30.4, 29.8, 30.2, 30.0 give $\bar{x} = 30.1$, $s \approx 0.224$; $t_{4,0.975} = 2.776$, so the 95% interval is 30.1 ± 0.28 dB:

$$\bar{x} \pm t_{4,0.975} \frac{s}{\sqrt{5}} = 30.1 \pm 2.776 \cdot \frac{0.224}{\sqrt{5}} = 30.1 \pm 0.28.$$

Comparing two systems. Use *paired* differences: run both systems on the same seeds and the same images, form $d_i = x_i - y_i$, and interval the d_i . Pairing removes image-to-image variance and is far more sensitive than two independent intervals. *Worked example:* system A PSNRs 30.1, 31.2, 29.5 vs system B 29.8, 30.9, 29.6 give differences $d = (0.3, 0.3, -0.1)$, $\bar{d} = 0.167$, $s_d = 0.231$; with $t_{2,0.975} = 4.303$ the 95% interval is $0.167 \pm 4.303 \cdot 0.231/\sqrt{3} = 0.167 \pm 0.574$, i.e. $(-0.41, 0.74)$. It contains zero: three images cannot declare A better, however suggestive the means look.

Seeds and error bars. Run every experiment on at least 3 random seeds (channel realizations, initializations) and put error bars on every RD curve. A “gain” smaller than the error bars is not a gain.

Multiple comparisons. Testing many variants and reporting the best inflates false wins: with m comparisons at level α , use Bonferroni α/m per test, or accept the finding as exploratory and confirm it on fresh data.

How many runs? (sample-size sketch.) To detect a true mean difference Δ with noise std σ at level α and power $1 - \beta$ (paired design),

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2}. \quad (6)$$

E.g. $\sigma = 0.5$ dB, $\Delta = 0.3$ dB, $\alpha = 0.05$, $1 - \beta = 0.8$ ($z_{0.975} = 1.96$, $z_{0.8} = 0.84$): $n \approx 2(2.8)^2(0.25)/0.09 \approx 44$ paired runs. Sobering, and exactly why published RD comparisons use dozens of images.

4 Supplement S4: Deployment tradeoffs

The project description stresses delay, power, complexity, and scalability for sensor networks. A classifier that needs a GPU server is not a sensor-network solution; the standard compression toolkit is:

Quantization (int8). Replace float weights/activations by $r \approx S(q - Z)$ with integer $q \in [0, 255]$, calibrated scale S and zero-point Z :

$$q = \text{clamp}(\text{round}(r/S) + Z, 0, 255), \quad \hat{r} = S(q - Z). \quad (7)$$

Typically $\sim 4\times$ smaller and $2\text{--}4\times$ faster on integer hardware, at $\lesssim 1\%$ accuracy cost with calibration (quantization-aware training recovers most of the rest). *Worked example:* weight $r = 0.37$ with calibrated $S = 0.0235$, $Z = 128$: $q = \text{round}(0.37/0.0235) + 128 = 16 + 128 = 144$, $\hat{r} = 0.0235 \times 16 = 0.376$, quantization error 0.006 — about 1.6% of the value, invisible next to channel noise.

Pruning. Delete weights with smallest magnitude (or by a learned saliency) and fine-tune; 50–90% sparsity is routine in CNNs before accuracy moves.

Knowledge distillation. Train a small student on a large teacher’s soft targets:

$$\mathcal{L} = \alpha \text{CE}(y, \sigma(z_s)) + (1 - \alpha) T^2 \text{KL}(\sigma(z_t/T) \parallel \sigma(z_s/T)),$$

temperature T softening the teacher distribution; the T^2 factor keeps gradient scales matched.

Mapping to the project. Budget the sensor node three ways: *latency* (encoding, transmission, and classification must beat the event timescale — this is the low-delay argument for JSCC from Part II), *energy* (every transmitted bit and every MAC costs battery; int8 and pruning buy headroom), and *memory* (model plus codebook must fit on-device SRAM). Evaluate the classifier of Part VII not only on accuracy versus channel quality but on accuracy-versus-energy and accuracy-versus-latency curves; that is the experiment that answers the project description’s sensor-network question.

Where to find what: topic index

Topic	Part	Location
6.02 dB/bit rule	III	formula companion
Adam optimizer	VII	§4.5
Alternating minimization	I	§10
Arithmetic coding	I	§5
Batch normalization	VII	§6
Baum–Welch / HMM fitting	VIII	S2
BCJR / turbo decoding	IV	turbo section
Belief propagation (LDPC)	IV	LDPC section
Berlekamp–Massey	IV	RS decoding
BPSK / QPSK / LLRs	VIII	S1
Burst channels (Gilbert–Elliott)	II	channel models; VIII S1–S2
Channel capacity, BSC/BEC	I	§3; II
Chien search	IV	RS decoding
Classification metrics (P/R/F1)	I	§13; VII §7
CNNs (convolution, pooling)	VII	§5
Confidence intervals	VIII	S3
COVQ: encoder/decoder conditions	II	COVQ; III derivations
CRC / cyclic codes	IV	§10
Cross-entropy from MLE	VII	§2.3
Deep JSCC (autoencoder)	I	§8; VII §9
Dropout	VII	§6
Erasures / BEC	I	§3; IV 2t+e+1
Error-and-erasure COVQ	II	COVQ erasures
Fading and bursts	VIII	S1
Finite fields $\text{GF}(2^m)$	IV	§§2–3
Forney magnitudes	IV	RS decoding
Fountain / LT / RaptorQ codes	IV	fountain section
Gaussian R(D)	I	§4
Gradient descent / SGD	VII	§4
Hamming (7,4) code	IV	§6
Huffman coding	I	§5; V JPEG
Index assignment	II	COVQ
Int8 quantization (neural nets)	VIII	S4
JPEG pipeline	V	JPEG sections
KL divergence	I	§3; VII §2.3
KLT / DCT / bit allocation	I	§5; II transform coding
Lagrange multipliers	I	§10
LDPC codes	IV	LDPC section
Lloyd–Max conditions	I	§5; III derivations
Log-likelihood ratios	VIII	S1
MMSE / conditional mean	I	§2
Modulation	VIII	S1
Noise-erasure channel + feedback	II	NEC; III capacity
Paired comparison of systems	VIII	S3
PNG pipeline	V	PNG sections
Polar codes	IV	polar section
Pólya channel	II	Pólya; III decoding
Pruning / distillation	VIII	S4
PyTorch training loop	VII	§8
QBC (queue-based channel)	II	QBC
$\Delta^2/12$ rule	III	quantization noise
Rate–distortion function	I	§4
Reed–Solomon codes	IV	RS sections
Residual connections	VII	§5.5
Separation theorem (and its limits)	I	§3; II intro
Soft decisions	VIII	S1; IV iterative decoders
Sphere-packing / Singleton / GV bounds	IV	bounds
Syndrome decoding	IV	block codes
Transfer learning	VII	§7
Viterbi algorithm	IV	convolutional codes
Water-filling	I	§4